

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Trịnh Nguyên Lương - Hoàng Lê Nam -
Trần Thảo Ngân - Nguyễn Thanh Nhã

XÂY DỰNG HỆ THỐNG THÔNG MINH
HỖ TRỢ HỌC TẬP

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng 07/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Trịnh Nguyên Lương - 22120198

Hoàng Lê Nam - 22120217

Trần Thảo Ngân - 22120225

Nguyễn Thanh Nhã - 22120243

**XÂY DỰNG HỆ THỐNG THÔNG MINH
HỖ TRỢ HỌC TẬP**

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN

TS. Nguyễn Thị Minh Tuyền

Tp. Hồ Chí Minh, tháng 07/2026

Lời cam đoan

Nhóm xin cam đoan Dự án “Xây dựng hệ thống thông minh hỗ trợ học tập” là công trình nghiên cứu độc lập của nhóm, với sự hướng dẫn tận tình của Nguyễn Thị Minh Tuyền.

Công trình được nhóm nghiên cứu và hoàn thành tại Trường Đại học Khoa học Tự nhiên từ tháng 10 năm 2025 đến tháng 07 năm 2026. Các tài liệu tham khảo, các số liệu thống kê phục vụ mục đích nghiên cứu công trình này được sử dụng đúng quy định, không vi phạm quy chế bảo mật của Nhà nước.

Những số liệu, bảng biểu phục vụ cho việc phân tích và dẫn dắt đề tài này được thu thập từ các nguồn tài liệu khác nhau được trích dẫn một cách đầy đủ, đồng thời ghi rõ ràng về nguồn gốc theo quy định của nhà trường.

Ngoài ra, đối với các tài liệu diễn giải để làm rõ thêm các luận điểm đã phân tích và trích dẫn trong phần phụ lục cũng được chú thích nguồn gốc dữ liệu. Nếu những lời cam đoan trên của nhóm không chính xác, nhóm xin chịu hoàn toàn trách nhiệm và chịu mọi hình thức kỷ luật từ khoa và nhà trường

Thuyết minh chỉnh sửa đề tài

Bản thuyết minh chỉnh sửa đề tài (nếu có thay đổi tên đề tài, nội dung báo cáo, ... trong cuốn báo cáo sau bảo vệ)

Nhận xét hướng dẫn

Bản nhận xét của giảng viên hướng dẫn (có chữ ký) do giáo vụ cung cấp.

Nhận xét phản biện

Bản nhận xét của giảng viên phản biện (có chữ ký) do giáo vụ cung cấp.

Lời cảm ơn

Dự án thực tập tốt nghiệp chuyên ngành Kỹ Thuật Phần Mềm "Xây dựng hệ thống thông minh hỗ trợ học tập" đánh dấu cột mốc hoàn thành.

Chúng em xin gửi lời cảm ơn chân thành đến Quý Ban Giám hiệu và toàn thể cán bộ giảng viên của Trường Đại học Khoa học Tự nhiên - Đại học Quốc gia TP.HCM, vì đã cung cấp cho chúng em môi trường học tập và nghiên cứu tuyệt vời trong suốt khoảng thời gian học tập và làm việc tại nhà trường.

Và chúng em xin chân thành cảm ơn sâu sắc dành cho cô Nguyễn Thị Minh Tuyền vì sự tận tâm, kiến thức sâu rộng và sự hỗ trợ quý báu trong suốt quá trình thực hiện đồ án tốt nghiệp. Cô đã dành thời gian và nỗ lực để chỉ dẫn, định hướng và đánh giá công việc của chúng em. Sự tận tâm và kiến thức chuyên môn của cô đã giúp chúng em vượt qua những khó khăn và phát triển kỹ năng nghiên cứu và thực hiện đồ án của mình.

Mặc dù chúng em đã cố gắng hoàn thành dự án thực tập tốt nghiệp với kết quả tốt nhất có thể trong khả năng, nhưng vẫn có những khiếm khuyết, sai sót nhất định do thiếu kinh nghiệm thực tế. Chúng em mong nhận được sự góp ý chân thành và dạy bảo của quý Thầy Cô để có thể hoàn thiện bản thân hơn nữa.

Lời cảm ơn cuối cùng dành cho gia đình, bạn bè và những người thân yêu đã luôn động viên và đồng hành cùng chúng em trong suốt quá trình nghiên cứu và thực hiện đồ án tốt nghiệp. Sự ủng hộ và động viên của họ là nguồn động lực vô cùng quan trọng giúp chúng em vượt qua khó khăn và hoàn thành công việc một cách tốt nhất.

Đề cương chi tiết

Xây dựng hệ thống thông minh hỗ trợ học tập

Building an Intelligent Learning Support System

1. Thông tin chung

Người hướng dẫn:

- TS. Nguyễn Thị Minh Tuyền (Khoa Công nghệ Thông tin)

Nhóm sinh viên thực hiện:

1. Trịnh Nguyên Lương (MSSV: 22120198)
2. Hoàng Lê Nam (MSSV: 22120217)
3. Trần Thảo Ngân (MSSV: 22120225)
4. Nguyễn Thanh Nhã (MSSV: 22120243)

Loại đề tài: Ứng dụng

Thời gian thực hiện: 10/2025 – 07/2026

2. Nội dung thực hiện

2.1 Giới thiệu về đề tài

Trong những năm gần đây, cùng với sự phát triển của công nghệ thông tin và Internet, các nền tảng học tập trực tuyến ngày càng trở nên phổ biến và đóng vai trò quan trọng trong việc hỗ trợ quá trình tự học của người học. Nhiều công cụ và hệ thống học tập đã được phát triển nhằm giúp người học tiếp cận tài liệu nhanh chóng, quản lý kiến thức và nâng cao hiệu quả học tập. Tuy nhiên, sự phong phú của các nguồn tài nguyên trên Internet cũng đặt ra nhiều thách thức trong việc tổ chức và khai thác thông tin một cách hiệu quả.

Hiện nay, tài nguyên học tập thường được phân tán trên nhiều nền tảng khác nhau, khiến người học gặp khó khăn trong việc tổng hợp và quản lý kiến thức. Bên cạnh đó, việc xây dựng lộ trình học tập phù hợp, duy trì động lực học tập và ghi nhớ kiến thức trong thời gian dài vẫn là những vấn đề phổ biến. Mặc dù các công nghệ trí tuệ nhân tạo (AI) đang phát triển mạnh mẽ, việc ứng dụng AI để hỗ trợ cá nhân hóa quá trình học tập và tối ưu hiệu quả ghi nhớ vẫn chưa được khai thác đầy đủ trong nhiều hệ thống học tập hiện nay.

Nhằm giải quyết những vấn đề trên, nhóm đề xuất xây dựng một hệ thống học tập tích hợp ứng dụng trí tuệ nhân tạo nhằm hỗ trợ người học quản lý tài liệu, tổ chức kiến thức và hỗ trợ quá trình tự học. Hệ thống hướng tới việc cá nhân hóa quá trình học tập và hỗ trợ ôn tập kiến thức thông qua các công cụ học tập hiện đại. Ngoài ra, nhóm tham khảo một số nền tảng như NotebookLM để định hướng phát triển, hướng tới xây dựng một hệ thống hỗ trợ học tập tích hợp và thuận tiện hơn cho người học.

2.2 Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng một hệ thống học tập tích hợp ứng dụng trí tuệ nhân tạo nhằm hỗ trợ người học quản lý tài liệu, tổ chức kiến thức và nâng cao hiệu quả tự học.

Thực hiện đề tài này, nhóm mong muốn mang lại các giá trị thực tế sau:

- Cung cấp nền tảng học tập tích hợp, hỗ trợ tổ chức và quản lý tài nguyên học tập.
- Ứng dụng AI giúp tiết kiệm thời gian thông qua tóm tắt, luyện tập và ôn tập.
- Cá nhân hóa lộ trình học tập, nâng cao hiệu quả và động lực học.
- Xây dựng cộng đồng học tập trực tuyến, thúc đẩy chia sẻ và hỗ trợ lẫn nhau.

Thông qua ứng dụng này, người học có thể cải thiện hiệu quả học tập, giảm thiểu tối đa thời gian tìm kiếm và tổ chức tài nguyên, kết nối cộng đồng và tăng động lực học tập.

2.3 Phạm vi của đề tài

Hệ thống bao gồm các thực thể chính:

- **Người học:** Đối tượng trung tâm sử dụng hệ thống để ghi chép, học tập và củng cố kiến thức.
- **Tài liệu học tập:** Bao gồm nhiều dạng nội dung như ghi chú (Notes), thẻ ghi nhớ (Flashcards), bài kiểm tra (Exams).
- **Mô hình AI (Agent/LLM):** Đóng vai trò trợ lý thông minh hỗ trợ phân tích tài liệu, tóm tắt nội dung và đề xuất phương pháp học tập.

Hệ thống cung cấp các nhóm chức năng chính:

- **Note:** Cho phép người dùng ghi chú thủ công khi đọc tài liệu (PDF, DOCX, ...) và sử dụng AI để giải thích hoặc tóm tắt nội dung.
- **Flashcards:** Tạo bộ thẻ ghi nhớ thủ công hoặc tự động từ ghi chú và tài liệu với các options cho người dùng custom khi tạo, đồng thời cũng đề xuất cho người dùng ôn luyện.
- **Exam:** Tạo bài kiểm tra từ ghi chú và tài liệu với nhiều dạng câu hỏi cùng với khả năng phân tích kết quả bài làm để xác định lỗ hổng kiến thức.
- Đề xuất lộ trình học từ tài liệu có sẵn.

Hệ thống được triển khai trên hai nền tảng là Web và Mobile. Thiết kế này giúp người dùng có thể học tập linh hoạt và tiện lợi trên nhiều thiết bị.

Đề tài áp dụng phương pháp tiếp cận kết hợp giữa phân tích các hệ thống hiện có và phát triển phần mềm theo quy trình hiện đại, nhằm đảm bảo tính khoa học và khả năng ứng dụng thực tiễn.

Dự án sử dụng mô hình Agile/Scrum với các Sprint khoảng 2 tuần để phát triển và cải tiến hệ thống.

Công nghệ sử dụng:

- **Frontend:** React (Web) và React Native (Mobile)
- **Backend:** Java Spring Framework xử lý nghiệp vụ và quản lý dữ liệu
- **AI Service:** Python FastAPI kết hợp LangChain để kết nối các mô hình ngôn ngữ lớn (LLM)

2.4 Kế hoạch thực hiện

1. Version 1: Chức năng học tập chính (Note, Flashcard và Exam)

Giai đoạn	Sprint (2 tuần)	Nội dung thực hiện
06/10/25 – 26/10/25	Sprint 01: Chuẩn bị đề tài và phân tích yêu cầu	Phân tích yêu cầu hệ thống, xây dựng Use Case, thiết kế giao diện Figma cho Mobile và Web
27/10/25 – 09/11/25	Sprint 02: Setup	Chuẩn bị Code Base cho AI Service, Backend, Frontend (Mobile & Web) và thiết lập DevOps
10/11/25 – 30/11/25	Sprint 03: Authentication và cơ bản chức năng Note	Xây dựng API đăng nhập, đăng ký và API cho chức năng Note; phát triển giao diện Dashboard, Set và Note
01/12/25 – 21/12/25	Sprint 04 - Hoàn thiện chức năng Note	Hoàn thiện API cho Note, thêm chức năng Explain Note và Summarize file, hỗ trợ tạo Flashcard thủ công
22/12/25 – 11/01/26	Sprint 05 - Hoàn thiện chức năng Flashcard	Phát triển Flashcard bằng AI, xây dựng giao diện Flashcard và chức năng Game trong Flashcard
21/02/26 – 14/03/26	Sprint 06 - Hoàn thiện chức năng Exam	Phát triển chức năng tạo Exam bằng AI và thủ công, xây dựng giao diện Exam
15/03/26 – 04/04/26	Sprint 07 - Nâng cấp 3 chức năng Note, Flashcard và Exam	Đồng bộ ghi chú Real-time giữa Mobile và Web, bổ sung options tạo Flashcard/Exam bằng AI, giải thích câu sai và sinh Exam ôn luyện, thêm Bookmark và Search
05/04/26 – 18/04/26	Sprint 08 - Share Note, Flashcard and Exam	Refactor AI API, xây dựng trang Admin, xử lý quyền truy cập khi chia sẻ tài nguyên và bảng xếp hạng

Giai đoạn	Sprint (2 tuần)	Nội dung thực hiện
19/04/26 – 02/05/26	Sprint 09 - Xây dựng trang Admin để quản lý hệ thống	

Bảng P.1: Kế hoạch thực hiện Version 1

2. Version 2: Chức năng học tập hỗ trợ (To Do List và Pomodoro)

Giai đoạn	Sprint (2 tuần)	Nội dung thực hiện
03/05/26 – 23/05/26	Sprint 10: Chức năng To do và Pomodoro	Phát triển chức năng To Do List và Pomodoro, tích hợp API và bổ sung hệ thống Streak theo dõi quá trình học
24/05/26 – 13/06/26	Sprint 11: Tạo lộ trình học	Phát triển và hoàn thiện giao diện cho chức năng tạo lộ trình học
Thời gian còn lại	Sprint 12: Kiểm thử, Fix bug và Nâng cấp chức năng	Kiểm thử toàn bộ hệ thống, sửa lỗi phát sinh và nâng cấp, tối ưu các chức năng trước khi hoàn thiện sản phẩm

Bảng P.2: Kế hoạch thực hiện Version 2

Mục lục

Lời cam đoan	i
Thuyết minh chỉnh sửa đề tài	ii
Nhận xét của GV hướng dẫn	iii
Nhận xét của GV phản biện	iv
Lời cảm ơn	v
Đề cương	vi
Mục lục	xii
Danh sách hình	xvi
Danh sách bảng	xviii
Tóm tắt	xix
1 Giới thiệu	1
1.1 Bối cảnh thực hiện đề tài	1
1.2 Mục tiêu đề tài	2
1.2.1 Mục tiêu tổng quát	3
1.2.2 Mục tiêu chức năng	3
1.2.3 Mục tiêu kỹ thuật	4

1.2.4	Đối tượng sử dụng	4
1.2.5	Đóng góp của đề tài	5
1.3	Phạm vi đề tài	6
1.3.1	Phạm vi nghiên cứu	6
1.3.2	Phạm vi chức năng	6
1.3.3	Phạm vi kỹ thuật	8
1.3.4	Giới hạn của đề tài	9
1.4	Phương pháp thực hiện	9
1.5	Cấu trúc báo cáo	10
2	Yêu cầu hệ thống	11
2.1	Các công trình liên quan	11
2.1.1	NotebookLM	11
2.1.2	Quizlet	15
2.1.3	Notion	18
2.2	So sánh tổng hợp các hệ thống liên quan	22
2.3	Những vấn đề còn tồn tại	22
2.3.1	Sự phân mảnh của các công cụ học tập	23
2.3.2	Thiếu sự liên kết giữa các giai đoạn học tập	24
2.3.3	Hạn chế trong việc theo dõi tiến trình học tập	25
2.3.4	Hạn chế trong việc cá nhân hóa trải nghiệm học tập	26
2.3.5	Khó khăn trong việc duy trì động lực học tập	27
2.3.6	Vai trò của trí tuệ nhân tạo vẫn chưa được khai thác toàn diện	27
2.4	Yêu cầu đặt ra đối với hệ thống đề xuất	28
2.5	Kết luận chương	40
3	Thiết kế và cài đặt	42
3.1	Kiến trúc tổng quan	42
3.1.1	Mô hình kiến trúc hệ thống	42
3.2	Tổ chức thành phần hệ thống	44
3.3	Thiết kế thành phần ứng dụng	46

3.3.1	Lớp giao diện người dùng	47
3.3.2	Lớp ứng dụng phía máy chủ	48
3.3.3	Lớp dữ liệu và hạ tầng	49
3.3.4	Tích hợp dịch vụ ngoài	50
3.4	Cài đặt hệ thống	51
3.4.1	Cài đặt Client	51
3.4.2	Đặc thù triển khai: di động so với web	59
3.4.3	Cài đặt backend	62
3.4.4	Cài đặt các core component tiêu biểu	65
3.4.5	Cài đặt cơ sở dữ liệu và lưu trữ trạng thái	69
3.4.6	Các vấn đề cần xử lý và giải pháp	73
3.4.7	Tổng kết cài đặt hệ thống	73
3.5	Tổng kết	75
4	Quản trị dự án	76
4.1	Công cụ sử dụng	76
4.2	Các giai đoạn của đề tài	77
4.2.1	Giai đoạn 1 – Chuẩn bị ý tưởng và xây dựng nền tảng (Tháng 9 – 12/2025)	77
4.2.2	Giai đoạn 2 – Phát triển, hoàn thiện và kiểm thử (Tháng 2 – 7/2026)	78
4.3	Quá trình làm việc nhóm	79
4.3.1	Vai trò của các thành viên	79
4.3.2	Phân chia công việc và quản lý mã nguồn	79
4.3.3	Lịch họp và theo dõi tiến độ	81
4.4	Những khó khăn gặp phải	81
4.5	Demo và đánh giá ứng dụng	82
4.5.1	Đánh giá chức năng hệ thống	88
4.6	Kiểm thử	90
4.6.1	Mục tiêu kiểm thử	90
4.6.2	Phạm vi kiểm thử	91

4.6.3	Chiến lược kiểm thử	92
4.6.4	Kiểm thử tích hợp với dịch vụ ngoài	92
4.6.5	Kết quả kiểm thử	93
4.6.6	Đánh giá sau kiểm thử	93
5	Kết luận và hướng phát triển	95
5.1	Triển khai thực nghiệm và Mô hình tài chính	95
5.2	Đánh giá kết quả đề tài	95
5.3	Kiến thức và kinh nghiệm đạt được	96
5.3.1	Về mặt kỹ thuật	96
5.3.2	Về kỹ năng mềm	97
5.4	Những vấn đề còn tồn đọng	97
5.5	Hướng phát triển trong tương lai	98
	Tài liệu tham khảo	100

Danh sách hình

2.1	Use Case: Đăng ký và Đăng nhập	29
2.2	Use Case: Quản lý hồ sơ người dùng	30
2.3	Use Case: Quản trị hệ thống	31
2.4	Use Case: Quản lý không gian học tập (Set)	32
2.5	Use Case: Ghi chú kiến thức	32
2.6	Use Case: Học tập với Flashcard	33
2.7	Use Case: Kiểm tra và đánh giá kiến thức	34
2.8	Use Case: Quản lý mục tiêu và công việc học tập	35
2.9	Use Case: Quản lý thời gian học tập (Pomodoro)	36
2.10	Use Case: Theo dõi và phân tích tiến trình học tập	37
2.11	Use Case: Lộ trình học tập	38
2.12	Use Case: Cộng đồng và khám phá tài nguyên	39
2.13	Use Case: Thông báo	39
3.1	Kiến trúc tổng quan của hệ thống ProLearning Platform .	45
3.2	Chi tiết các thành phần của hệ thống ProLearning Platform	47
3.3	Kiến trúc backend của hệ thống ProLearning Platform . .	63
3.4	Luồng điều phối AI Service	66
3.5	Luồng học và ôn tập flashcard theo SM-2	68
3.6	Luồng cộng tác thời gian thực trên ghi chú	69
3.7	Luồng xử lý notification pipeline	70
3.8	Luồng đồng bộ Todo với Google Calendar	71
3.9	Luồng thanh toán PayOS và nâng cấp tài khoản PRO . . .	72

4.1	Giao diện Web - Ghi chép kết hợp tài liệu	83
4.2	Giao diện Mobile - Flashcard	84
4.3	Giao diện Web - Bài kiểm tra	85
4.4	Giao diện Mobile - Lộ trình học tập	86
4.5	Giao diện Web - Phân tích năng lực người học	87
4.6	Giao diện Mobile - Quản lý Task	88
4.7	Giao diện Mobile - Pomodoro	88

Danh sách bảng

P.1	Kế hoạch thực hiện Version 1	xi
P.2	Kế hoạch thực hiện Version 2	xi
2.1	So sánh tổng hợp các hệ thống liên quan	23
3.1	Tổ chức các thành phần nghiệp vụ chính	44
3.2	Tổ chức các thành phần hỗ trợ và hạ tầng	46
3.3	Thiết kế lớp giao diện người dùng	47
3.4	Thiết kế các nhóm chức năng backend	48
3.5	Thiết kế lớp dữ liệu và hạ tầng	49
3.6	Thiết kế tích hợp dịch vụ ngoài	50
3.7	Đối chiếu triển khai di động (React Native/Expo) và web (React/Vite)	60
3.8	Tóm tắt tổ chức các lớp và thành phần backend	64
3.9	Các vấn đề triển khai và giải pháp xử lý	74
4.1	Phân công vai trò các thành viên trong nhóm	80
4.2	Lịch họp nhóm theo từng giai đoạn	81
4.3	Khó khăn và giải pháp trong quá trình thực hiện đề tài . .	82
4.4	Đánh giá chức năng hệ thống theo nhóm, đối chiếu kiểm thử nội bộ và khảo sát người dùng	89
4.5	Phạm vi kiểm thử hệ thống	91
4.6	Tổng hợp kết quả kiểm thử	93
4.7	Đánh giá sau kiểm thử	94

Tóm tắt

Trong bối cảnh học tập hiện đại, người học thường phải sử dụng đồng thời nhiều công cụ rời rạc để ghi chú, ôn tập, quản lý thời gian và tra cứu kiến thức, khiến dữ liệu học tập bị phân tán, khó liên kết và khó duy trì thói quen ôn tập chủ động. Các nền tảng hiện có phần lớn chỉ giải quyết tốt một nhu cầu riêng lẻ, chưa hỗ trợ toàn diện cả quá trình học từ tiếp nhận, tổ chức đến ôn tập và đánh giá kiến thức. Vì vậy, việc xây dựng một hệ thống học tập tích hợp ứng dụng trí tuệ nhân tạo là cần thiết nhằm hỗ trợ người học quản lý và nâng cao hiệu quả tự học một cách chủ động.

Mục tiêu của đề tài là phát triển một hệ thống học tập thông minh đa nền tảng, cho phép người dùng quản lý tài liệu, ghi chú, Flashcard, bài kiểm tra, mục tiêu và thời gian học tập trong cùng một môi trường thống nhất.

Đề tài kết hợp mô hình ngôn ngữ lớn (LLM) với các agent chuyên biệt đảm nhiệm từng tác vụ học tập như tóm tắt tài liệu, sinh Flashcard, sinh bài kiểm tra và đề xuất lộ trình học tập cá nhân hóa. Việc ôn tập được cá nhân hóa theo thuật toán lặp cách quãng dựa trên mức độ ghi nhớ của từng người học, trong khi tính năng cộng tác ghi chú cho phép nhiều người cùng chỉnh sửa và đồng bộ nội dung theo thời gian thực. Hệ thống được đánh giá thông qua kiểm thử chức năng toàn diện và triển khai thử nghiệm thực tế với người dùng.

Kết quả đạt được là một hệ thống phần mềm hoàn chỉnh ProLearning, gồm ứng dụng Web và ứng dụng Mobile, đã được triển khai và đưa vào sử dụng thực tế, hỗ trợ đầy đủ các chức năng ghi chú, Flashcard, bài kiểm

tra, quản lý mục tiêu và thời gian học tập. Kết quả kiểm thử cho thấy hệ thống hoạt động ổn định và đáp ứng tốt các mục tiêu đề ra, đồng thời cho thấy tiềm năng ứng dụng thực tiễn của trí tuệ nhân tạo trong việc hỗ trợ học tập cá nhân hóa.

Chương 1

Giới thiệu

1.1 Bối cảnh thực hiện đề tài

Trong những năm gần đây, sự phát triển của công nghệ thông tin, Internet và các nền tảng học tập số đã làm thay đổi đáng kể cách người học tiếp cận tri thức. Người học có thể dễ dàng sử dụng kho học liệu mở, khóa học trực tuyến, công cụ ghi chú, ứng dụng quản lý thời gian và các hệ thống trí tuệ nhân tạo để hỗ trợ quá trình tự học. Xu hướng này thúc đẩy mô hình học tập suốt đời (Lifelong Learning), trong đó người học chủ động lựa chọn tài liệu, xây dựng lộ trình và học tập linh hoạt theo nhu cầu cá nhân.

Tuy nhiên, việc sử dụng đồng thời nhiều công cụ cũng tạo ra sự phân tán trong quá trình học tập. Tài liệu có thể được lưu trên Google Drive hoặc OneDrive, ghi chú được quản lý bằng Notion hoặc OneNote, Flashcard được tạo trên Quizlet, thời gian học được theo dõi bằng ứng dụng Pomodoro, còn việc giải thích hoặc tóm tắt kiến thức lại được thực hiện bằng các công cụ như ChatGPT hoặc NotebookLM. Việc liên tục chuyển đổi giữa nhiều nền tảng làm tăng chi phí quản lý thông tin, khiến dữ liệu học tập khó liên kết và làm cho quá trình học trở nên rời rạc.

Bên cạnh đó, người học còn gặp khó khăn trong việc ghi nhớ, ôn tập và tự đánh giá mức độ hiểu biết. Nhiều nghiên cứu cho thấy học tập chủ

động thông qua trả lời câu hỏi, tự kiểm tra hoặc luyện tập thường mang lại hiệu quả ghi nhớ tốt hơn so với việc đọc lại tài liệu một cách thụ động [1], [2]. Tuy vậy, để áp dụng các phương pháp này, người học cần tự xây dựng nội dung ôn tập, duy trì thói quen học đều đặn và thường xuyên xác định các phần kiến thức còn yếu. Đây là thách thức lớn, đặc biệt khi các hoạt động học tập không được kết nối trong cùng một quy trình.

Sự phát triển của các mô hình ngôn ngữ lớn (Large Language Models – LLM) mở ra cơ hội xây dựng các hệ thống hỗ trợ học tập thông minh. Các hệ thống này có thể tóm tắt tài liệu, giải thích khái niệm, sinh nội dung học tập, hỗ trợ luyện tập và cá nhân hóa trải nghiệm người dùng. Tuy nhiên, khảo sát các nền tảng hiện có cho thấy phần lớn giải pháp chỉ tập trung vào một nhóm nhu cầu riêng lẻ: Notion mạnh về ghi chú, Quizlet hỗ trợ Flashcard, NotebookLM khai thác tài liệu bằng AI, trong khi các công cụ Pomodoro hoặc Todo hỗ trợ quản lý thời gian và công việc. Các nền tảng này chưa hỗ trợ đầy đủ toàn bộ quy trình học tập trên một môi trường thống nhất.

Từ những vấn đề trên, nhóm nhận thấy nhu cầu về việc hỗ trợ người học quản lý toàn bộ quá trình học tập trên cùng một nền tảng tích hợp. Xuất phát từ nhu cầu thực tế đó, nhóm thực hiện đề tài **“Hệ thống thông minh hỗ trợ học tập”** với mục tiêu xây dựng một nền tảng cho phép người dùng quản lý tài nguyên học tập, ghi chú kiến thức, tạo nội dung học tập bằng trí tuệ nhân tạo, ôn tập, đánh giá và theo dõi tiến trình trên nhiều nền tảng khác nhau. Hệ thống hướng tới việc giảm thao tác thủ công, nâng cao hiệu quả tự học và hỗ trợ người dùng xây dựng thói quen học tập bền vững.

1.2 Mục tiêu đề tài

Trên cơ sở các vấn đề đã phân tích, đề tài tập trung xây dựng một hệ thống học tập thông minh có khả năng kết nối các hoạt động học tập rời rạc thành một quy trình thống nhất. Phần này trình bày mục tiêu tổng

quát, mục tiêu chức năng, mục tiêu kỹ thuật, nhóm người dùng hướng đến và các lợi ích kỳ vọng của hệ thống.

1.2.1 Mục tiêu tổng quát

Mục tiêu của đề tài là xây dựng một hệ thống hỗ trợ học tập thông minh, cho phép người dùng quản lý tài liệu, ghi chú, Flashcard, bài kiểm tra, mục tiêu học tập và tiến trình học tập trên một nền tảng thống nhất. Hệ thống ứng dụng trí tuệ nhân tạo để hỗ trợ tạo nội dung học tập, tóm tắt và khai thác tài liệu, tự động hóa một số tác vụ chuẩn bị nội dung ôn tập và nâng cao hiệu quả tự học.

Thông qua việc liên kết các hoạt động học tập vốn thường được thực hiện rời rạc, hệ thống giúp người dùng giảm thời gian quản lý công cụ, tăng khả năng tổ chức tri thức cá nhân, chủ động luyện tập và theo dõi mức độ tiến bộ của bản thân. Bên cạnh việc đáp ứng nhu cầu học tập hiện tại, đề tài cũng đặt mục tiêu xây dựng nền tảng có khả năng mở rộng, tạo cơ sở cho việc tích hợp thêm các giải pháp trí tuệ nhân tạo và cá nhân hóa học tập trong tương lai.

1.2.2 Mục tiêu chức năng

Về mặt chức năng, hệ thống cần hỗ trợ người học trong các hoạt động chính của quá trình tự học, từ tổ chức tài nguyên, ghi chú kiến thức, tạo nội dung ôn tập, luyện tập, đánh giá kết quả đến theo dõi tiến trình học tập. Các chức năng chính gồm:

- Cung cấp không gian học tập tập trung để quản lý tài nguyên, ghi chú và nội dung học tập theo từng chủ đề hoặc môn học.
- Hỗ trợ tạo, lưu trữ, chia sẻ và khai thác ghi chú từ nhiều nguồn dữ liệu học tập.

- Ứng dụng LLM để tóm tắt tài liệu, giải thích khái niệm, sinh Flashcard, sinh câu hỏi kiểm tra và tạo tài nguyên học tập.
- Hỗ trợ ôn tập và ghi nhớ kiến thức thông qua Flashcard, bài kiểm tra và các hình thức học tập chủ động.
- Ghi nhận kết quả học tập, theo dõi tiến trình và hỗ trợ người dùng nhận biết mức độ tiến bộ của bản thân.
- Cho phép học tập linh hoạt trên nhiều nền tảng, đồng thời bảo đảm dữ liệu được quản lý và đồng bộ thống nhất.

1.2.3 Mục tiêu kỹ thuật

Về mặt kỹ thuật, hệ thống được thiết kế với kiến trúc rõ ràng, dễ mở rộng, dễ bảo trì và phù hợp với yêu cầu thực tế của một ứng dụng học tập hiện đại. Hệ thống được thiết kế theo mô hình Client–Server kết hợp định hướng dịch vụ, trong đó giao diện người dùng, nghiệp vụ, dữ liệu và trí tuệ nhân tạo được phân tách theo trách nhiệm riêng.

Các nghiệp vụ cốt lõi được xử lý tập trung tại Backend Service nhằm bảo đảm tính nhất quán dữ liệu, trong khi Web và Mobile đóng vai trò lớp giao diện phục vụ tương tác với người dùng. Các chức năng trí tuệ nhân tạo được triển khai dưới dạng AI Service độc lập, giúp hệ thống dễ tích hợp, thay đổi hoặc mở rộng mô hình AI trong tương lai. Bên cạnh đó, hệ thống cũng hướng tới khả năng tích hợp với cơ sở dữ liệu, cache, hàng đợi tác vụ và dịch vụ thông báo để đáp ứng yêu cầu về hiệu năng, tính sẵn sàng và khả năng vận hành thực tế.

1.2.4 Đối tượng sử dụng

Đối tượng sử dụng chính của hệ thống là học sinh, sinh viên và những người có nhu cầu tự học trong nhiều lĩnh vực khác nhau. Với học sinh và sinh viên, hệ thống hỗ trợ quản lý tài liệu, tổ chức kiến thức và ôn tập

phục vụ môn học hoặc kỳ thi. Với người tự học hoặc người đi làm, hệ thống hỗ trợ xây dựng kho tri thức cá nhân, quản lý quá trình học tập và phát triển kỹ năng chuyên môn. Ngoài ra, hệ thống cũng có thể được sử dụng trong nhóm học tập hoặc cộng đồng học tập thông qua các chức năng chia sẻ và cộng tác nội dung.

1.2.5 Đóng góp của đề tài

Đề tài có các đóng góp chính sau:

- Đề xuất và xây dựng một nền tảng học tập tích hợp, giúp giảm sự phân tán tài nguyên bằng cách tập trung các hoạt động quản lý kiến thức, ghi chú, ôn tập và đánh giá trên cùng một môi trường duy nhất.
- Ứng dụng mô hình ngôn ngữ lớn (LLM) vào quy trình học tập, tự động hóa các tác vụ chuẩn bị và tổ chức nội dung, giảm đáng kể khối lượng công việc thủ công cho người học.
- Cung cấp cơ chế hỗ trợ học tập chủ động thông qua Flashcard, bài kiểm tra và đánh giá định kỳ, giúp người học chuyển dần từ tiếp thu thụ động sang luyện tập có chủ đích.
- Xây dựng hệ thống theo dõi tiến trình học tập, giúp người dùng nhận biết sự tiến bộ và duy trì động lực trong dài hạn.
- Tạo nền tảng kỹ thuật có khả năng mở rộng, phục vụ cho các hướng nghiên cứu và phát triển tiếp theo về hệ thống học tập cá nhân hóa.

1.3 Phạm vi đề tài

1.3.1 Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và xây dựng một hệ thống hỗ trợ học tập thông minh cho nhiều giai đoạn của quá trình học tập, từ tiếp nhận kiến thức, tổ chức thông tin, ôn tập đến đánh giá kết quả. Trọng tâm của đề tài không nằm ở việc phát triển một mô hình trí tuệ nhân tạo mới, mà là nghiên cứu ứng dụng các mô hình ngôn ngữ lớn (LLM) tiên tiến của Google, kết hợp với khung phát triển ứng dụng AI dạng chuỗi (LangChain) và kiến trúc tác vụ AI đa luồng dạng đồ thị (LangGraph) nhằm giải quyết các bài toán thực tế trong hoạt động học tập cá nhân hóa.

Các hướng nghiên cứu chính của đề tài bao gồm quản lý tri thức cá nhân, học tập chủ động bằng Flashcard và bài kiểm tra, ứng dụng trí tuệ nhân tạo trong giáo dục, theo dõi tiến trình học tập, xây dựng hệ thống đa nền tảng, hỗ trợ cộng tác và tự động hóa quá trình tạo nội dung học tập từ nhiều nguồn dữ liệu.

1.3.2 Phạm vi chức năng

Trong phạm vi đề án, hệ thống tập trung triển khai các nhóm chức năng chính sau. Phần này chỉ trình bày ở mức tổng quát; yêu cầu chi tiết và Use Case được trình bày trong chương phân tích và thiết kế hệ thống.

Quản lý người dùng

Hệ thống hỗ trợ đăng ký, đăng nhập, quản lý hồ sơ cá nhân, xác thực và phân quyền người dùng. Nhóm chức năng này là nền tảng cho việc kiểm soát truy cập và cá nhân hóa dữ liệu học tập.

Quản lý không gian học tập

Hệ thống cho phép người dùng tổ chức nội dung học tập theo từng Set tương ứng với môn học hoặc chủ đề kiến thức. Mỗi Set liên kết các tài nguyên như ghi chú, Flashcard, bài kiểm tra, tài liệu học tập và dữ liệu theo dõi tiến trình.

Hệ thống Note

Hệ thống Note hỗ trợ tạo, chỉnh sửa, quản lý, chia sẻ và cộng tác trên ghi chú học tập. Ngoài ghi chú thủ công, hệ thống còn hỗ trợ khai thác nội dung từ nhiều nguồn dữ liệu để phục vụ quá trình xây dựng tri thức cá nhân.

Hệ thống Flashcard

Hệ thống Flashcard hỗ trợ tạo, quản lý và học Flashcard, bao gồm cả Flashcard do người dùng tạo thủ công và Flashcard được sinh bằng AI. Chức năng này hướng tới việc hỗ trợ ghi nhớ kiến thức thông qua học tập chủ động.

Hệ thống Exam

Hệ thống Exam hỗ trợ tạo và thực hiện bài kiểm tra trực tuyến, lưu kết quả làm bài, thống kê kết quả học tập và phân tích hiệu suất. Thông qua các bài kiểm tra, hệ thống giúp người dùng đánh giá mức độ hiểu bài và xác định nội dung cần ôn tập thêm.

Hệ thống Goal và Todo

Hệ thống Goal và Todo hỗ trợ thiết lập mục tiêu học tập, quản lý công việc, theo dõi mức độ hoàn thành và tổ chức kế hoạch học tập cá nhân. Nhóm chức năng này giúp người dùng duy trì động lực và kỷ luật trong quá trình học tập.

Hệ thống Pomodoro

Hệ thống Pomodoro hỗ trợ quản lý thời gian học tập thông qua việc thiết lập phiên học, theo dõi thời gian học, thời gian nghỉ và lịch sử học tập.

Hệ thống Notification

Hệ thống Notification hỗ trợ gửi các thông báo liên quan đến hoạt động học tập, nhắc nhở ôn tập, tiến độ mục tiêu và các hoạt động cộng tác trên nội dung học tập.

Hệ thống theo dõi học tập

Hệ thống lưu trữ và phân tích dữ liệu học tập nhằm hỗ trợ theo dõi tiến trình, thống kê hoạt động, đánh giá hiệu suất và tạo cơ sở cho việc cá nhân hóa trải nghiệm học tập.

Hệ thống AI hỗ trợ học tập

Hệ thống tích hợp các chức năng trí tuệ nhân tạo nhằm hỗ trợ tóm tắt nội dung, giải thích khái niệm, sinh ghi chú, sinh Flashcard, sinh bài kiểm tra và tạo tài nguyên học tập từ nhiều nguồn dữ liệu như PDF, DOCX, PPT/PPTX, nội dung website, video YouTube và ghi chú đã có trong hệ thống.

1.3.3 Phạm vi kỹ thuật

Về mặt kỹ thuật, hệ thống được xây dựng theo mô hình Client–Server kết hợp với một dịch vụ AI độc lập. Kiến trúc tổng thể bao gồm ứng dụng Web, ứng dụng Mobile, Backend Service, AI Service và hệ quản trị cơ sở dữ liệu.

Ứng dụng Web được phát triển bằng React và TypeScript, ứng dụng Mobile được phát triển bằng React Native, Backend Service được phát

triển bằng Java Spring Boot, còn AI Service được phát triển bằng Python FastAPI và ứng dụng bộ công cụ LangChain và mô hình trạng thái đa luồng LangGraph để xây dựng các quy trình xử lý dữ liệu thông minh, kết nối trực tiếp với các mô hình ngôn ngữ lớn (LLM) tiên tiến. Dữ liệu bền vững được lưu trữ chủ yếu trên PostgreSQL; Redis được sử dụng cho cache và trạng thái ngắn hạn; RabbitMQ hỗ trợ xử lý các tác vụ bất đồng bộ như gửi email hoặc thông báo nền.

1.3.4 Giới hạn của đề tài

Sau khoảng thời gian hơn 6 tháng phát triển, hệ thống vẫn còn một số giới hạn. Thứ nhất, chất lượng đầu ra của các chức năng AI phụ thuộc vào chất lượng dữ liệu đầu vào và khả năng của mô hình được sử dụng. Thứ hai, hệ thống tập trung vào các loại tài liệu học tập phổ biến như văn bản, PDF, bài trình chiếu và nội dung website; các dạng học liệu phức tạp hơn như mô phỏng tương tác hoặc thực tế ảo chưa nằm trong phạm vi nghiên cứu. Thứ ba, hệ thống được tối ưu chủ yếu cho học sinh, sinh viên và người tự học; các nhu cầu đặc thù của doanh nghiệp hoặc tổ chức đào tạo quy mô lớn chưa được xem xét đầy đủ.

1.4 Phương pháp thực hiện

Để thực hiện đề tài, nhóm áp dụng phương pháp phát triển phần mềm theo quy trình Agile Scrum. Cách tiếp cận này giúp tổ chức công việc theo Product Backlog và triển khai linh hoạt theo từng Sprint để liên tục điều chỉnh và phát triển các chức năng.

Quá trình thực hiện đề tài được chia thành các giai đoạn chính như sau:

- **Khảo sát và phân tích yêu cầu:** Khảo sát các hệ thống học tập hiện có và nghiên cứu nhu cầu người học để xác định phạm vi triển khai, xây dựng các yêu cầu chức năng và phi chức năng.

- **Thiết kế hệ thống:** Thiết kế kiến trúc tổng thể, cơ sở dữ liệu, giao diện người dùng và các hệ thống API phục vụ giao tiếp giữa các thành phần.
- **Triển khai hệ thống:** Phát triển song song các thành phần Frontend Web, Frontend Mobile, Backend Service và AI Service theo từng Sprint nhằm rút ngắn thời gian và tăng khả năng phối hợp.
- **Kiểm thử và hoàn thiện:** Tiến hành kiểm thử chức năng sau mỗi giai đoạn để xử lý lỗi phát sinh, đồng thời tận dụng phản hồi từ Sprint Review để cải tiến và hoàn thiện sản phẩm.

1.5 Cấu trúc báo cáo

Báo cáo được tổ chức thành năm chương như sau:

Chương 1 – Giới thiệu: Trình bày bối cảnh hình thành đề tài, mục tiêu nghiên cứu, phạm vi thực hiện và cấu trúc tổng thể của báo cáo.

Chương 2 – Các công trình liên quan: Khảo sát và phân tích các hệ thống hỗ trợ học tập hiện có, từ đó xác định các vấn đề còn tồn tại và các yêu cầu đặt ra đối với hệ thống đề xuất.

Chương 3 – Phân tích, thiết kế và triển khai hệ thống: Trình bày quá trình phân tích yêu cầu, thiết kế kiến trúc, thiết kế cơ sở dữ liệu, thiết kế giao diện và triển khai các thành phần của hệ thống.

Chương 4 – Quá trình thực hiện dự án: Trình bày quy trình quản lý dự án, phương pháp làm việc nhóm, phân công nhiệm vụ, theo dõi tiến độ và các khó khăn gặp phải trong quá trình thực hiện đề tài.

Chương 5 – Kết luận và hướng phát triển: Trình bày sản phẩm đạt được, các chức năng đã triển khai và kết quả triển khai thực nghiệm của hệ thống. Đồng thời tổng kết những kết quả đạt được, đánh giá mức độ hoàn thành mục tiêu đề tài, các bài học kinh nghiệm, những hạn chế còn tồn tại và định hướng phát triển trong tương lai.

Chương 2

Yêu cầu hệ thống

Chương này khảo sát một số hệ thống tiêu biểu có liên quan đến đề tài, bao gồm NotebookLM, Quizlet và Notion. Mỗi hệ thống được phân tích theo các khía cạnh: mục đích sử dụng, chức năng chính, ưu điểm, hạn chế và mức độ phù hợp đối với bài toán xây dựng một nền tảng hỗ trợ học tập tích hợp. Trên cơ sở đó, chương rút ra các vấn đề còn tồn tại và xác định những yêu cầu đặt ra đối với hệ thống đề xuất.

2.1 Các công trình liên quan

2.1.1 NotebookLM

Giới thiệu tổng quan

Sự phát triển mạnh mẽ của các mô hình ngôn ngữ lớn (Large Language Models – LLM) trong những năm gần đây đã tạo điều kiện cho việc hình thành nhiều hệ thống hỗ trợ học tập và nghiên cứu dựa trên trí tuệ nhân tạo. Trong số đó, NotebookLM là một trong những sản phẩm tiêu biểu được phát triển nhằm hỗ trợ người dùng khai thác và tương tác với các nguồn tài liệu cá nhân thông qua ngôn ngữ tự nhiên.

NotebookLM là một nền tảng được phát triển bởi Google, cho phép người dùng tải lên nhiều loại dữ liệu khác nhau như tài liệu PDF, văn bản,

website hoặc các nguồn nội dung trực tuyến để xây dựng một không gian tri thức riêng [3], [4]. Sau khi dữ liệu được cung cấp, hệ thống sử dụng các mô hình trí tuệ nhân tạo để phân tích nội dung và hỗ trợ người dùng khai thác thông tin thông qua các câu hỏi bằng ngôn ngữ tự nhiên.

Khác với các chatbot tổng quát hoạt động dựa trên tập dữ liệu huấn luyện có sẵn, NotebookLM tập trung vào việc trả lời dựa trên các nguồn dữ liệu mà người dùng cung cấp. Điều này giúp nâng cao tính liên quan của câu trả lời và giảm thiểu các thông tin không phù hợp với ngữ cảnh nghiên cứu hoặc học tập của người dùng.

Trong lĩnh vực giáo dục, NotebookLM được xem như một trợ lý học tập hỗ trợ người dùng đọc hiểu tài liệu, tổng hợp thông tin và khai thác tri thức từ nhiều nguồn dữ liệu khác nhau.

Các chức năng chính

NotebookLM cung cấp nhiều chức năng hỗ trợ quá trình nghiên cứu và học tập.

Trước hết, hệ thống cho phép người dùng tải lên các nguồn tài liệu khác nhau và xây dựng một không gian làm việc tập trung. Sau khi dữ liệu được xử lý, người dùng có thể tương tác với nội dung thông qua các câu hỏi tự nhiên giống như đang trao đổi với một trợ lý cá nhân.

Ngoài chức năng hỏi đáp dựa trên tài liệu, hệ thống còn hỗ trợ:

- Tóm tắt tài liệu học tập.
- Trích xuất các ý chính từ nội dung dài.
- Giải thích các khái niệm hoặc thuật ngữ chuyên môn.
- Tạo danh sách câu hỏi liên quan đến nội dung tài liệu.
- Hỗ trợ nghiên cứu và tổng hợp thông tin.
- Hỗ trợ ghi chú dựa trên nguồn dữ liệu đầu vào.

Các chức năng trên giúp người dùng giảm đáng kể thời gian đọc và xử lý tài liệu, đặc biệt trong các trường hợp phải làm việc với lượng lớn thông tin.

Ưu điểm

Một trong những ưu điểm lớn nhất của NotebookLM là khả năng khai thác thông tin dựa trên dữ liệu thực tế của người dùng.

Thay vì sử dụng hoàn toàn kiến thức được huấn luyện trước, hệ thống tập trung vào việc phân tích các nguồn dữ liệu được cung cấp. Điều này giúp tăng tính liên quan của kết quả và hỗ trợ tốt hơn cho các hoạt động nghiên cứu chuyên sâu.

Khả năng tóm tắt tài liệu cũng là một ưu điểm đáng chú ý. Đối với các tài liệu học thuật hoặc tài liệu kỹ thuật có độ dài lớn, việc nhanh chóng nắm bắt các nội dung quan trọng giúp người dùng tiết kiệm đáng kể thời gian nghiên cứu.

Bên cạnh đó, khả năng giải thích khái niệm và trả lời câu hỏi theo ngữ cảnh giúp NotebookLM trở thành một công cụ hữu ích đối với sinh viên và người học trong quá trình tìm hiểu kiến thức mới.

Một ưu điểm khác là khả năng truy xuất nguồn tham khảo. Người dùng có thể dễ dàng xác định vị trí xuất hiện của thông tin trong tài liệu gốc, từ đó tăng độ tin cậy và khả năng kiểm chứng của nội dung được cung cấp.

Hạn chế

Mặc dù mang lại nhiều lợi ích trong việc hỗ trợ khai thác tài liệu, NotebookLM vẫn tồn tại một số hạn chế nhất định khi được xem xét dưới góc độ của một hệ thống hỗ trợ học tập toàn diện.

Trước hết, hệ thống chủ yếu tập trung vào giai đoạn tiếp nhận và xử lý thông tin. Sau khi người dùng đã hiểu nội dung tài liệu, NotebookLM chưa cung cấp nhiều công cụ chuyên biệt để hỗ trợ các hoạt động học tập

tiếp theo như luyện tập, đánh giá kiến thức hoặc theo dõi tiến trình học tập.

Bên cạnh đó, hệ thống chưa tập trung vào việc xây dựng cơ chế ghi nhớ dài hạn. Mặc dù có thể tạo ra các câu hỏi hoặc bản tóm tắt từ nội dung tài liệu, NotebookLM không cung cấp các công cụ học tập chuyên sâu như Flashcard hoặc các hình thức ôn tập có tính hệ thống.

Một hạn chế khác là khả năng theo dõi tiến trình học tập còn tương đối hạn chế. Hệ thống chưa hỗ trợ việc ghi nhận kết quả học tập, phân tích mức độ hiểu bài hoặc đánh giá sự tiến bộ của người dùng theo thời gian.

Ngoài ra, NotebookLM chưa được thiết kế như một hệ sinh thái học tập hoàn chỉnh. Các hoạt động quản lý mục tiêu, quản lý thời gian học tập hoặc đánh giá hiệu quả học tập chưa phải là trọng tâm của nền tảng này.

Nhận xét

NotebookLM là một giải pháp mạnh trong việc hỗ trợ người dùng khai thác và nghiên cứu tài liệu bằng trí tuệ nhân tạo. Các chức năng hỏi đáp theo ngữ cảnh, tóm tắt nội dung và giải thích khái niệm giúp người học tiếp cận tri thức nhanh hơn và hiệu quả hơn.

Tuy nhiên, hệ thống chủ yếu hỗ trợ giai đoạn tiếp nhận kiến thức và nghiên cứu tài liệu. Các hoạt động như ghi nhớ, luyện tập, đánh giá kết quả học tập hoặc theo dõi tiến trình học tập chưa được hỗ trợ một cách toàn diện. Do đó, người dùng vẫn cần kết hợp với các công cụ khác để xây dựng một quy trình học tập hoàn chỉnh.

2.1.2 Quizlet

Giới thiệu tổng quan

Trong lĩnh vực hỗ trợ ghi nhớ và ôn tập kiến thức, Quizlet là một trong những nền tảng học tập trực tuyến phổ biến và được sử dụng rộng rãi nhất hiện nay. Được phát triển từ năm 2005, Quizlet ban đầu được xây dựng nhằm hỗ trợ học sinh ghi nhớ từ vựng ngoại ngữ thông qua hình thức Flashcard điện tử. Trải qua nhiều năm phát triển, hệ thống đã mở rộng phạm vi ứng dụng sang nhiều lĩnh vực khác nhau như khoa học tự nhiên, kỹ thuật, y học, kinh tế và các chương trình đào tạo chuyên môn.

Khác với các nền tảng tập trung vào việc quản lý tài liệu hoặc ghi chú, Quizlet được thiết kế xoay quanh hoạt động ôn tập và củng cố kiến thức. Triết lý cốt lõi của hệ thống là giúp người học ghi nhớ thông tin hiệu quả hơn thông qua việc chủ động truy xuất kiến thức thay vì chỉ đọc lại tài liệu một cách thụ động.

Hiện nay, Quizlet cung cấp cả phiên bản Web và Mobile, cho phép người dùng học tập linh hoạt trên nhiều thiết bị khác nhau. Với cộng đồng người dùng lớn và số lượng bộ Flashcard phong phú, Quizlet đã trở thành một trong những công cụ học tập quen thuộc đối với học sinh, sinh viên và người tự học trên toàn thế giới.

Các chức năng chính

Chức năng trung tâm của Quizlet là hệ thống Flashcard điện tử.

Người dùng có thể tạo các bộ Flashcard bao gồm hai mặt thông tin, trong đó một mặt thường chứa câu hỏi hoặc khái niệm cần ghi nhớ, mặt còn lại chứa câu trả lời hoặc nội dung giải thích tương ứng [5].

Bên cạnh chức năng Flashcard cơ bản, Quizlet còn cung cấp nhiều hình thức học tập khác nhau nhằm tăng cường khả năng ghi nhớ và giảm sự nhàm chán trong quá trình ôn tập.

Các chức năng tiêu biểu của hệ thống bao gồm:

- Tạo và quản lý bộ Flashcard.
- Học Flashcard theo hình thức lật thẻ.
- Chế độ kiểm tra kiến thức.
- Chế độ ghép cặp nội dung.
- Chế độ viết lại đáp án.
- Chế độ trắc nghiệm.
- Theo dõi tiến độ học tập.
- Chia sẻ Flashcard với cộng đồng.
- Tìm kiếm và sử dụng các bộ Flashcard công khai.

Trong những năm gần đây, Quizlet cũng bắt đầu tích hợp các tính năng trí tuệ nhân tạo nhằm hỗ trợ người dùng tự động tạo Flashcard từ văn bản đầu vào hoặc nội dung học tập có sẵn [6]. Điều này giúp giảm đáng kể thời gian chuẩn bị nội dung học tập và nâng cao trải nghiệm người dùng.

Ưu điểm

Ưu điểm nổi bật nhất của Quizlet nằm ở khả năng hỗ trợ ghi nhớ kiến thức.

Nhiều nghiên cứu trong lĩnh vực giáo dục cho thấy rằng việc chủ động truy xuất thông tin từ trí nhớ (Active Recall) mang lại hiệu quả học tập cao hơn so với việc chỉ đọc lại tài liệu [1], [2]. Flashcard là một trong những phương pháp phổ biến nhất để áp dụng nguyên lý này vào thực tế.

Thông qua việc liên tục đặt câu hỏi và yêu cầu người học nhớ lại thông tin, Quizlet giúp tăng cường khả năng ghi nhớ và củng cố kiến thức trong dài hạn.

Một ưu điểm khác là tính đơn giản trong quá trình sử dụng. Người dùng có thể nhanh chóng tạo hoặc sử dụng các bộ Flashcard mà không

cần nhiều kiến thức kỹ thuật. Giao diện trực quan và dễ sử dụng giúp hệ thống tiếp cận được nhiều nhóm người dùng khác nhau.

Quizlet cũng sở hữu một cộng đồng người dùng lớn với số lượng bộ Flashcard phong phú. Điều này cho phép người dùng tìm kiếm và tái sử dụng nội dung học tập đã được xây dựng bởi những người dùng khác, giúp tiết kiệm thời gian và công sức chuẩn bị tài liệu.

Ngoài ra, việc hỗ trợ đa nền tảng giúp người dùng có thể học tập mọi lúc, mọi nơi và duy trì tính liên tục trong quá trình ôn tập.

Hạn chế

Mặc dù rất hiệu quả trong việc hỗ trợ ghi nhớ kiến thức, Quizlet vẫn tồn tại một số hạn chế khi được xem xét như một hệ thống hỗ trợ học tập toàn diện.

Thứ nhất, hệ thống chủ yếu tập trung vào giai đoạn ôn tập sau khi kiến thức đã được tiếp nhận. Quizlet chưa cung cấp nhiều công cụ hỗ trợ cho việc nghiên cứu tài liệu hoặc xây dựng tri thức ban đầu. Người dùng thường phải sử dụng các nền tảng khác để đọc tài liệu, ghi chú và tổ chức kiến thức trước khi chuyển sang giai đoạn tạo Flashcard.

Thứ hai, khả năng quản lý tri thức còn tương đối hạn chế. Các bộ Flashcard thường được tổ chức theo chủ đề hoặc môn học, tuy nhiên hệ thống chưa hỗ trợ mạnh việc xây dựng mạng lưới kiến thức hoặc liên kết giữa các nội dung học tập khác nhau.

Thứ ba, mặc dù hệ thống có hỗ trợ các hình thức kiểm tra, việc đánh giá kiến thức vẫn chủ yếu xoay quanh nội dung của Flashcard. Khả năng phân tích kết quả học tập hoặc xác định những vùng kiến thức còn yếu của người dùng chưa thực sự chuyên sâu.

Thứ tư, Quizlet chưa tập trung vào việc xây dựng lộ trình học tập hoặc hỗ trợ người dùng quản lý mục tiêu học tập dài hạn. Người học vẫn cần sử dụng thêm các công cụ khác để theo dõi tiến trình học tập hoặc xây dựng kế hoạch học tập cá nhân.

Cuối cùng, mặc dù đã tích hợp một số chức năng AI, vai trò của trí tuệ nhân tạo trong Quizlet vẫn chủ yếu hỗ trợ tạo nội dung học tập thay vì tham gia vào toàn bộ quá trình học tập của người dùng.

Nhận xét

Quizlet là một giải pháp hiệu quả cho bài toán ghi nhớ và ôn tập kiến thức. Việc áp dụng Flashcard kết hợp với các hình thức học tập tương tác giúp nâng cao khả năng ghi nhớ và thúc đẩy quá trình học tập chủ động.

Tuy nhiên, hệ thống chủ yếu tập trung vào giai đoạn củng cố kiến thức thay vì toàn bộ quá trình học tập. Các hoạt động nghiên cứu tài liệu, quản lý tri thức, đánh giá tiến trình học tập và hỗ trợ xây dựng kế hoạch học tập vẫn cần được thực hiện thông qua các nền tảng khác.

Do đó, mặc dù Quizlet giải quyết tốt bài toán ghi nhớ kiến thức, người dùng vẫn cần kết hợp với nhiều công cụ khác để xây dựng một môi trường học tập hoàn chỉnh.

2.1.3 Notion

Giới thiệu tổng quan

Trong những năm gần đây, cùng với sự gia tăng về khối lượng thông tin và tài nguyên số, nhu cầu quản lý tri thức cá nhân (Personal Knowledge Management – PKM) ngày càng trở nên quan trọng đối với học sinh, sinh viên và người lao động tri thức. Trong số các nền tảng hỗ trợ quản lý tri thức hiện nay, Notion là một trong những giải pháp phổ biến và có ảnh hưởng lớn nhất.

Notion được phát triển với mục tiêu xây dựng một không gian làm việc thống nhất, cho phép người dùng ghi chú, quản lý tài liệu, theo dõi công việc và tổ chức thông tin trên cùng một nền tảng [7]. Khác với các công cụ ghi chú truyền thống chỉ tập trung vào việc lưu trữ văn bản, Notion cung cấp nhiều thành phần dữ liệu linh hoạt như cơ sở dữ liệu (Database), bảng

biểu (Table), Kanban Board, Calendar, Checklist và nhiều dạng nội dung khác nhằm hỗ trợ người dùng xây dựng hệ thống quản lý thông tin theo nhu cầu riêng.

Nhờ khả năng tùy biến cao, Notion nhanh chóng trở thành công cụ được sử dụng rộng rãi trong học tập, nghiên cứu, quản lý dự án và quản lý tri thức cá nhân. Đặc biệt, nhiều người học lựa chọn Notion như một “bộ não thứ hai” (Second Brain) để lưu trữ, tổ chức và liên kết các kiến thức thu thập được trong quá trình học tập.

Trong lĩnh vực giáo dục, Notion thường được sử dụng để quản lý tài liệu học tập, xây dựng hệ thống ghi chú, theo dõi tiến độ học tập và lập kế hoạch học tập cá nhân.

Các chức năng chính

Notion cung cấp một tập hợp chức năng đa dạng phục vụ việc tổ chức và quản lý thông tin.

Trước hết, hệ thống cho phép người dùng tạo và quản lý các trang nội dung (Pages). Mỗi trang có thể chứa văn bản, hình ảnh, liên kết, bảng dữ liệu hoặc các thành phần nội dung khác nhau.

Bên cạnh đó, Notion còn hỗ trợ các chức năng nổi bật như:

- Tạo và quản lý ghi chú.
- Xây dựng cơ sở dữ liệu tùy chỉnh.
- Quản lý công việc và nhiệm vụ.
- Tạo lịch học tập hoặc lịch làm việc.
- Tạo hệ thống wiki cá nhân.
- Liên kết giữa các nội dung khác nhau.
- Chia sẻ và cộng tác theo nhóm.

- Đồng bộ dữ liệu trên nhiều thiết bị.

Trong các phiên bản gần đây, Notion cũng đã tích hợp Notion AI nhằm hỗ trợ người dùng thực hiện một số tác vụ xử lý nội dung [8] như:

- Tóm tắt văn bản.
- Viết nội dung tự động.
- Gợi ý ý tưởng.
- Hỗ trợ chỉnh sửa nội dung.
- Trả lời câu hỏi dựa trên dữ liệu của người dùng.

Sự kết hợp giữa quản lý tri thức và trí tuệ nhân tạo giúp Notion trở thành một nền tảng linh hoạt trong nhiều lĩnh vực khác nhau.

Ưu điểm

Ưu điểm lớn nhất của Notion là khả năng tổ chức và quản lý thông tin.

Không giống các ứng dụng ghi chú truyền thống, Notion cho phép người dùng xây dựng cấu trúc dữ liệu phù hợp với nhu cầu cá nhân. Người dùng có thể tạo các không gian làm việc riêng cho từng mục đích như học tập, nghiên cứu, quản lý dự án hoặc quản lý công việc cá nhân.

Khả năng liên kết giữa các trang và cơ sở dữ liệu giúp hình thành mạng lưới tri thức có tính liên kết cao thay vì các ghi chú rời rạc. Điều này đặc biệt hữu ích đối với các hoạt động học tập dài hạn hoặc nghiên cứu chuyên sâu.

Một ưu điểm khác là khả năng tùy biến linh hoạt. Người dùng có thể xây dựng nhiều mô hình quản lý tri thức khác nhau mà không bị giới hạn bởi một cấu trúc cố định.

Notion cũng hỗ trợ cộng tác hiệu quả thông qua các chức năng chia sẻ và chỉnh sửa theo thời gian thực. Điều này tạo điều kiện thuận lợi cho các

nhóm học tập hoặc các nhóm nghiên cứu làm việc cùng nhau trên cùng một không gian dữ liệu.

Ngoài ra, việc hỗ trợ trên nhiều nền tảng giúp người dùng dễ dàng truy cập và quản lý dữ liệu ở bất kỳ đâu.

Hạn chế

Mặc dù là một công cụ mạnh trong quản lý tri thức, Notion vẫn tồn tại một số hạn chế khi được xem xét dưới góc độ của một hệ thống hỗ trợ học tập toàn diện.

Thứ nhất, Notion chủ yếu tập trung vào việc lưu trữ và tổ chức thông tin. Hệ thống chưa cung cấp nhiều công cụ chuyên biệt cho các hoạt động học tập như ôn tập kiến thức, kiểm tra năng lực hoặc đánh giá tiến trình học tập.

Thứ hai, người dùng thường phải tự xây dựng phương pháp học tập của riêng mình. Notion cung cấp môi trường lưu trữ và quản lý kiến thức nhưng không hướng dẫn hoặc hỗ trợ trực tiếp cách khai thác kiến thức một cách hiệu quả.

Thứ ba, khả năng ghi nhớ và ôn tập kiến thức chưa phải là trọng tâm của nền tảng. Mặc dù người dùng có thể tự xây dựng các hệ thống Flashcard hoặc quy trình học tập trong Notion, việc này đòi hỏi nhiều công sức thiết lập và bảo trì.

Thứ tư, Notion chưa hỗ trợ mạnh việc đánh giá kiến thức hoặc phân tích dữ liệu học tập. Hệ thống không cung cấp các cơ chế chuyên biệt để xác định mức độ hiểu bài, theo dõi kết quả học tập hoặc phát hiện các vùng kiến thức còn yếu của người dùng.

Ngoài ra, mặc dù đã tích hợp Notion AI, các chức năng AI hiện nay chủ yếu hỗ trợ xử lý nội dung và nâng cao năng suất làm việc, thay vì tham gia sâu vào quá trình học tập hoặc hỗ trợ cá nhân hóa việc học.

Nhận xét

Notion là một nền tảng quản lý tri thức mạnh mẽ với khả năng tổ chức thông tin linh hoạt và khả năng tùy biến cao. Hệ thống đặc biệt phù hợp với những người dùng cần xây dựng kho tri thức cá nhân hoặc quản lý lượng lớn thông tin trong thời gian dài.

Tuy nhiên, Notion không được thiết kế như một hệ thống học tập chuyên biệt. Các hoạt động ôn tập, đánh giá kiến thức, theo dõi tiến trình học tập và hỗ trợ học tập bằng trí tuệ nhân tạo vẫn chưa được triển khai ở mức độ sâu.

Do đó, mặc dù đóng vai trò quan trọng trong việc quản lý tri thức cá nhân, Notion thường cần được kết hợp với các công cụ khác để hình thành một môi trường học tập hoàn chỉnh.

2.2 So sánh tổng hợp các hệ thống liên quan

Từ phần phân tích trên, có thể tổng hợp đặc điểm của các hệ thống NotebookLM, Quizlet và Notion theo một số tiêu chí chính như Bảng 2.1.

Bảng so sánh cho thấy mỗi hệ thống hiện có đều giải quyết tốt một nhóm nhu cầu nhất định, nhưng chưa bao phủ toàn bộ quá trình học tập. Đây là cơ sở để đề tài hướng tới một nền tảng tích hợp, trong đó các hoạt động tiếp nhận kiến thức, tổ chức tri thức, ôn tập, đánh giá và theo dõi tiến trình được kết nối trong cùng một quy trình.

2.3 Những vấn đề còn tồn tại

Qua quá trình khảo sát và phân tích các nền tảng NotebookLM, Quizlet và Notion, có thể nhận thấy rằng mỗi hệ thống đều giải quyết hiệu quả một nhóm vấn đề cụ thể trong hoạt động học tập.

NotebookLM hỗ trợ người dùng khai thác và tương tác với tài liệu bằng trí tuệ nhân tạo. Quizlet tập trung vào việc ghi nhớ kiến thức thông qua

Bảng 2.1: So sánh tổng hợp các hệ thống liên quan

Hệ thống	Trọng tâm hỗ trợ	Ưu điểm chính	Hạn chế
Note-bookLM	Khai thác và tương tác với tài liệu bằng trí tuệ nhân tạo	Hỗ trợ hỏi đáp theo ngữ cảnh, tóm tắt tài liệu, giải thích khái niệm và truy xuất nguồn tham khảo	Chưa tập trung vào ghi nhớ dài hạn, luyện tập, đánh giá kiến thức, quản lý mục tiêu và theo dõi tiến trình học tập
Quizlet	Ôn tập và ghi nhớ kiến thức thông qua Flashcard	Hỗ trợ học tập chủ động, nhiều chế độ luyện tập, cộng đồng Flashcard lớn và khả năng học tập đa nền tảng	Chưa hỗ trợ mạnh việc quản lý tài liệu, ghi chú, tổ chức tri thức, khai thác tài liệu và cá nhân hóa toàn bộ quy trình học tập
Notion	Quản lý tri thức cá nhân, ghi chú và tổ chức thông tin	Linh hoạt trong tổ chức dữ liệu, hỗ trợ cơ sở dữ liệu, quản lý công việc, chia sẻ và cộng tác	Chưa phải hệ thống học tập chuyên biệt; còn hạn chế về Flashcard, kiểm tra kiến thức, phân tích kết quả học tập và đề xuất ôn tập
ProLearning	Nền tảng học tập tích hợp từ quản lý tài nguyên đến ôn tập, đánh giá và theo dõi tiến trình	Kết hợp quản lý tài liệu, ghi chú, AI, Flashcard, bài kiểm tra, mục tiêu học tập, theo dõi tiến trình và đa nền tảng	Cần được thiết kế và triển khai đồng bộ để đảm bảo tính liên kết giữa các chức năng, chất lượng nội dung AI và trải nghiệm người dùng

Flashcard và các hình thức học tập chủ động. Trong khi đó, Notion cung cấp môi trường quản lý tri thức và tổ chức thông tin linh hoạt.

Mặc dù các nền tảng trên đều đạt được thành công trong phạm vi bài toán mà chúng hướng tới, quá trình khảo sát cho thấy vẫn còn tồn tại một số vấn đề chưa được giải quyết triệt để khi xét dưới góc độ của một hệ thống hỗ trợ học tập toàn diện.

2.3.1 Sự phân mảnh của các công cụ học tập

Một trong những khó khăn phổ biến nhất đối với người học hiện nay là sự phân tán của tài nguyên và công cụ học tập.

Trong thực tế, để hoàn thành một quá trình học tập tương đối đầy đủ,

người dùng thường phải sử dụng nhiều hệ thống khác nhau cho từng mục đích riêng biệt.

Ví dụ:

- Sử dụng Google Drive hoặc OneDrive để lưu trữ tài liệu.
- Sử dụng Notion để ghi chú và tổ chức kiến thức.
- Sử dụng NotebookLM để tìm hiểu nội dung tài liệu.
- Sử dụng Quizlet để tạo Flashcard và ôn tập.
- Sử dụng các công cụ quản lý công việc để theo dõi mục tiêu học tập.
- Sử dụng các ứng dụng Pomodoro để quản lý thời gian học.

Mặc dù mỗi công cụ đều thực hiện tốt chức năng của mình, việc phải liên tục chuyển đổi giữa nhiều nền tảng khiến quá trình học tập trở nên rời rạc và làm tăng chi phí quản lý thông tin.

Người học không chỉ phải ghi nhớ kiến thức mà còn phải dành thời gian để tổ chức dữ liệu, đồng bộ nội dung và quản lý quy trình học tập giữa các hệ thống khác nhau.

Điều này đặc biệt ảnh hưởng đến những người học đang phải học tập và làm việc đồng thời, khi quỹ thời gian dành cho việc học vốn đã bị hạn chế.

2.3.2 Thiếu sự liên kết giữa các giai đoạn học tập

Hoạt động học tập không chỉ bao gồm việc đọc tài liệu hoặc ghi nhớ kiến thức. Một quá trình học tập hiệu quả thường trải qua nhiều giai đoạn liên tiếp như:

- Tiếp nhận kiến thức.
- Tổ chức và ghi chú nội dung.

- Chuyển đổi kiến thức thành tài nguyên học tập.
- Ôn tập và ghi nhớ.
- Kiểm tra mức độ hiểu bài.
- Phân tích kết quả học tập.
- Xác định các nội dung cần cải thiện.
- Tiếp tục ôn tập và củng cố kiến thức.

Qua khảo sát các hệ thống hiện có có thể nhận thấy rằng phần lớn các nền tảng chỉ tập trung mạnh vào một hoặc một số giai đoạn nhất định trong quy trình trên.

Ví dụ, NotebookLM hỗ trợ tốt việc tiếp nhận và khai thác tài liệu nhưng chưa tập trung vào các hoạt động đánh giá kiến thức. Quizlet hỗ trợ tốt việc ghi nhớ kiến thức nhưng chưa hỗ trợ mạnh cho việc tổ chức tài liệu hoặc quản lý tri thức. Notion hỗ trợ quản lý tri thức hiệu quả nhưng chưa cung cấp nhiều công cụ chuyên biệt cho hoạt động ôn tập và đánh giá.

Việc thiếu sự liên kết giữa các giai đoạn học tập khiến người dùng phải tự xây dựng quy trình học tập của riêng mình bằng cách kết hợp nhiều công cụ khác nhau.

2.3.3 Hạn chế trong việc theo dõi tiến trình học tập

Một vấn đề khác được ghi nhận là khả năng theo dõi và đánh giá tiến trình học tập vẫn còn tương đối hạn chế trên nhiều nền tảng hiện nay.

Phần lớn các hệ thống tập trung vào việc cung cấp công cụ học tập nhưng chưa chú trọng đến việc xây dựng hồ sơ học tập dài hạn cho người dùng.

Trong quá trình học tập, người dùng thường có nhu cầu trả lời các câu hỏi như:

- Mình đã học được bao nhiêu nội dung?
- Những chủ đề nào còn yếu?
- Khả năng ghi nhớ hiện tại đang ở mức nào?
- Mức độ tiến bộ của bản thân trong thời gian qua ra sao?
- Nội dung nào cần được ôn tập thêm?

Tuy nhiên, việc trả lời các câu hỏi trên thường đòi hỏi phải tổng hợp dữ liệu từ nhiều nguồn khác nhau hoặc dựa vào đánh giá chủ quan của người học.

Việc thiếu các công cụ theo dõi tiến trình học tập một cách hệ thống có thể khiến người học khó nhận biết được sự tiến bộ của bản thân và khó xây dựng kế hoạch học tập phù hợp.

2.3.4 Hạn chế trong việc cá nhân hóa trải nghiệm học tập

Mỗi người học có nền tảng kiến thức, mục tiêu học tập và tốc độ tiếp thu khác nhau.

Tuy nhiên, nhiều hệ thống hiện nay vẫn áp dụng cách tiếp cận tương đối giống nhau đối với tất cả người dùng.

Trong thực tế, hai người học cùng sử dụng một bộ tài liệu có thể gặp những khó khăn hoàn toàn khác nhau. Một người có thể cần tập trung vào việc hiểu khái niệm, trong khi người còn lại cần ôn tập để ghi nhớ kiến thức hoặc chuẩn bị cho kỳ kiểm tra.

Việc thiếu dữ liệu theo dõi học tập và cơ chế phân tích hiệu suất học tập khiến các hệ thống khó đưa ra các đề xuất phù hợp với từng người dùng.

Điều này làm giảm khả năng cá nhân hóa trải nghiệm học tập và khiến người học phải tự xác định các bước tiếp theo trong quá trình học tập.

2.3.5 Khó khăn trong việc duy trì động lực học tập

Bên cạnh các vấn đề về kiến thức, việc duy trì động lực học tập trong thời gian dài cũng là một thách thức đối với nhiều người học.

Nhiều người bắt đầu học tập với quyết tâm cao nhưng dần mất động lực sau một khoảng thời gian nhất định.

Một số nguyên nhân phổ biến bao gồm:

- Thiếu mục tiêu học tập rõ ràng.
- Không theo dõi được tiến trình học tập.
- Không nhận thấy sự tiến bộ của bản thân.
- Không có cơ chế nhắc nhở hoặc hỗ trợ duy trì thói quen học tập.

Mặc dù hiện nay đã có nhiều công cụ hỗ trợ quản lý công việc hoặc quản lý thời gian, các công cụ này thường hoạt động độc lập với các nền tảng học tập và chưa được tích hợp chặt chẽ vào quá trình học tập của người dùng.

2.3.6 Vai trò của trí tuệ nhân tạo vẫn chưa được khai thác toàn diện

Sự phát triển của các mô hình ngôn ngữ lớn đã mở ra nhiều cơ hội mới trong lĩnh vực giáo dục.

Tuy nhiên, trong nhiều hệ thống hiện nay, trí tuệ nhân tạo chủ yếu được sử dụng cho các tác vụ như:

- Trả lời câu hỏi.
- Tóm tắt tài liệu.
- Hỗ trợ viết nội dung.
- Giải thích khái niệm.

Mặc dù các chức năng trên mang lại nhiều lợi ích, AI vẫn chưa được khai thác đầy đủ trong việc hỗ trợ toàn bộ quá trình học tập.

Các hướng ứng dụng tiềm năng vẫn chưa được khai thác đồng bộ trên một nền tảng duy nhất bao gồm:

- Tích hợp sinh Flashcard và bài kiểm tra trực tiếp từ tài liệu người dùng vào trong cùng một quy trình học tập.
- Hỗ trợ xây dựng tài nguyên ôn tập liên kết với ghi chú và mục tiêu học tập cá nhân.
- Hỗ trợ đánh giá kiến thức và phân tích kết quả học tập.
- Hỗ trợ theo dõi tiến trình học tập dựa trên dữ liệu thực tế của người dùng.
- Đề xuất hoạt động học tập cá nhân hóa dựa trên lịch sử và kết quả học tập.

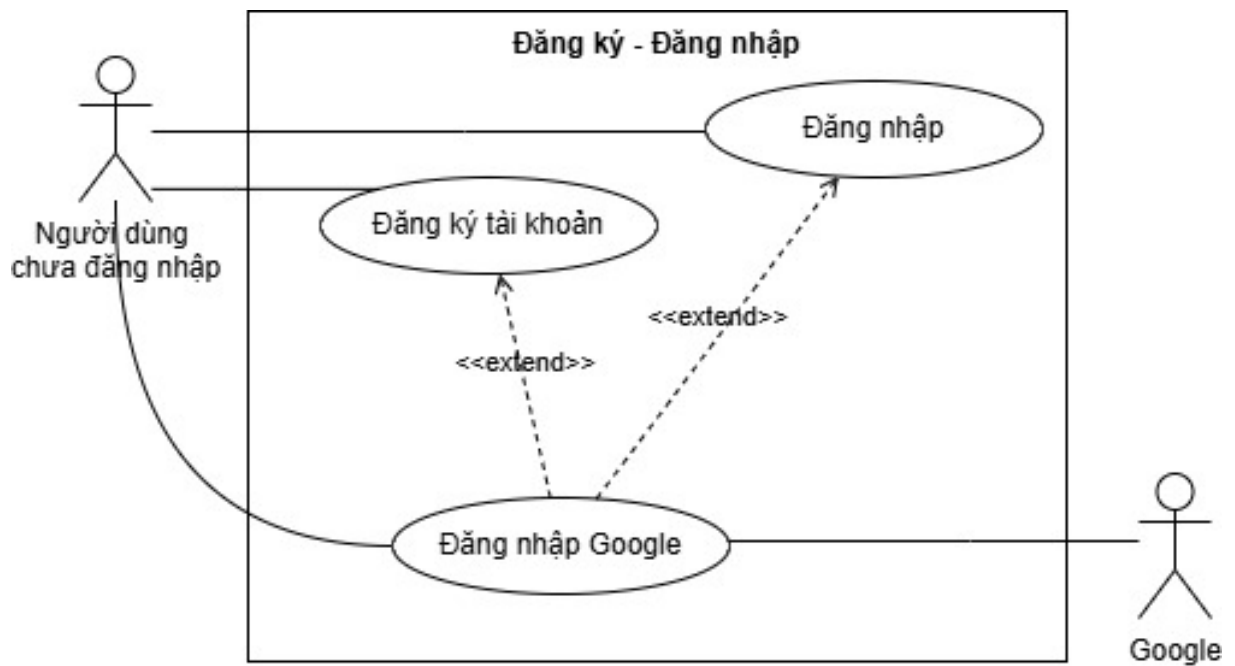
Do đó, việc ứng dụng LLM một cách toàn diện vào quy trình học tập vẫn là hướng phát triển còn nhiều tiềm năng.

2.4 Yêu cầu đặt ra đối với hệ thống đề xuất

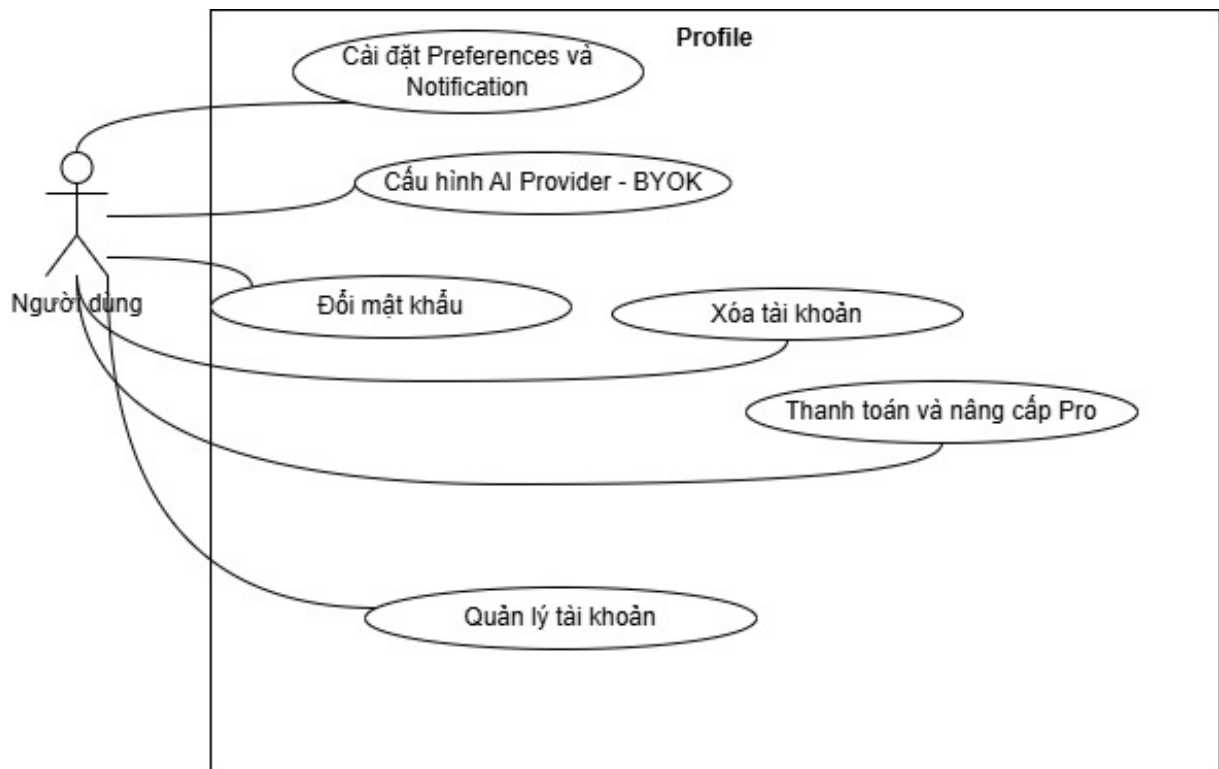
Từ kết quả khảo sát và phân tích các hệ thống NotebookLM, Quizlet và Notion, có thể nhận thấy rằng mỗi nền tảng đều giải quyết hiệu quả một hoặc một số khía cạnh của hoạt động học tập. Tuy nhiên, các vấn đề liên quan đến sự phân mảnh của công cụ học tập, thiếu sự liên kết giữa các giai đoạn học tập, hạn chế trong theo dõi tiến trình học tập và khả năng cá nhân hóa vẫn chưa được giải quyết một cách toàn diện.

Xuất phát từ những vấn đề còn tồn tại đã được phân tích ở mục trước, hệ thống được đề xuất trong đề tài cần đáp ứng một số yêu cầu quan trọng nhằm hỗ trợ người dùng trong toàn bộ quá trình học tập. Các Use Case

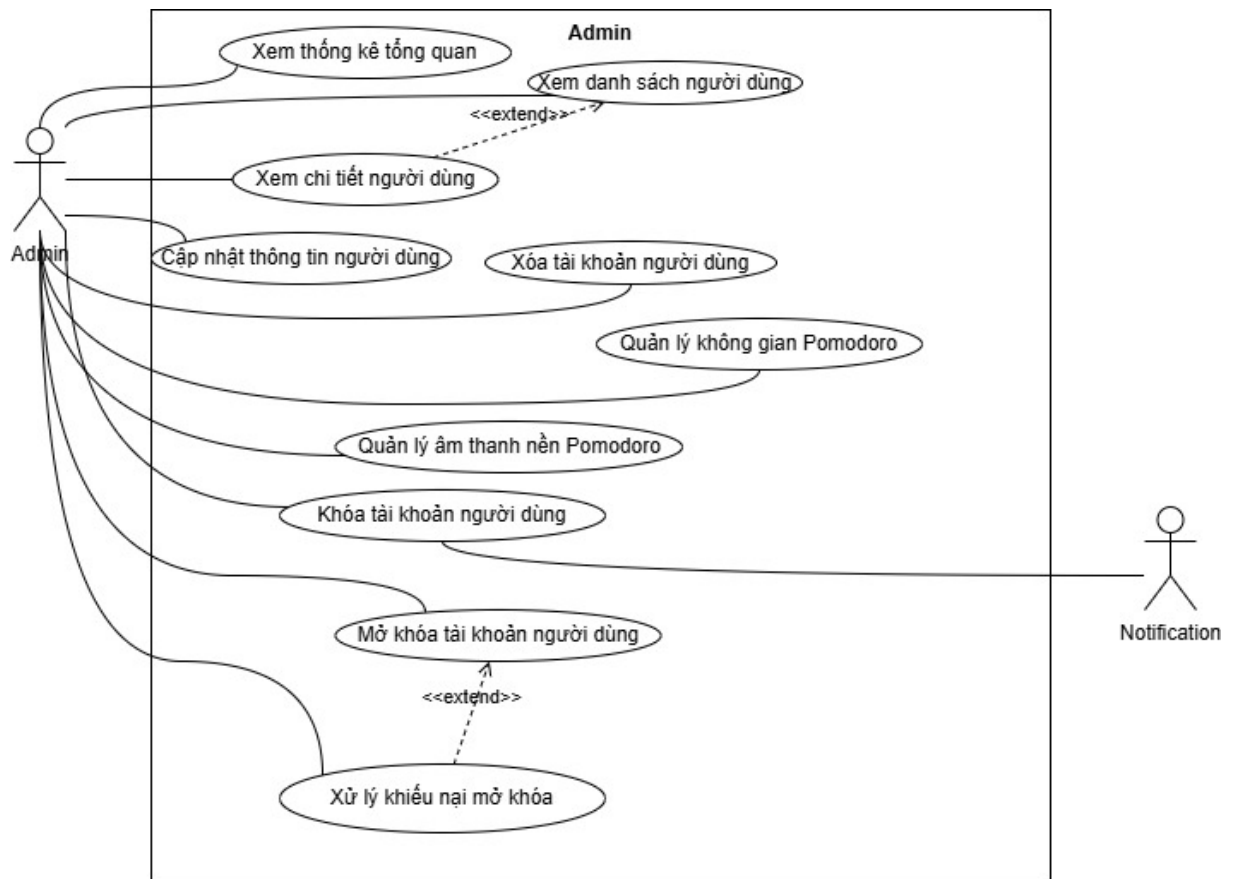
Diagram dưới đây cung cấp cái nhìn tổng quan về các chức năng hệ thống theo từng nhóm nghiệp vụ.



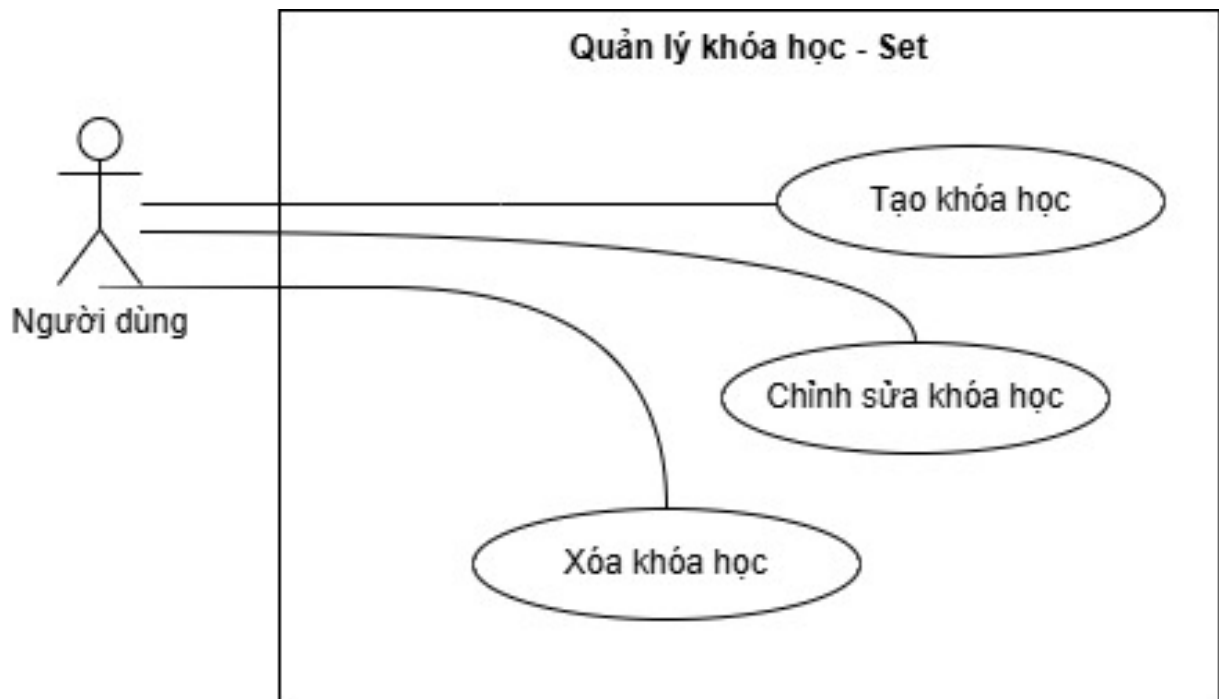
Hình 2.1: Use Case: Đăng ký và Đăng nhập



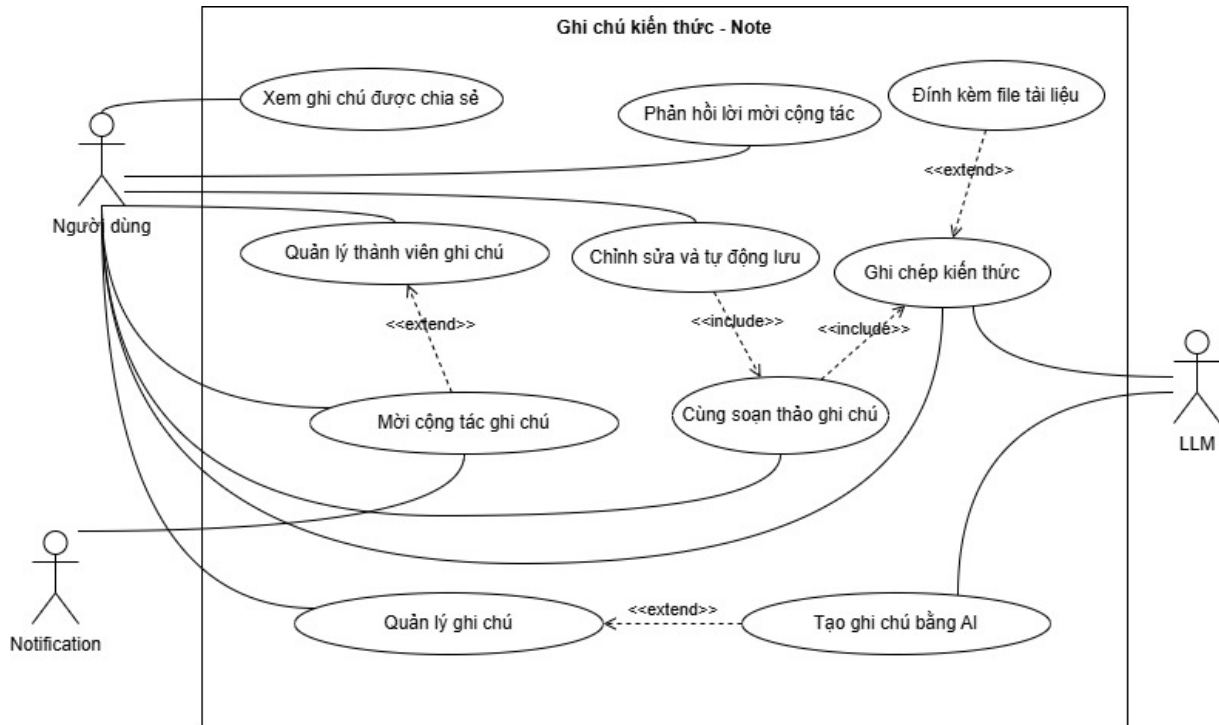
Hình 2.2: Use Case: Quản lý hồ sơ người dùng



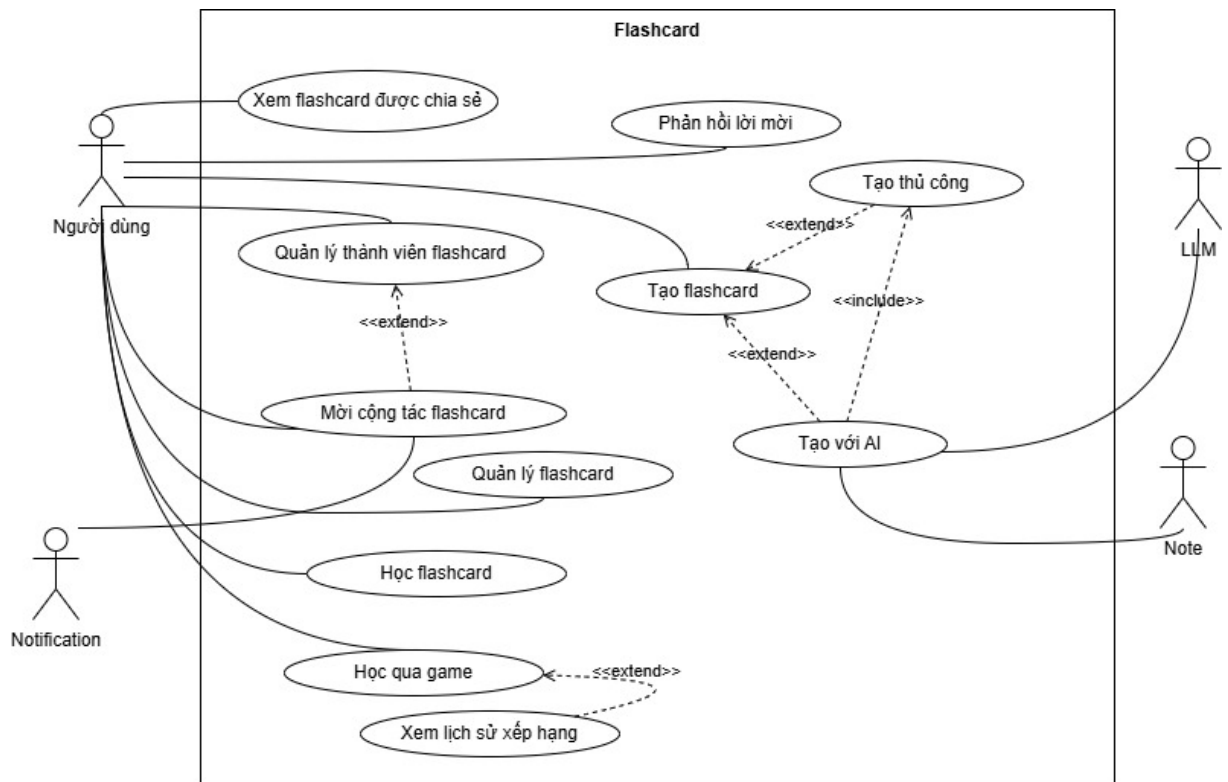
Hình 2.3: Use Case: Quản trị hệ thống



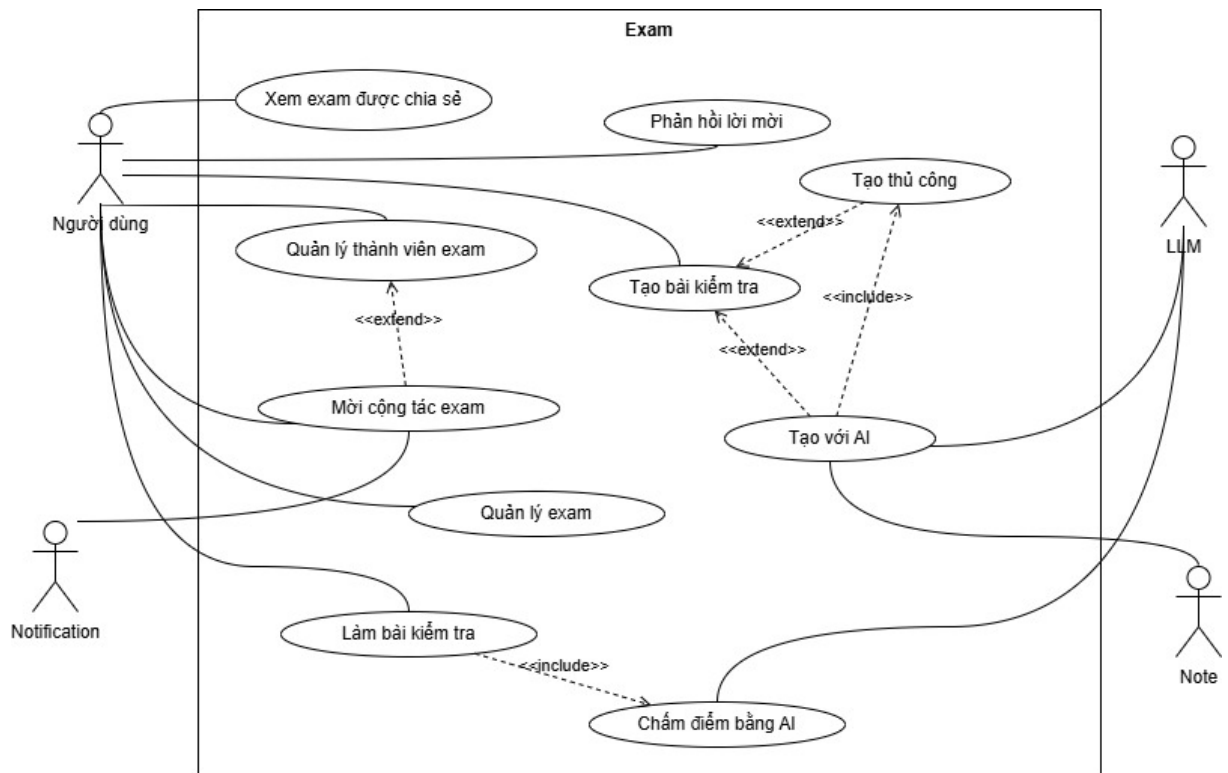
Hình 2.4: Use Case: Quản lý không gian học tập (Set)



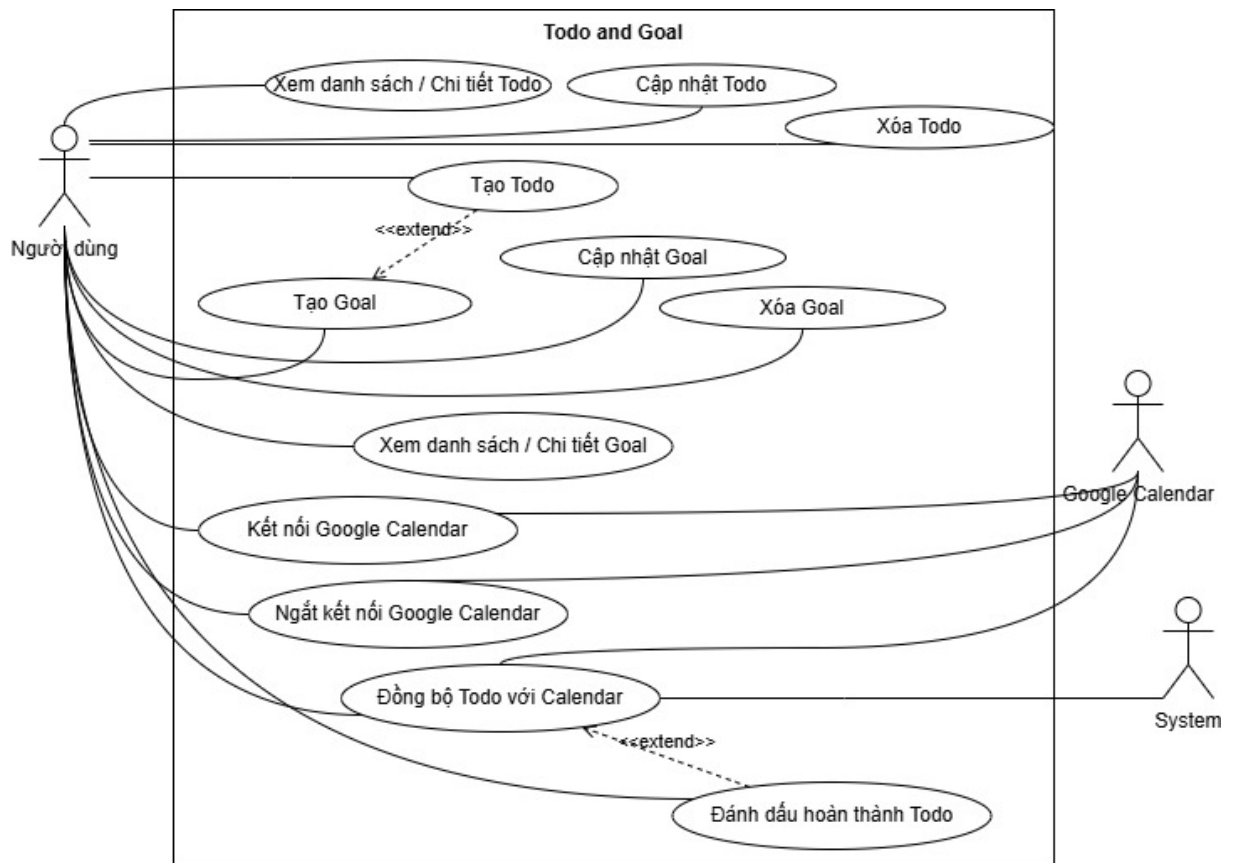
Hình 2.5: Use Case: Ghi chú kiến thức



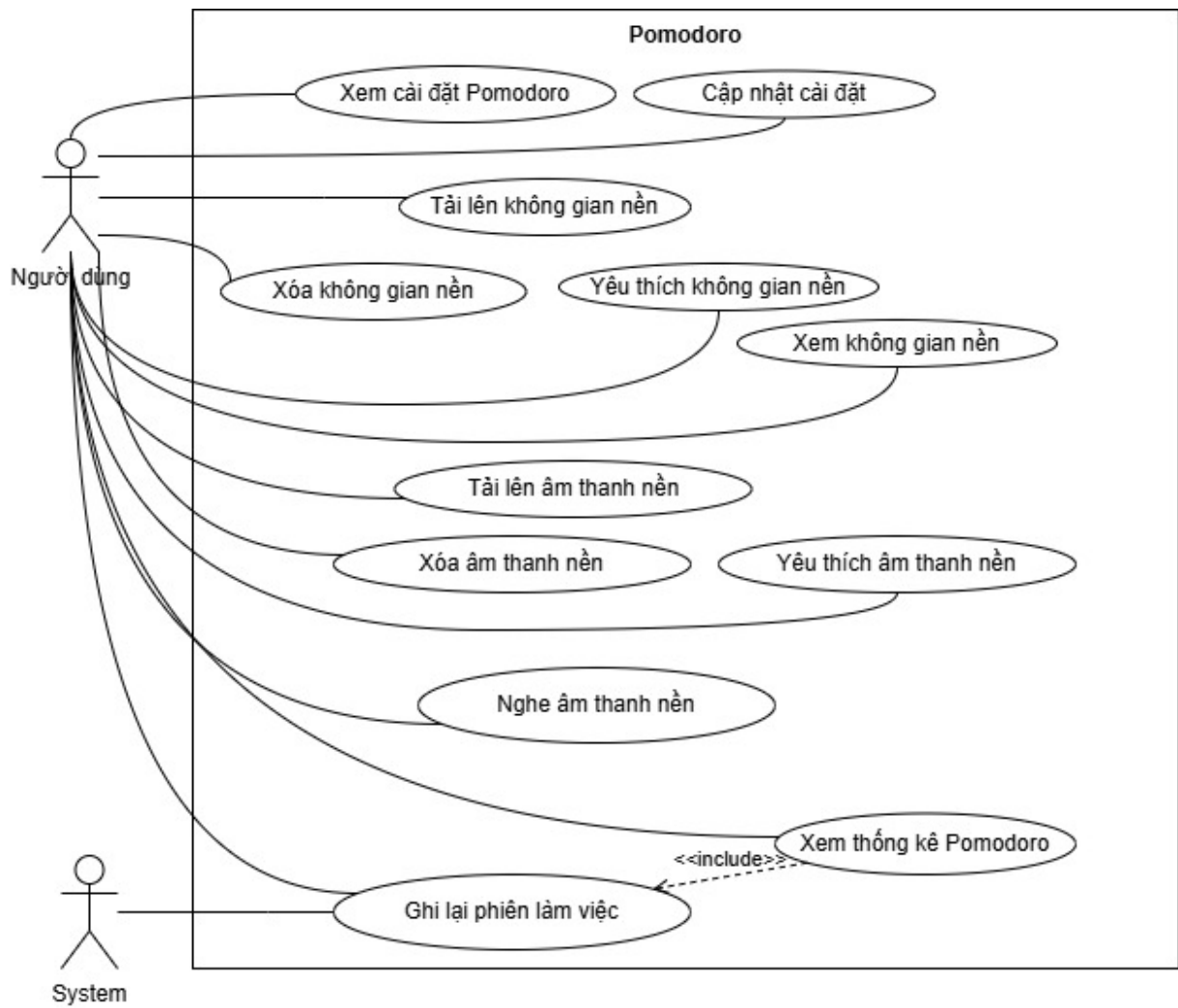
Hình 2.6: Use Case: Học tập với Flashcard



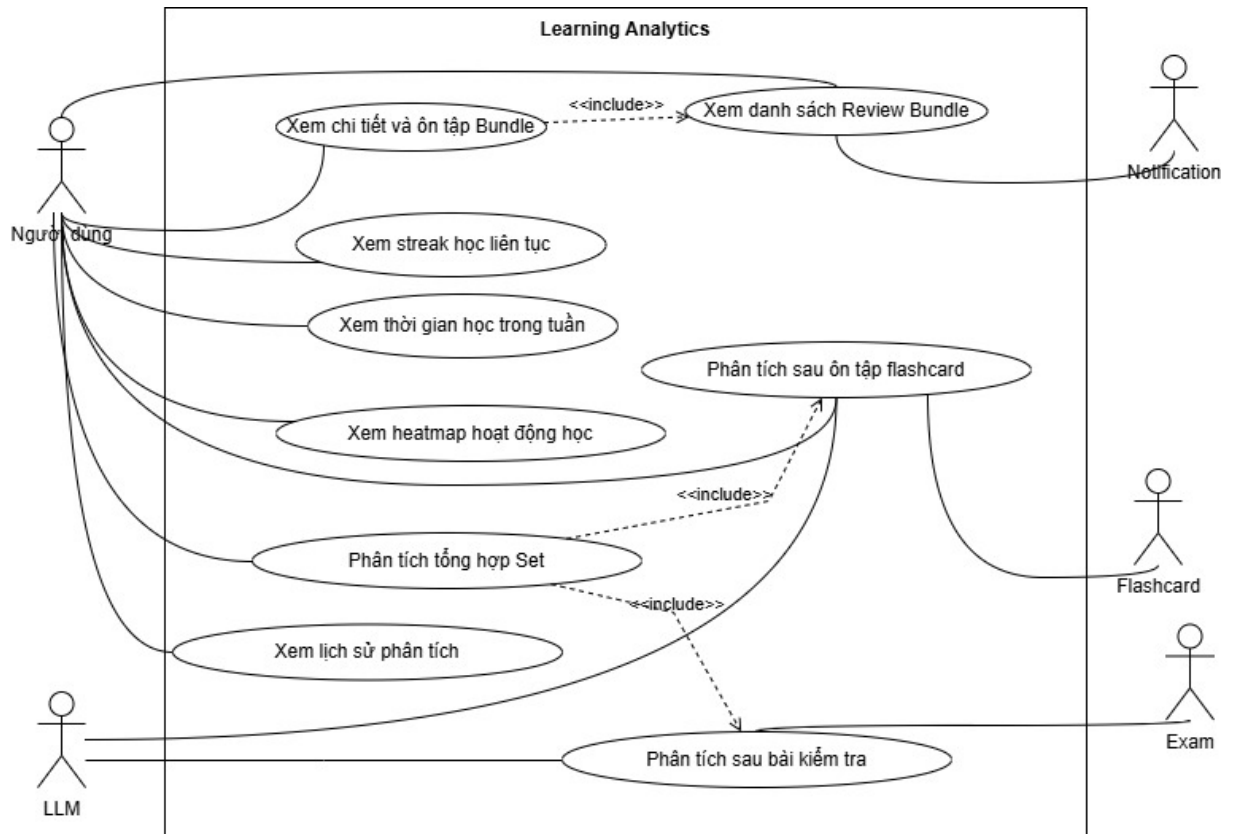
Hình 2.7: Use Case: Kiểm tra và đánh giá kiến thức



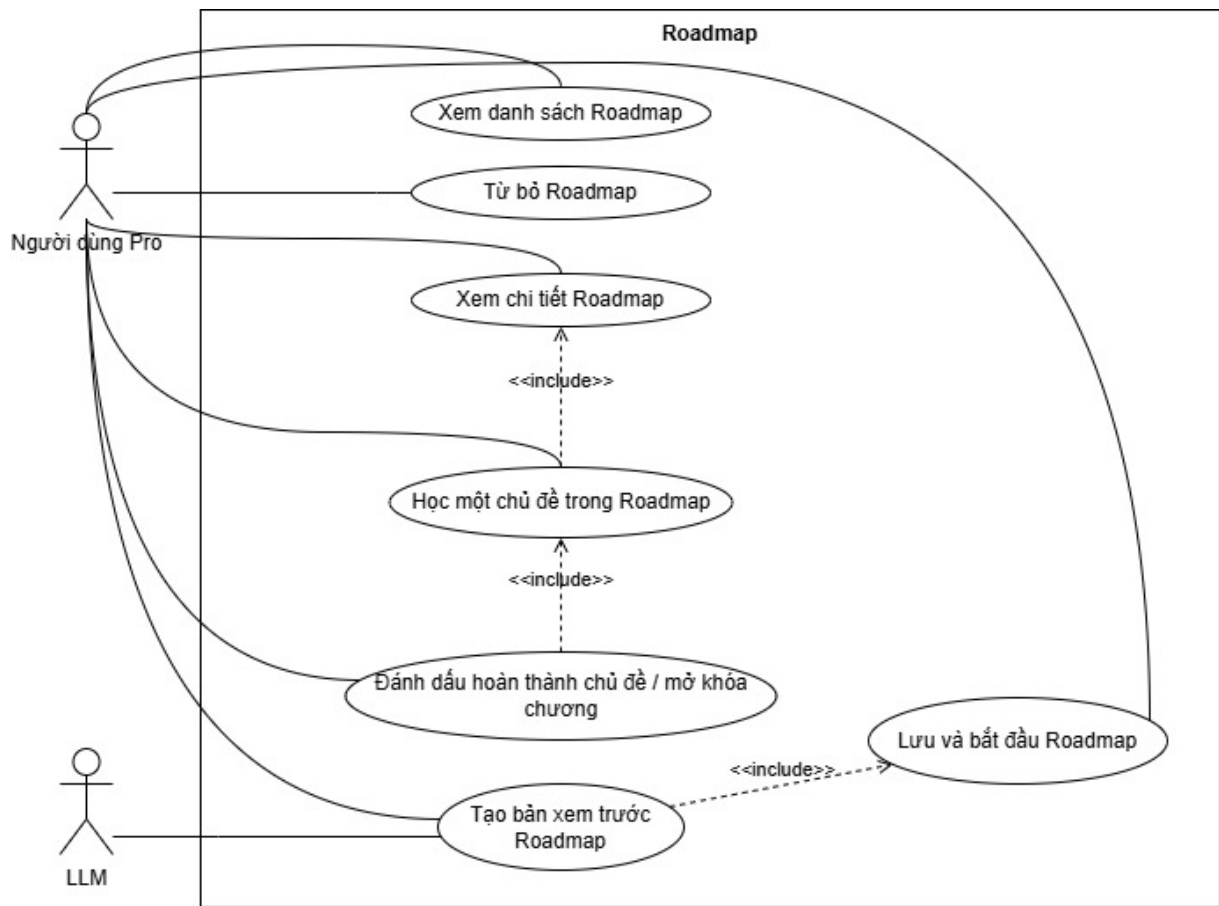
Hình 2.8: Use Case: Quản lý mục tiêu và công việc học tập



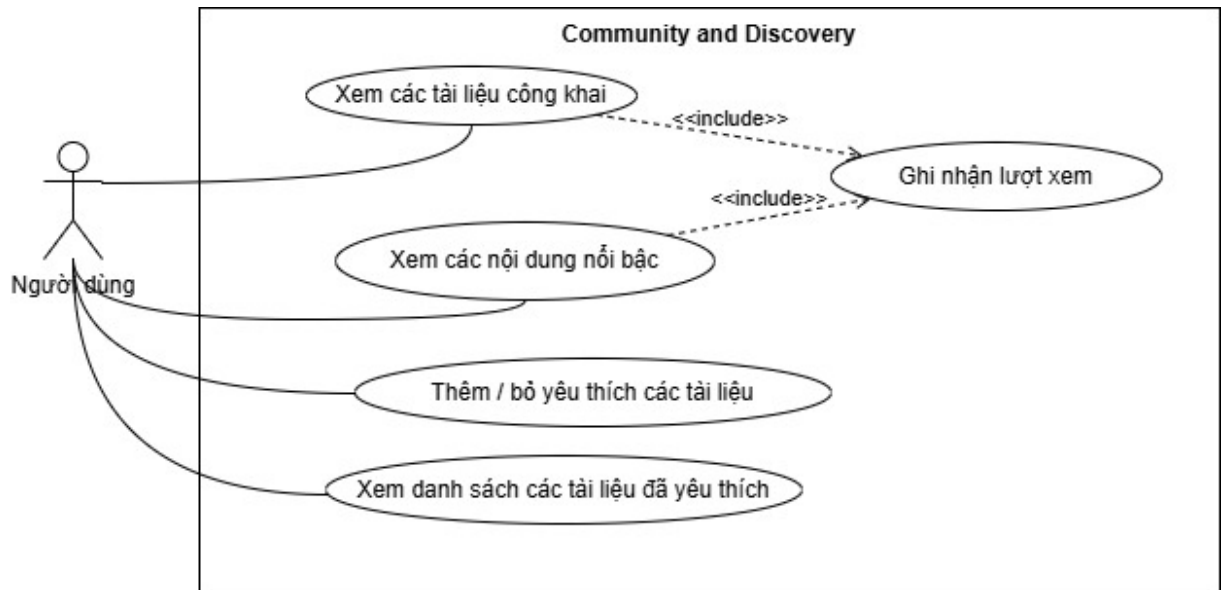
Hình 2.9: Use Case: Quản lý thời gian học tập (Pomodoro)



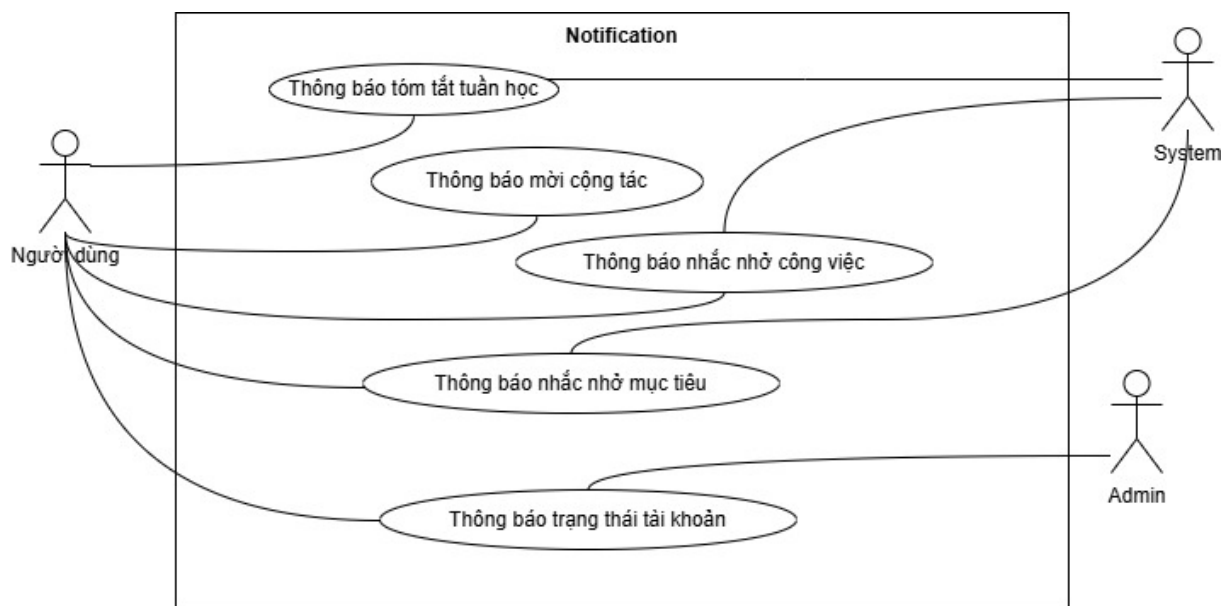
Hình 2.10: Use Case: Theo dõi và phân tích tiến trình học tập



Hình 2.11: Use Case: Lộ trình học tập



Hình 2.12: Use Case: Cộng đồng và khám phá tài nguyên



Hình 2.13: Use Case: Thông báo

Dựa trên kết quả phân tích, hệ thống đề xuất cần đáp ứng các nhóm yêu cầu sau:

Quản lý tài nguyên và tri thức: Hệ thống cần cung cấp môi trường tập trung để người dùng lưu trữ, tổ chức và truy cập các tài nguyên học

tập (ghi chú, PDF, DOCX, bài trình chiếu, nội dung website, video) trên cùng một nền tảng. Người dùng có thể tạo, chỉnh sửa và liên kết các ghi chú thành kho tri thức cá nhân có cấu trúc.

Hỗ trợ học tập bằng LLM và học chủ động: Hệ thống ứng dụng mô hình ngôn ngữ lớn (LLM) để tóm tắt tài liệu, giải thích khái niệm, sinh ghi chú, Flashcard và bài kiểm tra từ nội dung đầu vào. Người học được hỗ trợ ôn tập qua Flashcard, tự kiểm tra và luyện tập qua các bài đánh giá để chuyển từ tiếp nhận thụ động sang học tập có chủ đích.

Đánh giá, theo dõi và cá nhân hóa: Hệ thống cần cung cấp cơ chế tạo và thực hiện bài kiểm tra trực tuyến, chấm điểm tự động và lưu trữ kết quả. Dữ liệu học tập (tiến độ Flashcard, kết quả kiểm tra, thời gian học) được ghi nhận để theo dõi sự tiến bộ và đề xuất nội dung ôn tập phù hợp với từng người dùng.

Quản lý mục tiêu, cộng tác và đa nền tảng: Hệ thống hỗ trợ người dùng duy trì thói quen học tập qua quản lý mục tiêu, công việc, thời gian (Pomodoro) và nhắc nhở. Các chức năng chia sẻ nội dung và cộng tác nhóm giúp mở rộng khả năng học tập cộng đồng. Toàn bộ hệ thống hoạt động đồng bộ trên cả nền tảng Web và thiết bị di động.

2.5 Kết luận chương

Chương này đã trình bày và phân tích ba nền tảng tiêu biểu trong lĩnh vực hỗ trợ học tập và quản lý tri thức gồm NotebookLM, Quizlet và Notion. Mỗi hệ thống đều sở hữu những ưu điểm riêng và giải quyết hiệu quả một nhóm vấn đề cụ thể liên quan đến học tập, nghiên cứu hoặc quản lý tri thức cá nhân.

Tuy nhiên, kết quả khảo sát cho thấy vẫn tồn tại nhiều vấn đề chưa được giải quyết một cách toàn diện như sự phân mảnh của công cụ học tập, thiếu sự liên kết giữa các giai đoạn học tập, hạn chế trong theo dõi tiến trình học tập, khả năng cá nhân hóa còn hạn chế và việc khai thác trí tuệ nhân tạo chưa thực sự bao phủ toàn bộ quá trình học tập.

Từ những vấn đề đó, chương đã xác định các yêu cầu quan trọng đối với hệ thống đề xuất nhằm xây dựng một nền tảng học tập tích hợp có khả năng hỗ trợ người dùng trong nhiều giai đoạn khác nhau của quá trình học tập.

Các yêu cầu được xác định trong chương này sẽ là cơ sở cho việc phân tích yêu cầu, thiết kế kiến trúc và triển khai hệ thống được trình bày trong chương tiếp theo.

Chương 3

Thiết kế và cài đặt

3.1 Kiến trúc tổng quan

3.1.1 Mô hình kiến trúc hệ thống

ProLearning Platform được đề xuất theo mô hình Kiến trúc phân tầng tách biệt với dịch vụ AI chuyên biệt. Thành phần trung tâm là một Spring Boot API server, chịu trách nhiệm xác thực người dùng, kiểm soát quyền truy cập, điều phối nghiệp vụ và kết nối với các dịch vụ hạ tầng. Ở phạm vi hiện tại, hệ thống được triển khai với một server backend chính; khi số lượng người dùng tăng, kiến trúc này có thể mở rộng bằng cách bổ sung thêm các instance backend phía sau load balancer, trong khi trạng thái dùng chung vẫn được đặt tại PostgreSQL, Redis, RabbitMQ hoặc các dịch vụ hạ tầng tương ứng. Cách tổ chức này phù hợp với hệ thống học tập có nhiều miền dữ liệu cần được quản lý nhất quán như người dùng, học liệu, tiến độ học tập, thông báo và thanh toán [9], [10], [11].

Kiến trúc tổng quan của hệ thống gồm các nhóm thành phần chính sau:

- **Client layer:** Bao gồm Web Client và Mobile Client, cùng sử dụng REST API và chỉ dùng WebSocket/STOMP cho các chức năng cần tương tác thời gian thực [12].

- **Application layer:** Spring Boot API là lớp xử lý nghiệp vụ chính, hiện được triển khai dưới dạng một backend server. Thành phần này bao gồm controller, service, repository, mapper, scheduler, security filter và các adapter tích hợp. Mọi thao tác ghi dữ liệu lỗi đều đi qua lớp này để đảm bảo kiểm tra quyền và xử lý lỗi thống nhất.
- **Domain contexts:** Backend được chia thành các bounded context như định danh, learning workspace, collaboration, AI learning, review/progress, notification, payment/subscription và admin/moderation nhằm làm rõ trách nhiệm từng nhóm chức năng.
- **AI Service:** AI Service hỗ trợ sinh flashcard, đề kiểm tra, giải thích ghi chú, phân tích kiến thức và tạo roadmap. Client không gọi trực tiếp dịch vụ này; backend đóng vai trò điều phối, kiểm tra quyền và lưu kết quả vào tài nguyên học tập tương ứng.
- **Data and infrastructure layer:** PostgreSQL lưu dữ liệu quan hệ chính, Redis lưu cache và trạng thái ngắn hạn, RabbitMQ hỗ trợ tác vụ bất đồng bộ. Hệ thống cũng tích hợp Cloudinary, Firebase Cloud Messaging, Google OAuth/Calendar và PayOS [13], [14], [15], [16], [17], [18], [19].

Từ góc nhìn triển khai, kiến trúc trên nhấn mạnh ba nguyên tắc:

1. **Tập trung kiểm soát nghiệp vụ tại backend:** Client chỉ đóng vai trò giao diện và không ghi trực tiếp vào cơ sở dữ liệu hoặc dịch vụ ngoài.
2. **Tách biệt tích hợp bên ngoài:** AI Service, FCM, Google Calendar, Cloudinary và PayOS được bọc bởi service/adaptor riêng.
3. **Phân tách dữ liệu theo đặc tính sử dụng:** Dữ liệu bền vững lưu ở PostgreSQL; dữ liệu ngắn hạn lưu ở Redis hoặc bộ nhớ ứng dụng; tác vụ không cần đồng bộ tức thời được chuyển sang scheduler hoặc message queue.

Sơ đồ dưới đây thể hiện thiết kế tổng thể của hệ thống:

3.2 Tổ chức thành phần hệ thống

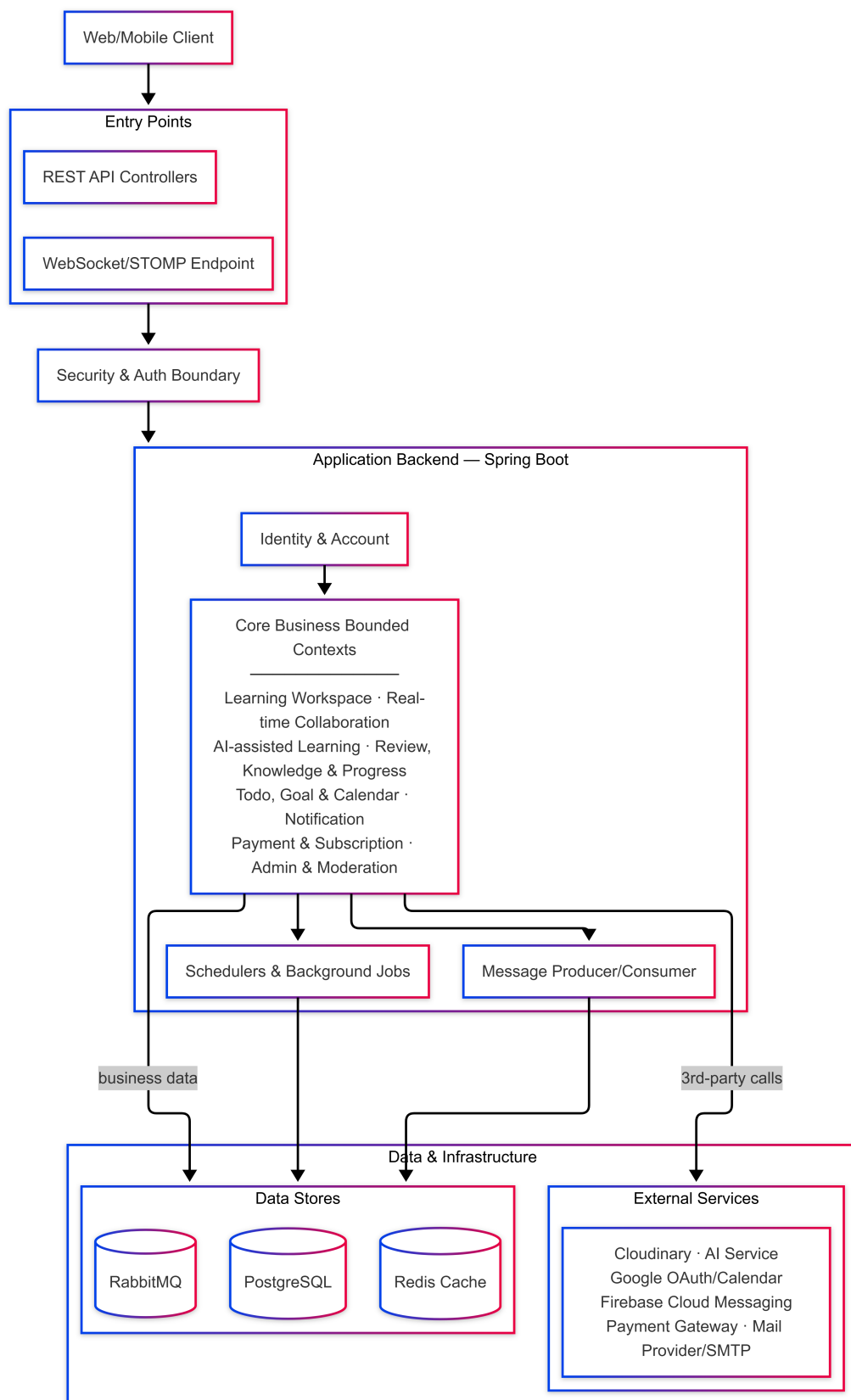
ProLearning Platform được tổ chức theo hướng phân lớp kết hợp phân rã theo miền nghiệp vụ. Ở mức tổng quan, client chỉ đảm nhiệm giao diện và trạng thái hiển thị; Spring Boot API là nơi tập trung xử lý nghiệp vụ, xác thực, phân quyền và điều phối tích hợp; các dịch vụ hạ tầng được đặt phía sau service/adapter riêng để giảm phụ thuộc trực tiếp giữa domain logic và API bên ngoài [9], [10].

Sau đây là bảng 3.1 và Bảng 3.2 tóm tắt các nhóm thành phần chính trong hệ thống.

Bảng 3.1: Tổ chức các thành phần nghiệp vụ chính

Nhóm thành phần	Vai trò	Chức năng tiêu biểu
Client layer	Giao diện tương tác và gọi API backend.	Web phục vụ quản lý học liệu; mobile phục vụ phiên học ngắn, nhắc học và push notification. REST API là kênh chính; WebSocket/STOMP dùng cho realtime [12].
Security and identity	Xác thực và kiểm soát quyền truy cập.	Đăng ký, đăng nhập, JWT, Google OAuth, role-based access và kiểm tra quyền trên tài nguyên học tập [10].
Learning workspace	Quản lý không gian học tập.	Set, folder, note, flashcard, exam, roadmap, file đính kèm, chia sẻ tài nguyên và quyền cộng tác.
AI learning	Sinh nội dung và cá nhân hóa học tập.	Tạo flashcard, đề kiểm tra, giải thích ghi chú, phân tích kết quả và tạo roadmap thông qua AI Service nội bộ.
Collaboration	Hỗ trợ học tập/cộng tác chung.	Chia sẻ note/set, kiểm tra quyền truy cập và đồng bộ chỉnh sửa ghi chú theo thời gian thực.
Review and progress	Theo dõi quá trình học.	Lưu attempt, review log, mastery, lịch ôn flashcard và thống kê tiến độ theo topic.

Hình 3.2 minh họa cách các nhóm thành phần trên liên kết với nhau trong kiến trúc chi tiết. Các dependency nghiệp vụ được định hướng đi



Hình 3.1: Kiến trúc tổng quan của hệ thống ProLearning Platform

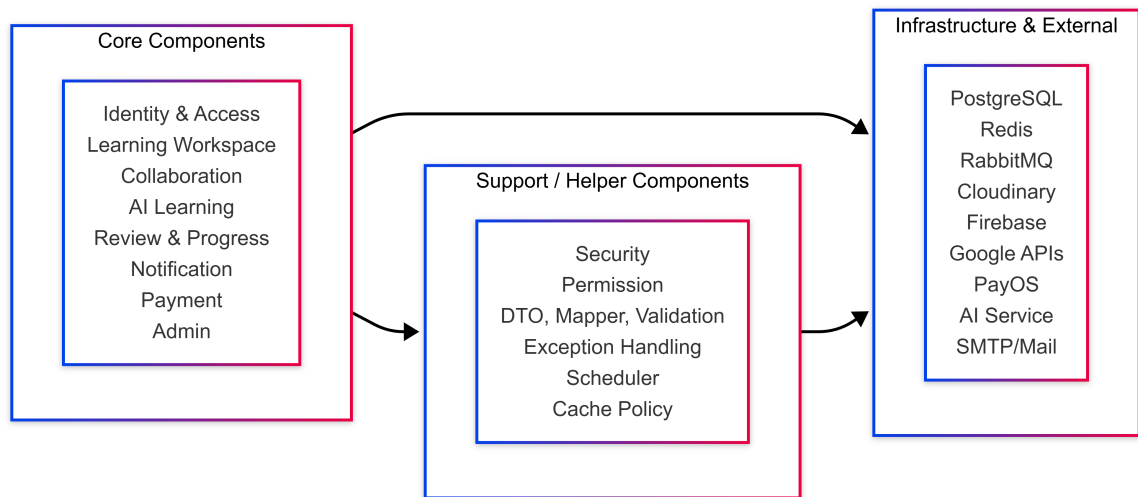
Bảng 3.2: Tổ chức các thành phần hỗ trợ và hạ tầng

Nhóm thành phần	Vai trò	Chức năng tiêu biểu
Planning and notification	Quản lý kế hoạch và nhắc học.	Todo, goal, pomodoro, reminder, weekly summary, notification trong ứng dụng, email và push notification.
Payment and subscription	Quản lý nâng cấp tài khoản.	Tạo payment link, nhận webhook, xác minh giao dịch, lưu transaction và cập nhật subscription qua PayOS [19].
Admin and moderation	Hỗ trợ vận hành nền tảng.	Quản lý tài khoản, khóa/mở khóa người dùng, onboarding analytics và xử lý appeal.
Application boundary	Chuẩn hóa request/response.	Controller, DTO, mapper, validation, exception handler và response wrapper.
Support services	Cơ chế dùng chung trong backend.	Permission service, logging, error handling, scheduler, cache helper và xử lý bất đồng bộ.
Infrastructure and integration	Lưu trữ và kết nối dịch vụ ngoài.	PostgreSQL, Redis, RabbitMQ, Cloudinary, Firebase Cloud Messaging, Google Calendar, PayOS và SMTP provider.

qua application layer, trong khi kết nối với dịch vụ ngoài được đóng gói bằng adapter/service chuyên biệt để dễ thay thế, kiểm thử và kiểm soát lỗi.

3.3 Thiết kế thành phần ứng dụng

Sau khi xác định tổ chức tổng thể ở Mục 3.2, phần này mô tả chi tiết hơn các thành phần của ProLearning Platform ở mức ứng dụng. Mục này tập trung vào vai trò của các nhóm thành phần trong một hệ thống hoàn chỉnh: lớp giao diện, lớp ứng dụng phía máy chủ, lớp dữ liệu-hạ tầng và các dịch vụ tích hợp ngoài.



Hình 3.2: Chi tiết các thành phần của hệ thống ProLearning Platform

3.3.1 Lớp giao diện người dùng

Lớp giao diện là điểm tiếp xúc trực tiếp với người dùng cuối. Hệ thống có hai loại client chính: web client phục vụ các thao tác cần không gian hiển thị rộng và mobile client phục vụ các phiên học ngắn trên thiết bị cá nhân. Cả hai client cùng sử dụng API của backend, không truy cập trực tiếp cơ sở dữ liệu hoặc dịch vụ ngoài. Nhờ vậy, các luật nghiệp vụ như phân quyền, kiểm tra trạng thái subscription, xác thực thanh toán hoặc lưu tiến độ học tập luôn được xử lý nhất quán tại máy chủ.

Bảng 3.3: Thiết kế lớp giao diện người dùng

Thành phần	Vai trò	Phạm vi xử lý
Web Client	Giao diện chính cho quản lý học liệu và thao tác học tập có nhiều nội dung.	Quản lý set, note, flashcard, exam, roadmap, todo/goal, notification, cộng tác ghi chú và thanh toán.
Mobile Client	Giao diện học tập nhanh trên thiết bị cá nhân.	Ôn flashcard, xem học liệu, theo dõi nhiệm vụ, nhận nhắc học và push notification qua Firebase Cloud Messaging [17].
Realtime Channel	Kênh giao tiếp cho chức năng cần cập nhật tức thời.	Dùng WebSocket/STOMP cho cộng tác ghi chú, trạng thái người dùng đang hoạt động và một số sự kiện realtime [12].

Về nguyên tắc thiết kế, client chỉ giữ trạng thái phục vụ hiển thị như

form, bộ lọc, dữ liệu đang xem hoặc phiên đăng nhập cục bộ. Khi người dùng thực hiện thao tác nghiệp vụ, client gửi request đến backend để kiểm tra quyền và cập nhật dữ liệu chính. Điều này giảm rủi ro sai lệch giữa web và mobile, đồng thời giúp hệ thống dễ mở rộng thêm loại client khác trong tương lai.

3.3.2 Lớp ứng dụng phía máy chủ

Backend là thành phần trung tâm của hệ thống, được xây dựng theo hướng API tập trung bằng Spring Boot. Thành phần này nhận request từ client, xác thực người dùng, kiểm tra quyền trên tài nguyên, điều phối các service nghiệp vụ và gọi tới lớp dữ liệu hoặc dịch vụ tích hợp khi cần [9], [10]. Thay vì mô tả chi tiết từng controller/service, Bảng 3.4 nhóm backend theo các miền chức năng chính.

Bảng 3.4: Thiết kế các nhóm chức năng backend

Nhóm chức năng	Trách nhiệm chính	Dữ liệu/luồng tiêu biểu
Identity and Access	Quản lý tài khoản, xác thực và phân quyền.	Đăng ký, đăng nhập, JWT/refresh token, Google OAuth, xác minh email, đặt lại mật khẩu và kiểm tra quyền truy cập tài nguyên.
Learning Workspace	Quản lý học liệu và hoạt động học tập cốt lõi.	Set, folder, note, flashcard, exam, roadmap, file đính kèm, quyền chia sẻ và dữ liệu cộng tác.
AI Learning	Điều phối các chức năng học tập thông minh.	Sinh flashcard/exam/roadmap, giải thích ghi chú, tóm tắt nội dung và phân tích điểm mạnh–điểm yếu.
Planning and Progress	Theo dõi kế hoạch và tiến độ học.	Todo, goal, Pomodoro, lịch ôn flashcard, review log, attempt và knowledge analysis.
Notification and Subscription	Duy trì tương tác và quản lý gói dịch vụ.	Tạo notification, gửi email/push, tạo payment link, nhận webhook, lưu transaction và kích hoạt quyền lợi Pro [19].

Trong mỗi nhóm chức năng, backend vẫn tuân theo cấu trúc nhiều lớp quen thuộc: controller nhận request và chuẩn hóa response; service thực hiện nghiệp vụ; repository truy cập dữ liệu quan hệ qua Spring Data JPA; adapter/service tích hợp che giấu chi tiết của hệ thống ngoài [11]. Cách tổ chức này giúp mã nguồn dễ bảo trì nhưng vẫn giữ ranh giới rõ ràng giữa xử lý giao diện, nghiệp vụ, lưu trữ và tích hợp.

3.3.3 Lớp dữ liệu và hạ tầng

Lớp dữ liệu và hạ tầng cung cấp năng lực lưu trữ, cache, xử lý nền và vận hành cho toàn bộ ứng dụng. Thiết kế phân loại dữ liệu theo vòng đời sử dụng: dữ liệu nghiệp vụ bền vững được lưu ở PostgreSQL; dữ liệu tạm thời hoặc cần truy xuất nhanh được lưu ở Redis; tác vụ không cần phản hồi tức thời được đẩy qua RabbitMQ hoặc scheduler. Bảng 3.5 tóm tắt vai trò của các thành phần chính.

Bảng 3.5: Thiết kế lớp dữ liệu và hạ tầng

Thành phần	Vai trò	Cách sử dụng trong hệ thống
PostgreSQL	Nguồn dữ liệu chính và bền vững.	Lưu user, học liệu, quyền chia sẻ, attempt, review log, notification, transaction, subscription và calendar setting [13].
Redis	Cache và trạng thái ngắn hạn.	Lưu OTP, OAuth state, token blacklist, cache dữ liệu đọc nhiều và trạng thái tạm trong các luồng xác nhận/callback [14].
RabbitMQ and Scheduler	Xử lý tác vụ nền và bất đồng bộ.	Gửi email, thông báo nền, cleanup dữ liệu, nhắc học định kỳ và các tác vụ có thể retry khi dịch vụ phụ trợ lỗi [15].
File and Asset Storage	Lưu trữ tài nguyên do người dùng tải lên.	Backend kiểm soát upload, lưu metadata trong database và lưu nội dung file/asset trên Cloudinary [16].

Trong thiết kế này, PostgreSQL là nguồn dữ liệu đáng tin cậy cho các quyết định nghiệp vụ. Redis chỉ giữ dữ liệu có thời gian sống ngắn nên không thay thế database quan hệ. RabbitMQ và scheduler giúp tách các tác vụ nền khỏi request chính, từ đó giảm độ trễ khi người dùng thao tác

và tăng khả năng phục hồi khi một dịch vụ phụ trợ phản hồi chậm hoặc tạm thời không khả dụng.

3.3.4 Tích hợp dịch vụ ngoài

Các dịch vụ ngoài được sử dụng để bổ sung năng lực mà hệ thống không cần tự triển khai hoàn toàn, chẳng hạn lưu trữ asset, gửi push notification, đồng bộ lịch và xử lý thanh toán. Tuy nhiên, backend vẫn là điểm kiểm soát trung tâm: client không gọi trực tiếp các API nhạy cảm, còn trạng thái nghiệp vụ quan trọng luôn được lưu lại trong database để có thể truy vết và khôi phục.

Bảng 3.6: Thiết kế tích hợp dịch vụ ngoài

Dịch vụ	Năng lực cung cấp	Nguyên tắc tích hợp
Cloudinary	Lưu trữ và phân phối file/asset.	Backend cấp quyền upload, kiểm tra metadata và liên kết file với học liệu tương ứng [16].
Firebase Cloud Messaging	Gửi push notification tới thiết bị.	Mobile đăng ký FCM token; backend lưu token và gửi push khi có nhắc học, todo đến hạn hoặc cập nhật quan trọng [17].
Google Calendar	Đồng bộ kế hoạch học với lịch cá nhân.	Backend dùng OAuth token để tạo/cập nhật sự kiện và lưu trạng thái đồng bộ với todo hoặc goal [18].
PayOS	Xử lý thanh toán và nâng cấp gói.	Backend tạo payment link, xác minh webhook, cập nhật transaction và kích hoạt Pro sau khi giao dịch hợp lệ [19].

Nhìn chung, thiết kế thành phần của ProLearning Platform ưu tiên phân tách trách nhiệm theo ranh giới ứng dụng thay vì dồn logic vào một lớp duy nhất. Frontend tập trung vào trải nghiệm và trạng thái hiển thị; backend kiểm soát nghiệp vụ và bảo mật; hạ tầng đảm bảo lưu trữ, cache và xử lý nền; dịch vụ ngoài được đặt sau adapter để giảm phụ thuộc trực tiếp. Cấu trúc này phù hợp với một hệ thống học tập có nhiều loại dữ liệu, nhiều kênh sử dụng và nhiều luồng tích hợp cần đảm bảo tính nhất quán.

3.4 Cài đặt hệ thống

Phần này trình bày cách ProLearning Platform được cài đặt từ thiết kế ở các mục trước. Nội dung được sắp xếp theo luồng từ phía người dùng vào hệ thống: client web/mobile, backend API, các core component, cơ sở dữ liệu, vấn đề phát sinh và tổng kết. Mục tiêu của phần này không phải mô tả chi tiết toàn bộ mã nguồn, mà tóm tắt cách các thành phần được hiện thực để người đọc hình dung được luồng xử lý và trách nhiệm của từng lớp trong hệ thống.

Về tổng thể, client web/mobile gửi yêu cầu tới backend API; backend xác thực người dùng, kiểm tra quyền, gọi service nghiệp vụ, cập nhật dữ liệu chính trong PostgreSQL và kích hoạt các tác vụ phụ như gửi thông báo, đồng bộ lịch, gọi AI Service hoặc xử lý thanh toán khi cần. Các tích hợp ngoài được đặt sau adapter/service riêng để giảm phụ thuộc trực tiếp và giữ backend làm điểm kiểm soát nghiệp vụ trung tâm.

Web/Mobile client → Backend API → Xác thực và phân quyền
→ Service nghiệp vụ → PostgreSQL/Redis → AI
Service/FCM/Calendar/PayOS/RabbitMQ

Các mục dưới đây lần lượt mô tả phần cài đặt của từng nhóm thành phần ở mức đủ để thấy cách hệ thống vận hành.

3.4.1 Cài đặt Client

Hệ thống ProLearning Platform cung cấp hai ứng dụng client cho người dùng cuối: ứng dụng di động xây dựng bằng React Native trên nền tảng Expo, và ứng dụng web xây dựng bằng React kết hợp Vite. Cả hai đều đóng vai trò trình bày giao diện, điều hướng màn hình, quản lý trạng thái phía client, lấy và lưu đệm dữ liệu từ máy chủ, lưu trữ phiên đăng nhập và kết nối tới các dịch vụ hỗ trợ; các luật nghiệp vụ và dữ liệu nghiệp vụ vẫn tập trung tại backend, hai client chỉ trao đổi với backend qua API HTTP và một số kênh thời gian thực [20], [21], [22], [23].

Hai ứng dụng được phát triển dựa trên cùng một kiến trúc phân lớp, cùng luồng xử lý dữ liệu và cùng tập tính năng nghiệp vụ; điểm khác biệt chủ yếu nằm ở công nghệ nền tảng (thư viện điều hướng, thư viện quản lý trạng thái cục bộ, API lưu trữ và API trình duyệt/thiết bị). Phần này trình bày các nội dung dùng chung cho cả hai nền tảng; các đặc thù triển khai riêng của từng nền tảng được đối chiếu trong Bảng 3.7.

Kiến trúc chung và tổ chức mã nguồn

Mã nguồn của cả hai ứng dụng được tổ chức theo trách nhiệm, tách biệt giữa lớp điều hướng, lớp trình bày, lớp trạng thái và lớp giao tiếp dữ liệu, giúp từng nhóm mã có phạm vi thay đổi rõ ràng và thuận tiện khi mở rộng tính năng:

- **Lớp điều hướng:** Định nghĩa cây màn hình/trang và phân luồng truy cập theo trạng thái xác thực. Cơ chế định tuyến cụ thể (định tuyến theo tệp hay theo cấu hình tập trung) khác nhau giữa hai nền tảng và được đối chiếu trong Bảng 3.7.
- **Lớp trình bày (components):** Tập hợp thành phần giao diện tái sử dụng và các mô-đun tính năng theo từng miền nghiệp vụ (flashcard, ghi chú, bài kiểm tra, pomodoro, công việc, mục tiêu, phân tích tri thức, nâng cấp gói). Các thành phần nền tảng như nút bấm, ô nhập liệu, thẻ nội dung và hộp thoại được chuẩn hóa để bảo đảm tính nhất quán giao diện.
- **Lớp trạng thái cục bộ:** Quản lý trạng thái không thuộc về máy chủ, ví dụ phiên đăng nhập, chủ đề giao diện, cờ onboarding và trạng thái các luồng thao tác nhiều bước. Thư viện/cách tiếp cận cụ thể khác nhau đáng kể giữa hai nền tảng và được đối chiếu trong Bảng 3.7.
- **Lớp truy vấn dữ liệu (hooks):** Cung cấp các hook tùy biến bọc quanh TanStack React Query (`useQuery`, `useInfiniteQuery`,

`useMutation`), định nghĩa khóa truy vấn, trạng thái tải, xử lý lỗi và chiến lược làm mới bộ đệm theo từng miền nghiệp vụ. Đây là điểm dùng chung tuyệt đối giữa hai nền tảng vì cùng sử dụng React Query.

- **Lớp dịch vụ (services):** Đóng gói lời gọi API tới backend theo từng miền nghiệp vụ — dựng đường dẫn, tham số, thân yêu cầu và trích xuất dữ liệu trả về — giúp tầng hook không phụ thuộc trực tiếp vào chi tiết HTTP.
- **Lớp hạ tầng dùng chung:** Cấu hình Axios client, interceptor đính kèm/làm mới token, React Query client, xử lý lỗi, ghi log và các tiện ích dùng chung.
- **Lớp dữ liệu và cấu hình:** Định nghĩa kiểu dữ liệu, lược đồ kiểm tra ràng buộc, context dùng chung, tài nguyên đa ngôn ngữ và hằng số cấu hình.

Điều hướng và phân luồng truy cập

Cả hai ứng dụng phân luồng màn hình/trang theo trạng thái xác thực: một nhóm tuyến công khai dành cho người dùng chưa đăng nhập (đăng nhập, đăng ký, quên mật khẩu, xác thực email/OTP), và một nhóm tuyến được bảo vệ chỉ mở khi người dùng đã xác thực, chứa toàn bộ khu vực chức năng chính (trang chủ, học liệu, lộ trình học, công việc và mục tiêu, pomodoro, hộp ôn tập, mục yêu thích, học liệu chia sẻ, mạng xã hội học tập). Trạng thái đăng nhập được giữ ở tầng trạng thái cục bộ và được khôi phục khi khởi động ứng dụng, nhờ đó việc chuyển đổi giữa khu vực công khai và khu vực được bảo vệ diễn ra tự động theo trạng thái xác thực thay vì điều hướng thủ công rải rác trong từng màn hình/trang.

Một số chức năng cao cấp được kiểm soát ngay tại tầng điều hướng: mục lộ trình học chỉ dành cho tài khoản PRO, khi tài khoản thường truy cập thì ứng dụng chặn điều hướng và hiển thị lời mời nâng cấp gói. Cơ

chế định tuyến cụ thể (thư viện, cách khai báo nhóm tuyến, cách gating) khác nhau giữa hai nền tảng và được đối chiếu trong Bảng 3.7.

Luồng xử lý dữ liệu

Để bảo đảm tính nhất quán và dễ bảo trì, các thao tác đọc/ghi dữ liệu trên cả hai nền tảng đều đi qua một luồng phân lớp thống nhất:

1. Thành phần giao diện gọi hook truy vấn hoặc đột biến tương ứng với nghiệp vụ.
2. Hook quản lý khóa truy vấn, trạng thái tải, lỗi và việc làm mới bộ đệm, sau đó ủy thác lời gọi cho tầng dịch vụ.
3. Hàm dịch vụ dựng yêu cầu HTTP và gọi Axios client dùng chung.
4. Interceptor yêu cầu lấy access token từ nơi lưu trữ và gắn vào tiêu đề `Authorization` trước khi gửi đi.
5. Backend xử lý yêu cầu và trả phản hồi; interceptor phản hồi xử lý lỗi chung, đặc biệt là trường hợp phiên hết hạn.
6. React Query lưu dữ liệu trả về vào bộ đệm theo khóa truy vấn và cập nhật lại giao diện theo cơ chế phản ứng.

Với các thao tác ghi, hook đột biến vô hiệu hóa những khóa truy vấn liên quan sau khi thành công để dữ liệu hiển thị được làm mới tự động; một số thao tác áp dụng cập nhật lạc quan, khôi phục về trạng thái cũ nếu thất bại. Luồng này hoàn toàn giống nhau giữa hai nền tảng vì dùng chung React Query và cùng một kiểu tổ chức service/Axios; khác biệt duy nhất nằm ở nơi lưu trữ token mà interceptor đọc ra, được trình bày tại Mục 3.4.1.

Quản lý trạng thái

Cả hai ứng dụng phân biệt rõ trạng thái máy chủ và trạng thái cục bộ. Trạng thái máy chủ được quản lý bằng TanStack React Query v5 trên cả hai nền tảng, với chính sách ưu tiên đọc từ bộ đệm và làm mới khi kết nối mạng được khôi phục; chính sách thử lại được tùy biến để không lặp lại với lỗi phía client nhưng thử lại với lỗi mạng/máy chủ kèm độ trễ tăng dần. Cả hai nền tảng đều áp dụng cơ chế lưu bền (persist) bộ nhớ đệm React Query xuống bộ nhớ cục bộ của thiết bị/trình duyệt để dữ liệu hiển thị được ngay cả khi vừa khởi động lại, trước khi đồng bộ lại với máy chủ; cơ chế lưu bền cụ thể (thư viện, nơi lưu) khác nhau theo nền tảng và được trình bày riêng.

Trạng thái cục bộ (phiên đăng nhập, chủ đề giao diện, cờ onboarding, trạng thái các luồng thao tác nhiều bước...) được quản lý theo cách tiếp cận khác nhau đáng kể giữa hai nền tảng — di động dùng nhiều store nhỏ theo chức năng, web dùng Context kết hợp hook — được đối chiếu chi tiết trong Bảng 3.7.

Xác thực và bảo mật phiên đăng nhập

Cơ chế xác thực của cả hai ứng dụng dựa trên JWT với cặp access token và refresh token, áp dụng cùng một mẫu thiết kế interceptor trên Axios client:

1. Interceptor yêu cầu đọc access token từ nơi lưu trữ và gắn vào tiêu đề **Authorization** dạng **Bearer**, ngoại trừ chính lời gọi làm mới token để tránh vòng lặp.
2. Interceptor phản hồi bắt lỗi 401, gọi API làm mới token đúng một lần cho mỗi đợt lỗi (cơ chế single-flight, tránh gọi làm mới trùng lặp khi nhiều request cùng thất bại) và xếp hàng các yêu cầu đồng thời đang chờ.

3. Khi có token mới, các yêu cầu trong hàng đợi được gửi lại với token cập nhật.
4. Nếu làm mới thất bại, ứng dụng xóa token đã lưu và đưa người dùng về luồng đăng nhập.

Ngoài đăng nhập bằng email/mật khẩu, cả hai ứng dụng hỗ trợ đăng nhập Google: mã định danh nhận được từ Google được gửi tới backend để đổi lấy cặp token của hệ thống. Nơi lưu trữ token, cách phát/khôi phục sự kiện đăng xuất và một số chi tiết triển khai khác nhau theo nền tảng, được đối chiếu trong Bảng 3.7.

Giao diện và hệ thống thiết kế

Giao diện của cả hai ứng dụng được xây dựng theo hướng hệ thống thiết kế dựa trên Tailwind CSS, dùng chung khái niệm token màu được đặt tên theo ngữ nghĩa (nền, nền nổi, viền, chữ, nhấn mạnh) thay vì mã màu cố định, hỗ trợ chế độ sáng/tối và được cung cấp xuyên suốt cây thành phần thông qua **ThemeProvider**. Riêng cách Tailwind CSS được tích hợp khác nhau theo nền tảng: di động dùng NativeWind/uniwind để áp dụng cú pháp Tailwind trong React Native, còn web dùng Tailwind CSS v4 tích hợp trực tiếp qua plugin Vite.

Tính năng nghiệp vụ chính

Cả hai ứng dụng cung cấp đầy đủ các tính năng học tập chính của hệ thống, mỗi tính năng được tổ chức thành mô-đun riêng và liên kết với các tuyến tương ứng:

- **Flashcard và ôn tập lặp lại:** Quản lý bộ thẻ, học theo phiên, lật thẻ, chơi ghép cặp và đồng bộ tiến trình học lên máy chủ. Người dùng có thể tạo thẻ thủ công, nhập từ tệp hoặc sinh thẻ bằng AI từ ghi chú, tài liệu và đường dẫn web.

- **Ghi chú với trình soạn thảo phong phú và cộng tác thời gian thực:** Sử dụng BlockNote làm trình soạn thảo, hỗ trợ tiêu đề, danh sách, bảng, danh sách việc cần làm, ảnh, lưu tự động có debounce và các thao tác AI như giải thích hoặc tóm tắt nội dung. Cộng tác soạn thảo nhiều người được triển khai theo mô hình CRDT bằng Yjs kết hợp Hocuspocus: mỗi ghi chú tương ứng một phòng cộng tác định danh theo mã ghi chú, kết nối qua WebSocket với phiên được xác thực bằng token có khả năng làm mới; cơ chế awareness của Yjs truyền trạng thái hiện diện (người dùng, vị trí con trỏ) của các thành viên đang cùng chỉnh sửa, nhờ đó các chỉnh sửa đồng thời được hợp nhất tự động mà không cần khóa tài liệu. Vì BlockNote vốn là thư viện cho nền web, trên di động trình soạn thảo được nhúng qua WebView để tái sử dụng, trong khi trên web trình soạn thảo và kết nối Hocuspocus chạy trực tiếp không cần lớp trung gian này (chi tiết trong Bảng 3.7).
- **Bài kiểm tra và đề thi:** Hỗ trợ câu hỏi trắc nghiệm, đúng/sai và tự luận; làm bài theo phiên có tính giờ; xem kết quả, đáp án và giải thích sau khi hoàn thành.
- **Pomodoro:** Cung cấp bộ đếm thời gian tập trung, tự động chuyển phiên, trình trộn âm thanh nền, danh sách âm thanh yêu thích và thống kê theo tuần.
- **Công việc, mục tiêu và lộ trình học:** Quản lý công việc có độ ưu tiên, hạn hoàn thành và liên kết với mục tiêu; tài khoản PRO có thể tạo và theo dõi lộ trình học theo chủ đề.
- **Chia sẻ, cộng tác và phân tích tri thức:** Hỗ trợ khám phá học liệu công khai, chia sẻ ghi chú/flashcard/bài kiểm tra, phân quyền thành viên, lưu mục yêu thích, tạo hộp ôn tập và xem phân tích điểm mạnh/điểm yếu bằng AI.

Thông báo

Cả hai ứng dụng đều cần thông báo cho người dùng về các sự kiện của hệ thống (lời mời cộng tác, nhắc nhở, kết quả thao tác...), tuy nhiên cơ chế truyền tải khác biệt căn bản theo năng lực nền tảng: di động sử dụng thông báo đẩy hệ điều hành thông qua Firebase Cloud Messaging, còn web chỉ hiển thị thông báo trong ứng dụng thông qua truy vấn định kỳ, không sử dụng Service Worker/Web Push. Chi tiết triển khai từng nền tảng được đối chiếu trong Bảng 3.7.

Tích hợp dịch vụ ngoài

Tương tự backend, cả hai ứng dụng client đều cô lập việc tích hợp dịch vụ bên thứ ba trong các dịch vụ/hook chuyên trách, giúp hạn chế phạm vi thay đổi khi nhà cung cấp hoặc định dạng API thay đổi. Ba tích hợp được triển khai trên cả hai nền tảng với cùng bản chất nghiệp vụ nhưng khác cơ chế vận chuyển cụ thể:

- **Tải tệp lên Cloudinary (signed direct upload):** Client lấy chữ ký tải lên từ backend (`signature`, `timestamp`, `apiKey`, `cloudName`, `assetId...`), gửi tệp trực tiếp lên Cloudinary bằng yêu cầu HTTP riêng (không qua client Axios có gắn JWT), sau đó callback xác nhận về backend để cập nhật thông tin tệp. File được truyền thẳng từ client lên Cloudinary, backend chỉ ký và xác nhận chứ không đóng vai trò proxy.
- **Thanh toán qua PayOS:** Client yêu cầu backend tạo đơn nâng cấp gói (backend phân biệt nền tảng tạo đơn), mở trang thanh toán PayOS, sau đó quay lại ứng dụng và kiểm tra trạng thái đơn theo chu kỳ cho tới khi đơn được xác nhận hoặc hết hạn.
- **Đồng bộ Google Calendar:** Ứng dụng hỗ trợ kết nối tài khoản Google Calendar theo luồng OAuth, bật/tắt đồng bộ và kiểm tra

trạng thái kết nối để phục vụ việc đồng bộ công việc và mục tiêu lên lịch.

Cách mở trang thanh toán/uỷ quyền OAuth và cách quay lại ứng dụng sau khi hoàn tất khác nhau rõ rệt giữa hai nền tảng (mở trình duyệt hệ thống và quay lại qua liên kết sâu trên di động, so với điều hướng/cửa sổ popup trên web); chi tiết được đối chiếu trong Bảng 3.7.

Đa ngôn ngữ

Cả hai ứng dụng hỗ trợ song ngữ Việt–Anh thông qua i18next kết hợp react-i18next, lưu lại lựa chọn ngôn ngữ của người dùng để bảo đảm trải nghiệm nhất quán giữa các phiên sử dụng. Cách tổ chức tài nguyên dịch (tách namespace hay dùng một namespace mặc định) và thứ tự ưu tiên xác định ngôn ngữ ban đầu (có dùng ngôn ngữ hệ thống/trình duyệt hay không) khác nhau theo nền tảng, được đối chiếu trong Bảng 3.7.

3.4.2 Đặc thù triển khai: di động so với web

Mục này đối chiếu các điểm khác biệt triển khai giữa hai nền tảng — những phần không dùng chung được do khác biệt về công nghệ nền tảng hoặc năng lực hệ điều hành. Các nội dung kiến trúc, luồng dữ liệu và tính năng nghiệp vụ dùng chung đã được trình bày tại Mục 3.4.1.

Bảng 3.7: Đối chiếu triển khai di động (React Native/Expo) và web (React/Vite)

Tiêu chí	Di động	Web
Nền tảng & điều hướng	Expo Router, điều hướng theo tệp trong <code>app/</code> ; kết hợp Drawer (cấp cao) và Stack (luồng chi tiết); <code>app/_layout.tsx</code> dùng <code>Stack.Protected</code> chia 4 nhánh (khởi động, onboarding, (auth), (drawer)); hỗ trợ deep link	React 19 + Vite 7 (không Next.js/CRA); <code>react-router-dom v7</code> (<code>BrowserRouter + useRoutes</code>), cấu hình tuyến tập trung 1 tệp, chia 3 nhóm: <code>public/protected/admin</code>
Trạng thái cục bộ	Nhiều store Zustand nhỏ theo chức năng (phiên đăng nhập, onboarding, luồng tạo flashcard/test, pomodoro...); một số dùng <code>persist + AsyncStorage</code> , cờ <code>_hasHydrated</code> tránh sai trạng thái khi chưa nạp xong	Không Redux/Zustand — React Context + hook tùy biến (<code>useAuth</code> , <code>ThemeProvider</code>); cache React Query persist xuống <code>localStorage</code> qua thư viện <code>query-sync-storage-persister</code>
Xác thực: lưu token & đăng xuất	Token lưu Expo SecureStore qua lớp <code>tokenStorage</code> ; đồng bộ đăng xuất bằng <code>authEventEmitter</code> (observer) — tầng API phát sự kiện khi phiên hết hạn/refresh thất bại	Token lưu <code>localStorage</code> (<code>token</code> , <code>refreshToken</code> , <code>user</code>), không cookie/session server; refresh dùng single-flight; thất bại thì xóa token, điều hướng <code>/login</code> (trừ tuyến công khai)
Thông báo	Push notification qua <code>expo-notifications</code> + Firebase Cloud Messaging (Android); đăng ký token thiết bị sau đăng nhập; riêng Pomodoro có nút tạm dừng/tiếp tục/dừng ngay trên thông báo	Chỉ in-app, không Web Push/Service Worker; polling React Query mỗi 60s, hiển thị qua chuông thông báo + toast

Tiêu chí	Di động	Web
Soạn thảo ghi chú & cộng tác	BlockNote (thư viện web) được nhúng qua WebView để tái sử dụng; kết nối Hocuspocus/WebSocket khởi tạo bên trong WebView	BlockNote chạy trực tiếp, có bản dự phòng khi mất kết nối cộng tác; Hocuspocus/WebSocket khởi tạo trực tiếp, token lấy từ <code>localStorage</code>
Tích hợp Cloudinary	Lấy chữ ký từ backend, tải trực tiếp ảnh/âm thanh/tài liệu lên Cloudinary, theo dõi tiến độ, callback xác nhận	Cùng mô hình signed direct upload qua Axios instance riêng (không gắn interceptor JWT); có thêm luồng tải tệp từ URL có sẵn
Tích hợp PayOS	Mở trang thanh toán bằng <code>expo-web-browser</code> (trình duyệt hệ thống), quay lại qua deep link, polling kiểm tra trạng thái đơn	Lưu mã đơn vào <code>sessionStorage</code> , điều hướng cùng tab sang PayOS (không tab mới/iframe); polling mỗi 2s tới khi kết thúc hoặc hết giới hạn lần kiểm tra
Tích hợp Google Calendar	Luồng OAuth mở trong trình duyệt hệ thống/ứng dụng, quay lại qua deep link	Mở cửa sổ popup xác thực; trang callback đọc kết quả qua query param rồi tự đóng popup hoặc điều hướng về trang hồ sơ
Đa ngôn ngữ	i18next, tài nguyên dịch chia nhiều namespace theo module; ưu tiên: đã lưu cục bộ → hồ sơ người dùng → ngôn ngữ thiết bị (<code>expo-localization</code>) → mặc định	i18next, một namespace mặc định (không chia module); ưu tiên <code>localStorage</code> → mặc định tiếng Việt; không dùng ngôn ngữ trình duyệt

Tiêu chí	Di động	Web
Build & phát hành	Cấu hình tập trung <code>app.json</code> (deep link, bundle ID, plugin, quyền); biến môi trường <code>EXPO_PUBLIC_</code> ; build bằng EAS (development/preview/production); cập nhật OTA qua <code>expo-updates</code> ; Metro + NativeWind	<code>tsc -b</code> rồi <code>vite build</code> (kiểm tra kiểu trước khi đóng gói); Tailwind CSS v4 qua <code>@tailwindcss/vite</code> ; không Webpack

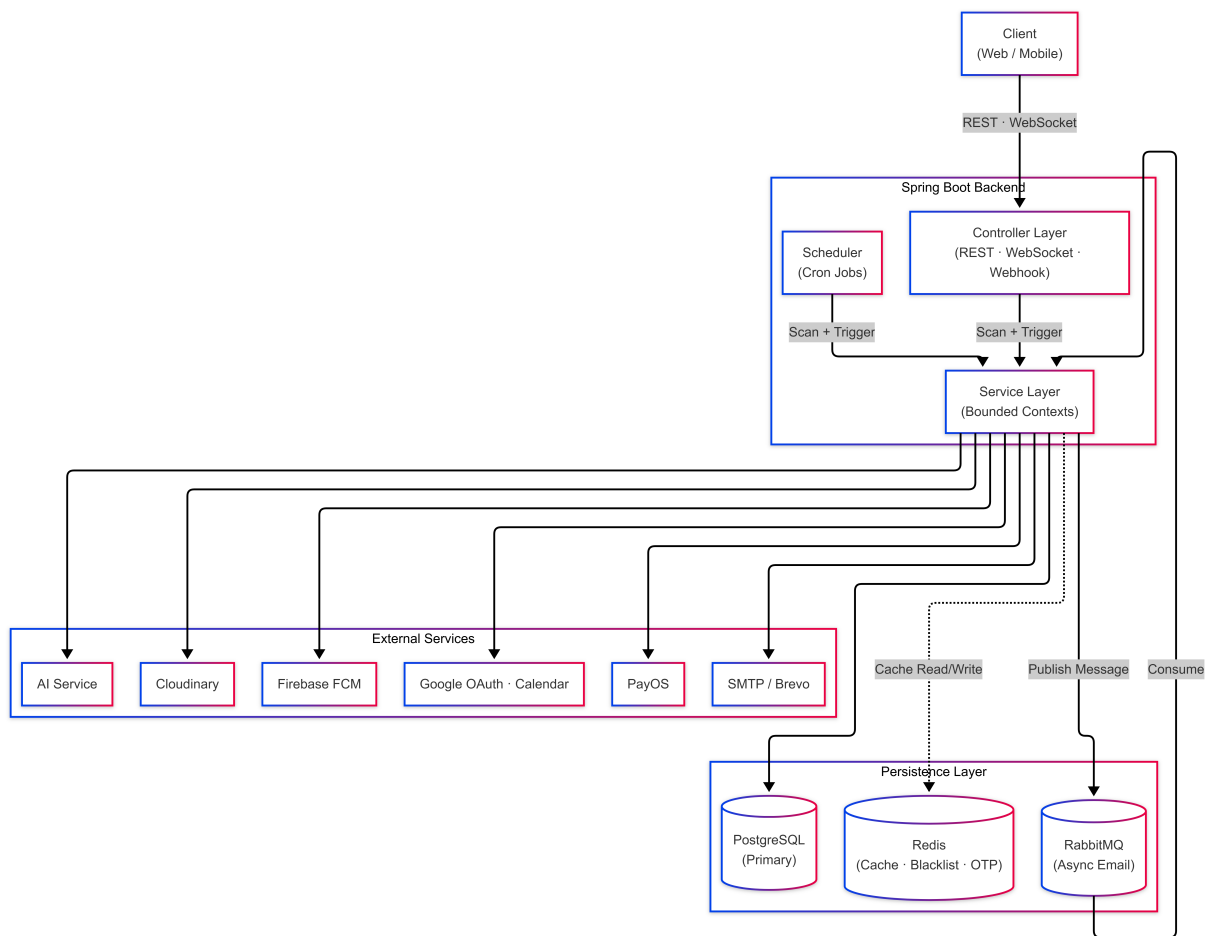
3.4.3 Cài đặt backend

Backend của ProLearning Platform được triển khai bằng Spring Boot và đóng vai trò điểm điều phối chính của hệ thống. Trong phạm vi hiện tại, hệ thống sử dụng một backend server chính để nhận request từ web/mobile, xác thực người dùng, kiểm tra quyền truy cập tài nguyên, thực hiện nghiệp vụ, lưu dữ liệu vào PostgreSQL và gọi các dịch vụ hạ tầng hoặc dịch vụ ngoài khi cần [9], [10], [11]. Nếu số lượng người dùng tăng, backend có thể được mở rộng theo chiều ngang bằng cách chạy thêm các instance cùng loại; khi đó các trạng thái dùng chung như dữ liệu nghiệp vụ, cache, hàng đợi và token ngắn hạn vẫn được quản lý thông qua PostgreSQL, Redis và RabbitMQ thay vì phụ thuộc vào bộ nhớ cục bộ của từng instance.

Về cấu trúc, backend được chia thành các lớp quen thuộc: controller nhận request và trả response; service cài đặt workflow nghiệp vụ; repository truy cập dữ liệu; DTO/mapper tách dữ liệu API khỏi entity; configuration/infrastructure cấu hình security, scheduler, Redis, RabbitMQ, WebSocket và các service tích hợp. Cách chia này giúp mỗi thay đổi thường chỉ ảnh hưởng tới một lớp hoặc một bounded context cụ thể.

Bảng 3.8 tóm tắt các thành phần backend chính theo trách nhiệm và vấn đề mà mỗi thành phần giải quyết.

Luồng request đồng bộ thường bắt đầu từ JWT/security filter, đi qua controller, sau đó vào service để kiểm tra quyền và thực hiện transaction.



Hình 3.3: Kiến trúc backend của hệ thống ProLearning Platform

Bảng 3.8: Tóm tắt tổ chức các lớp và thành phần backend

Lớp/thành phần	Trách nhiệm chính	Vấn đề giải quyết
Security filter	Xác thực JWT, khôi phục thông tin người dùng và chặn request không hợp lệ trước khi vào controller.	Tập trung kiểm soát truy cập, tránh lặp logic xác thực trong từng API.
Controller và DTO	Nhận request, kiểm tra dữ liệu đầu vào cơ bản, ánh xạ request/response theo hợp đồng API.	Tách định dạng API khỏi entity nội bộ và chuẩn hóa phản hồi cho client.
Service nghiệp vụ	Cài đặt workflow chính, kiểm tra quyền trên tài nguyên, điều phối repository và adapter.	Giữ luật nghiệp vụ ở một nơi, bảo đảm transaction và tính nhất quán dữ liệu.
Repository	Truy vấn và cập nhật dữ liệu quan hệ thông qua Spring Data JPA.	Cô lập chi tiết truy cập PostgreSQL khỏi logic nghiệp vụ.
Mapper	Chuyển đổi giữa entity, DTO và các mô hình dữ liệu trung gian.	Giảm phụ thuộc giữa tầng API, tầng nghiệp vụ và tầng lưu trữ.
Adapter tích hợp	Bao gói lời gọi AI Service, FCM, Google Calendar, Cloudinary và PayOS.	Cô lập API bên ngoài, giúp dễ thay thế, retry và xử lý lỗi tích hợp.
Scheduler/Async	Thực thi các tác vụ nền như gửi thông báo, email, reminder, cleanup và đồng bộ phụ trợ.	Giảm thời gian phản hồi request chính và hỗ trợ xử lý lại khi dịch vụ phụ trợ chậm hoặc lỗi.

Những thao tác không cần hoàn tất ngay trong request chính như gửi email, gửi push notification, cleanup dữ liệu, reminder định kỳ hoặc một số bước tích hợp được chuyển sang scheduler/async để giảm thời gian phản hồi. Với luồng realtime như cộng tác ghi chú, backend dùng WebSocket/STOMP để publish sự kiện theo topic thay vì yêu cầu client polling liên tục [12].

Các hệ thống ngoài như AI Service, Firebase Cloud Messaging, Google Calendar, Clouldinary và PayOS không được gọi trực tiếp từ controller mà được bọc sau service/adapter riêng. Nhờ đó, backend vẫn giữ vai trò source of truth cho trạng thái nghiệp vụ, trong khi chi tiết request/response của từng nhà cung cấp được cô lập ở lớp tích hợp.

3.4.4 Cài đặt các core component tiêu biểu

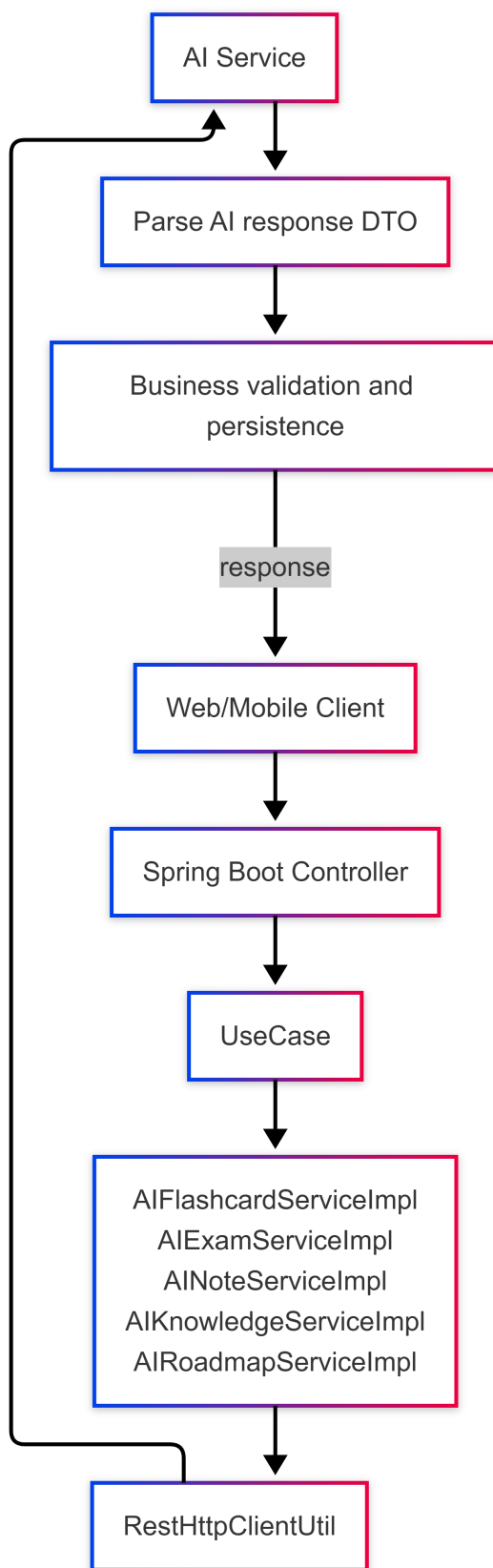
Phần này tóm tắt các flow quan trọng nhất của hệ thống: điều phối AI Service, ôn tập flashcard theo SM-2, cộng tác realtime trên ghi chú, notification pipeline, đồng bộ Google Calendar và thanh toán PayOS. Các flow này được chọn vì thể hiện rõ cách backend phối hợp giữa domain service, persistence, scheduler, realtime channel và dịch vụ ngoài.

Điều phối AI Service cho học liệu thông minh

AI Service được tách khỏi backend chính để cô lập phần sinh nội dung và phân tích ngôn ngữ. Backend nhận yêu cầu tạo flashcard, exam, giải thích note, phân tích kiến thức hoặc sinh roadmap; sau đó kiểm tra quyền, chuẩn hóa dữ liệu đầu vào, gọi AI Service qua adapter và chỉ lưu kết quả sau khi đã kiểm tra định dạng. Nhờ vậy, client không gọi trực tiếp AI Service và các học liệu được sinh ra vẫn nằm trong workflow quản lý học liệu của backend.

Thuật toán SM-2 cho flashcard review

Flashcard review được cài đặt dựa trên biến thể SM-2 để lên lịch ôn tập cá nhân [24]. Mỗi thẻ lưu trạng thái như `nextReviewAt`, `intervalDays`,



Hình 3.4: Luồng điều phối AI Service

`easeFactor` và `repetitions`; backend dùng các trường này để lấy thẻ đến hạn, ưu tiên thẻ có chu kỳ ngắn và bổ sung thẻ mới khi cần. Sau mỗi lượt học, service cập nhật lại lịch ôn, độ dễ và trạng thái nắm/không nắm thẻ để phục vụ các phiên review tiếp theo.

Cộng tác thời gian thực trên ghi chú

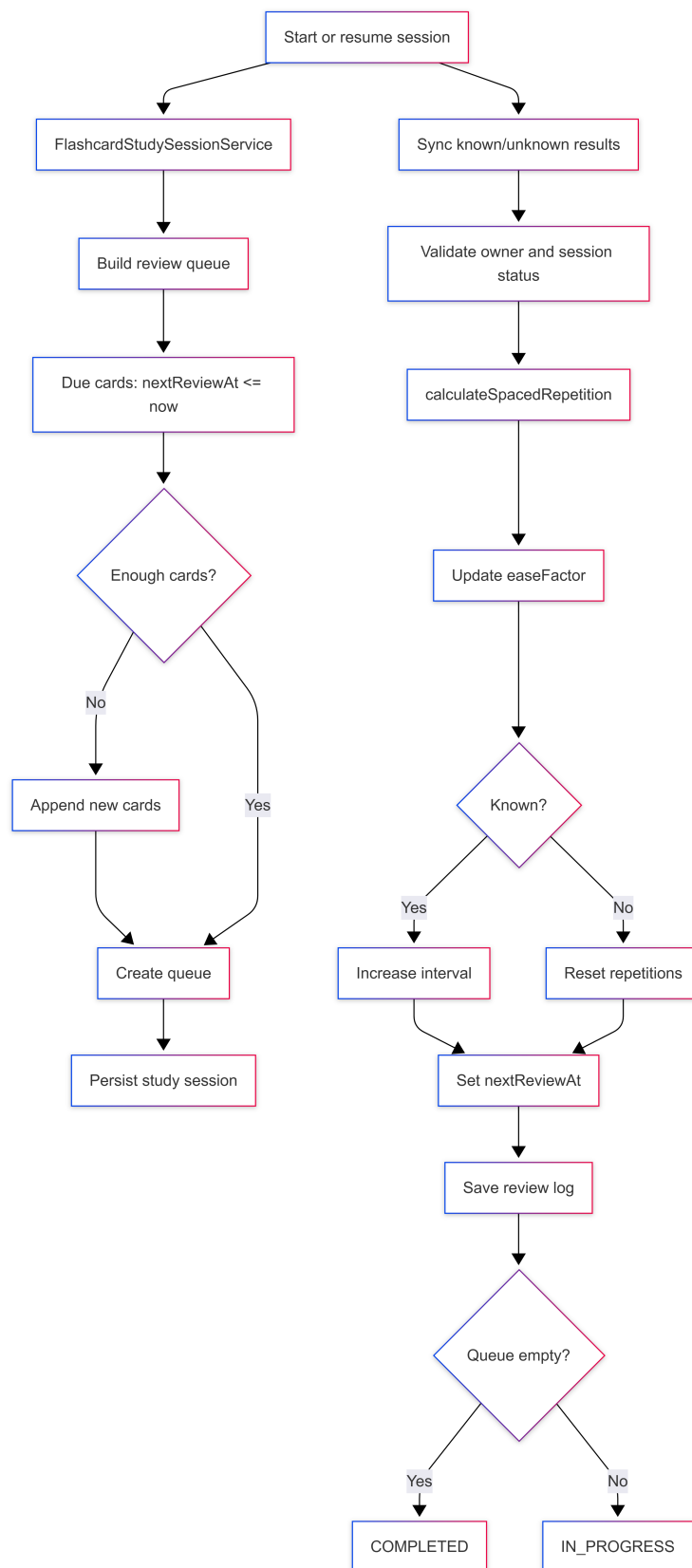
Cộng tác ghi chú sử dụng WebSocket/STOMP để truyền sự kiện theo topic và Yjs để hợp nhất chỉnh sửa đồng thời [12]. Backend kiểm tra quyền tham gia note, quản lý kênh realtime, nhận snapshot cần lưu bền vững và lưu lại trạng thái có thể khôi phục trong PostgreSQL. Presence và cursor chỉ giữ ngắn hạn để giảm tải ghi database, còn nội dung chính của note được lưu theo snapshot.

Pipeline thông báo đa nguồn và gửi push bất đồng bộ

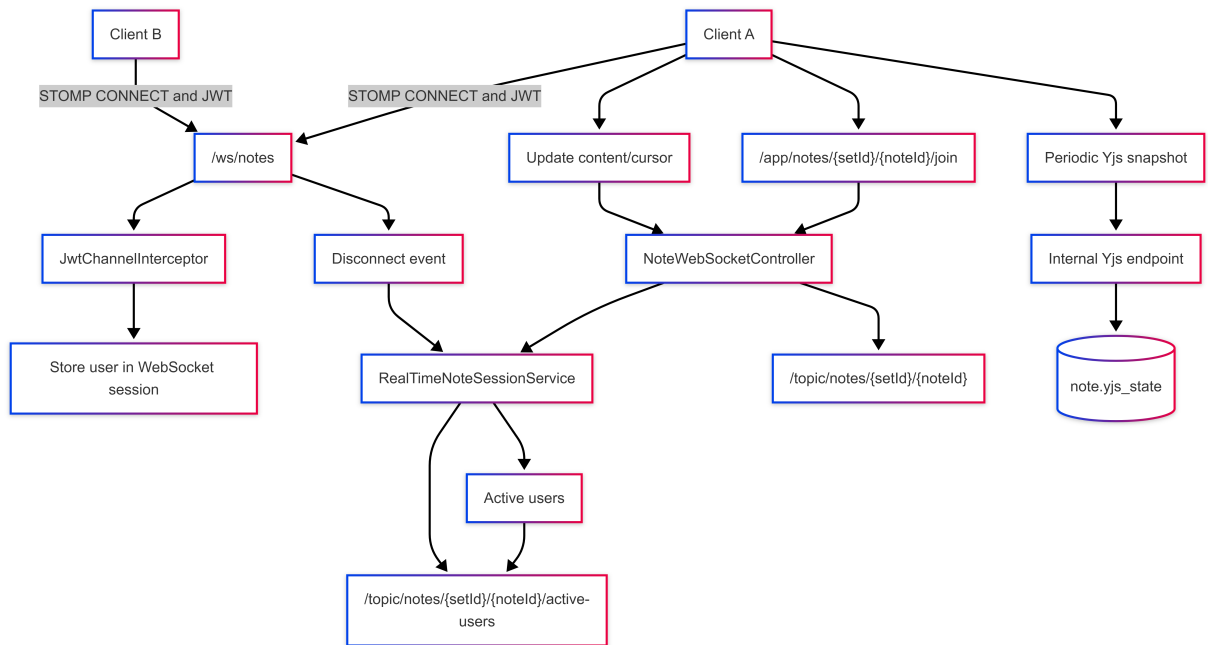
Notification pipeline nhận sự kiện từ nhiều nguồn như flashcard đến hạn, todo/goal sắp hết hạn, weekly summary hoặc thay đổi subscription. Backend chuẩn hóa thông báo, lưu inbox vào PostgreSQL và gửi push qua Firebase Cloud Messaging khi người dùng có thiết bị đã đăng ký [17]. Các bước gửi push/email có thể chạy bất đồng bộ để không làm chậm request chính.

Đồng bộ Todo với Google Calendar

Đồng bộ Google Calendar được triển khai thông qua service tích hợp riêng. Backend tạo OAuth authorization URL, lưu `state` ngắn hạn trong Redis, nhận callback để lưu credential hợp lệ và dùng Google Calendar API để tạo, cập nhật hoặc xóa event tương ứng với todo có ngày đến hạn [14], [18]. Trạng thái đồng bộ và mã event được lưu lại để backend có thể đối soát khi todo thay đổi.



Hình 3.5: Luồng học và ôn tập flashcard theo SM-2



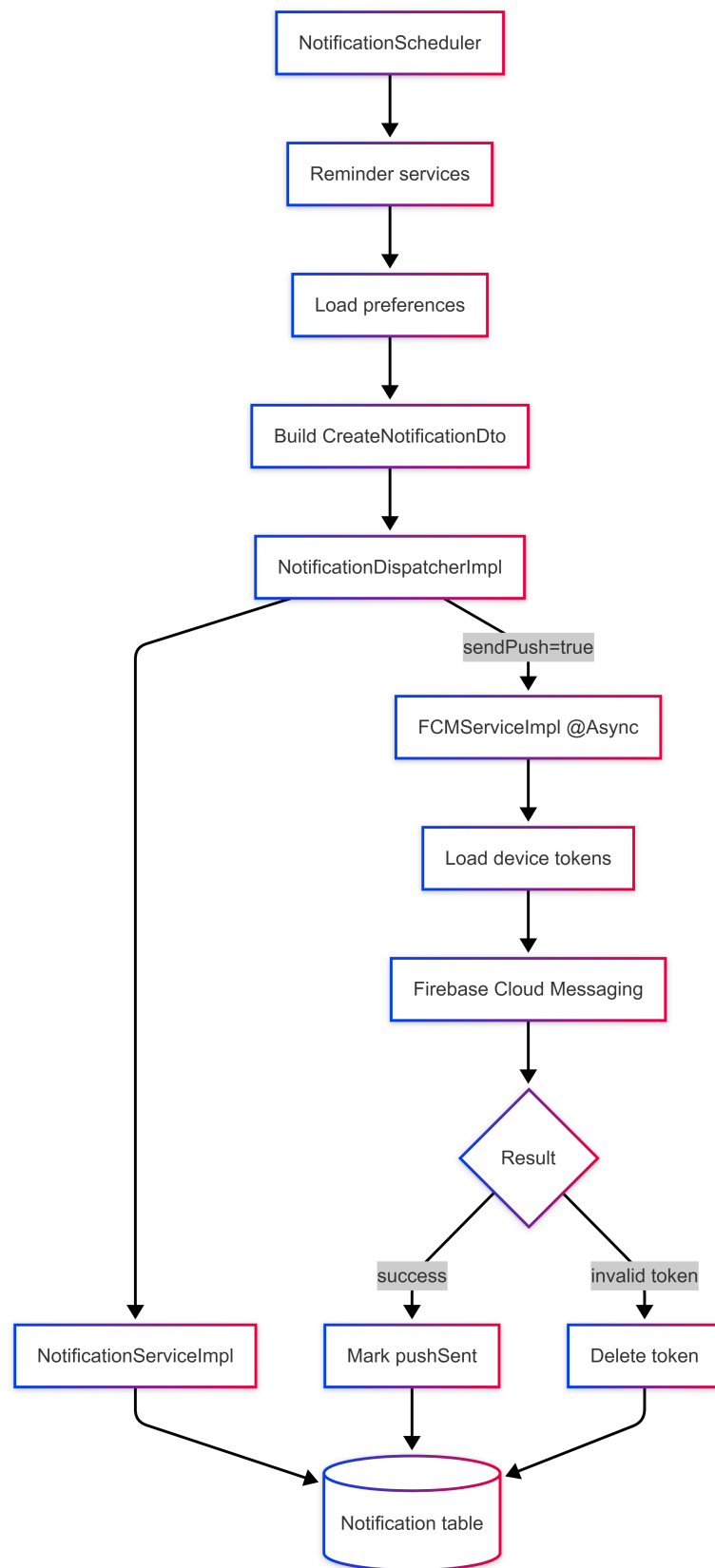
Hình 3.6: Luồng cộng tác thời gian thực trên ghi chú

Thanh toán PayOS và kích hoạt tài khoản PRO

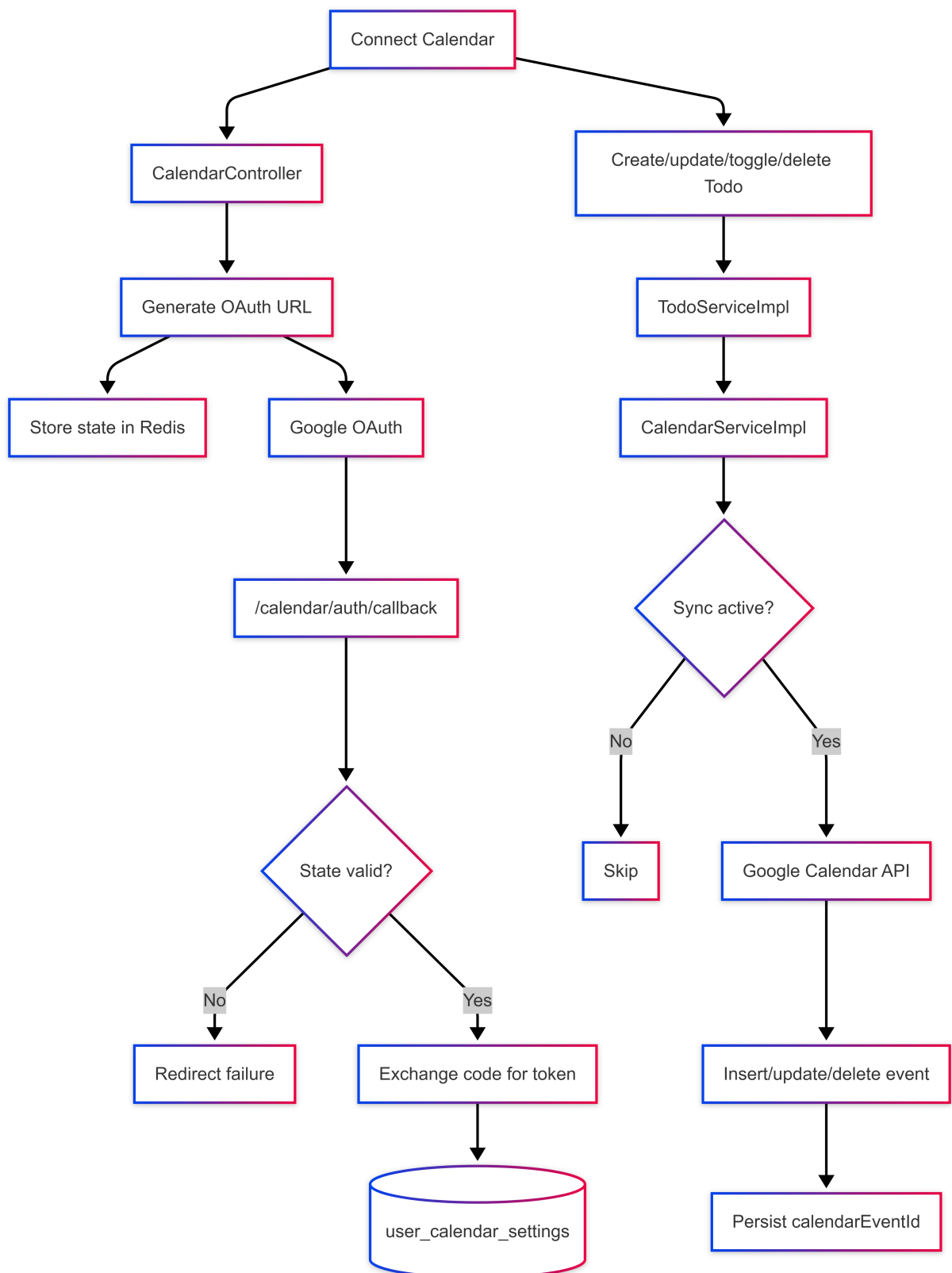
Luồng thanh toán được cài đặt theo hướng backend tạo payment link, lưu order ở trạng thái chờ và chỉ kích hoạt PRO sau khi nhận webhook hợp lệ từ PayOS. Webhook được xác minh chữ ký, đối chiếu order code và xử lý theo hướng idempotent để tránh cập nhật trùng hoặc payload giả mạo [19]. Sau khi giao dịch thành công, backend lưu transaction, cập nhật subscription và phát sinh notification/email xác nhận.

3.4.5 Cài đặt cơ sở dữ liệu và lưu trữ trạng thái

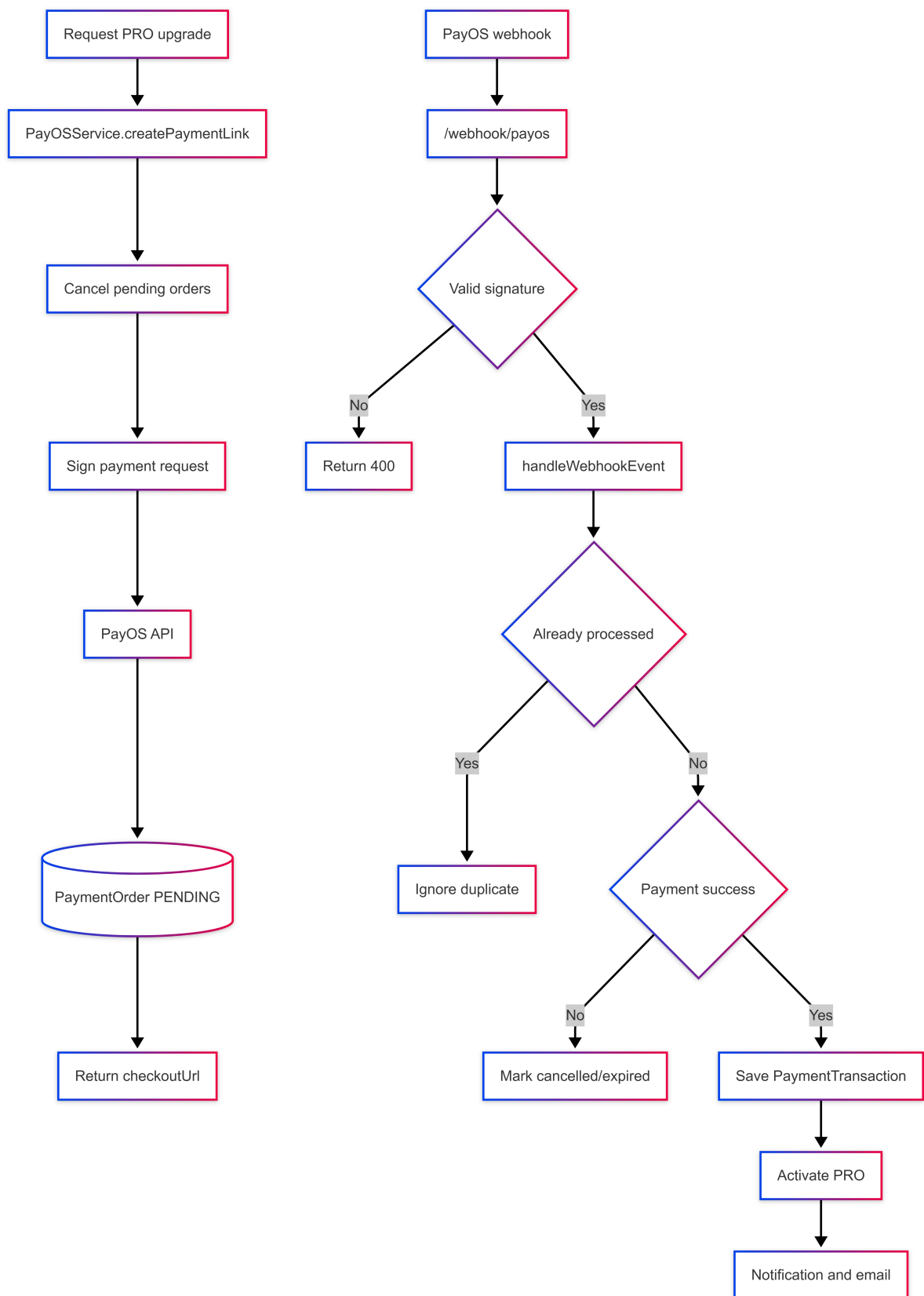
PostgreSQL là nguồn dữ liệu chính của hệ thống, dùng để lưu tài khoản, học liệu, note, flashcard, exam, todo, notification, subscription, payment order và các dữ liệu cần truy vết lâu dài [13]. Redis được dùng cho dữ liệu ngắn hạn như OAuth state, cache, token blacklist hoặc giá trị có TTL; RabbitMQ hỗ trợ các tác vụ bất đồng bộ như email, reminder và một số bước xử lý nền [14], [15].



Hình 3.7: Luồng xử lý notification pipeline



Hình 3.8: Luồng đồng bộ Todo với Google Calendar



Hình 3.9: Luồng thanh toán PayOS và nâng cấp tài khoản PRO

Thiết kế dữ liệu được tổ chức theo bounded context. Nhóm identity/subscription lưu người dùng, vai trò và quyền lợi gói; nhóm learning workspace lưu note, folder, file, flashcard, exam và roadmap; nhóm progress lưu review log, attempt, session và phân tích kiến thức; nhóm todo/calendar lưu nhiệm vụ và mã event đồng bộ; nhóm notification/payment lưu inbox, trạng thái gửi push, order và transaction. Các tài nguyên đều gắn với chủ sở hữu hoặc workspace để service có thể kiểm tra quyền trước khi đọc/ghi.

Những flow có vòng đời dài như payment, subscription, flashcard review và notification dùng trường trạng thái rõ ràng thay vì suy diễn từ nhiều bản ghi rời rạc. Với dữ liệu tích hợp ngoài, backend lưu đủ metadata như event id, order code, token hoặc trạng thái đồng bộ để có thể retry, đối soát hoặc thu hồi khi cần. Dữ liệu realtime ngắn hạn như presence/cursor không ghi liên tục vào database; chỉ snapshot hoặc trạng thái cần khôi phục mới được lưu bền vững.

3.4.6 Các vấn đề cần xử lý và giải pháp

Trong quá trình cài đặt, các vấn đề chính đến từ việc phối hợp giữa request đồng bộ, tác vụ bất đồng bộ, realtime data và external API. Bảng 3.9 tóm tắt các vấn đề quan trọng cùng hướng xử lý ở mức triển khai.

Giải pháp chung là giữ backend làm điểm kiểm soát nghiệp vụ, adapter hóa tích hợp ngoài, phân loại dữ liệu theo vòng đời và không đưa logic quyết định trạng thái cuối cùng xuống client.

3.4.7 Tổng kết cài đặt hệ thống

Qua các phần cài đặt, hệ thống được hiện thực theo một số nguyên tắc nhất quán: backend là điểm kiểm soát nghiệp vụ và bảo mật; web/mobile tập trung vào trải nghiệm và gọi API; PostgreSQL lưu dữ liệu bền vững; Redis và bộ nhớ ứng dụng lưu trạng thái ngắn hạn; RabbitMQ, scheduler và

Bảng 3.9: Các vấn đề triển khai và giải pháp xử lý

Vấn đề	Giải pháp
Request cần phản hồi nhanh nhưng nhiều flow có bước gửi email, push hoặc gọi dịch vụ ngoài.	Tách phần bắt buộc trong transaction chính khỏi phần có thể chạy async/scheduler; chỉ trả kết quả sau khi dữ liệu lõi đã được cập nhật an toàn.
Client không được phép tự ghi trạng thái quan trọng như payment, subscription hoặc lịch ôn tập.	Backend giữ vai trò source of truth; mọi cập nhật trạng thái quan trọng đi qua service nghiệp vụ, kiểm tra quyền và transaction.
External API có thể timeout, trả lỗi hoặc gửi webhook trùng lặp.	Bọc tích hợp sau adapter, lưu trạng thái trung gian, xác minh chữ ký webhook và xử lý idempotent theo order/event id.
Realtime collaboration tạo nhiều sự kiện ngắn hạn.	Giữ presence/cursor trong bộ nhớ hoặc kênh realtime; chỉ lưu snapshot hoặc dữ liệu cần khôi phục vào PostgreSQL.
OAuth state, token và cache có vòng đời khác nhau.	Dùng Redis TTL cho dữ liệu ngắn hạn; lưu credential cần thiết trong bảng settings và kiểm soát refresh/revoke ở service tích hợp.
Notification đến từ nhiều nguồn nên dễ trùng logic tạo/gửi.	Chuẩn hóa qua dispatcher; inbox lưu trong PostgreSQL, còn push/email được gửi bất đồng bộ theo preference và device token.

async executor xử lý tác vụ nền; các dịch vụ như AI Service, FCM, Google Calendar, Clouddinary và PayOS được đặt sau adapter/service riêng.

Cách triển khai này giúp cùng một nghiệp vụ có thể dùng lại cho nhiều loại client, hạn chế trùng lặp logic ở giao diện và giúp hệ thống dễ mở rộng khi bổ sung tính năng học tập mới hoặc thay đổi nhà cung cấp dịch vụ tích hợp.

3.5 Tổng kết

Chương này đã trình bày kiến trúc, tổ chức thành phần, thiết kế dữ liệu, tích hợp hạ tầng và các luồng cài đặt tiêu biểu của ProLearning Platform. Với Spring Boot API làm trung tâm, hệ thống đáp ứng yêu cầu thiết kế về phân tách trách nhiệm, kiểm soát nghiệp vụ tại backend, lưu trữ dữ liệu nhất quán và tích hợp được các dịch vụ cần thiết cho học tập cá nhân hóa, nhắc học, cộng tác và thanh toán. Các luồng cài đặt tiêu biểu cũng cho thấy hệ thống có thể phục vụ đồng thời web và mobile trong khi vẫn duy trì tính mở rộng, khả năng bảo trì và khả năng thay thế từng thành phần tích hợp khi cần.

Chương 4

Quản trị dự án

4.1 Công cụ sử dụng

Trong quá trình thực hiện đề tài, nhóm đã sử dụng một số công cụ hỗ trợ nhằm đảm bảo quá trình phát triển diễn ra có tổ chức, hiệu quả và dễ dàng cộng tác. Các công cụ được lựa chọn dựa trên nhiều tiêu chí như: sự quen thuộc và kinh nghiệm sử dụng trước đó của các thành viên, tính năng phong phú đáp ứng nhu cầu của nhóm, cũng như mức độ phổ biến trong cộng đồng phát triển phần mềm hiện nay.

Cụ thể, các công cụ được nhóm sử dụng bao gồm:

- **Jira**: Được sử dụng để quản lý công việc (task management), theo dõi tiến độ các sprint và phân công nhiệm vụ cho từng thành viên.
- **GitHub**: Được sử dụng để quản lý mã nguồn (source control). Nhóm tổ chức thành 3 repository riêng biệt: một cho back-end, một cho front-end web và một cho front-end mobile. GitHub cũng được sử dụng trong quy trình tạo pull request và review code.
- **draw.io**: Được sử dụng để thiết kế các sơ đồ kiến trúc hệ thống, sơ đồ luồng dữ liệu và các biểu đồ kỹ thuật khác.
- **Google Meet**: Được sử dụng làm nền tảng họp trực tuyến hàng

tuần giữa các thành viên trong nhóm cũng như giữa nhóm và giáo viên hướng dẫn.

- **Google Drive:** Được sử dụng để lưu trữ và chia sẻ tài liệu chung của nhóm như biên bản họp, tài liệu đặc tả yêu cầu, báo cáo và các file liên quan khác.
- **Zalo:** Được sử dụng làm kênh trao đổi nội bộ hàng ngày giữa các thành viên trong nhóm, phù hợp với việc liên lạc nhanh và thông báo kịp thời.
- **Trello:** Được sử dụng để theo dõi và quản lý danh sách bug trong quá trình kiểm thử, giúp nhóm nắm rõ trạng thái xử lý của từng lỗi.

4.2 Các giai đoạn của đề tài

Quá trình thực hiện đề tài được chia thành hai giai đoạn chính, với một khoảng thời gian nghỉ giữa hai giai đoạn để phục vụ thi cuối kỳ.

4.2.1 Giai đoạn 1 – Chuẩn bị ý tưởng và xây dựng nền tảng (Tháng 9 – 12/2025)

Giai đoạn đầu tiên diễn ra từ tháng 9 đến tháng 12 năm 2025, là giai đoạn khởi động của đề tài, tập trung vào việc hình thành ý tưởng, phân tích yêu cầu và xây dựng các thành phần cơ bản ban đầu.

- **Mục tiêu:** Xác định phạm vi và mục tiêu của đề tài; thu thập, phân tích yêu cầu hệ thống; thiết kế kiến trúc tổng thể và lựa chọn công nghệ phù hợp; thiết lập môi trường phát triển, cấu hình repository và các công cụ hỗ trợ; xây dựng các module cơ bản (xác thực người dùng, cấu trúc cơ sở dữ liệu, giao diện khung).

- **Kết quả:** Hoàn thiện bản đặc tả yêu cầu phần mềm, thiết kế cơ sở dữ liệu, sơ đồ kiến trúc hệ thống và triển khai được các chức năng nền tảng. Cơ sở hạ tầng dự án được thiết lập đầy đủ, sẵn sàng cho giai đoạn phát triển chính.
- **Bài học:** Việc dành đủ thời gian cho phân tích và thiết kế ở giai đoạn đầu giúp giảm thiểu đáng kể các thay đổi lớn về kiến trúc về sau; đồng thời cần thống nhất quy ước đặt tên, cấu trúc thư mục và quy trình làm việc ngay từ đầu.

Ghi chú: Sau giai đoạn 1, nhóm tạm dừng dự án trong tháng 1/2026 để tập trung cho kỳ thi cuối kỳ.

4.2.2 Giai đoạn 2 – Phát triển, hoàn thiện và kiểm thử (Tháng 2 – 7/2026)

Giai đoạn 2 là giai đoạn phát triển chính của đề tài, kéo dài từ tháng 2 đến tháng 7 năm 2026. Giai đoạn này được chia thành hai nửa với nhịp độ làm việc khác nhau.

Nửa đầu giai đoạn 2 (Tháng 2 – 3/2026)

- **Mục tiêu:** Tiếp tục phát triển các chức năng chính theo thứ tự ưu tiên đã xác định, đảm bảo nhịp độ đều đặn sau kỳ nghỉ.
- **Kết quả:** Hoàn thiện phần lớn các tính năng cốt lõi, hệ thống vận hành ổn định ở môi trường phát triển.
- **Bài học:** Cần duy trì kỷ luật làm việc sau kỳ nghỉ dài; việc họp hàng tuần giúp nhóm nhanh chóng lấy lại nhịp độ.

Nửa sau giai đoạn 2 (Tháng 4 – 7/2026)

- **Mục tiêu:** Tăng tốc độ phát triển, hoàn thiện các tính năng còn lại, tiến hành kiểm thử toàn diện và viết báo cáo đề tài.
- **Kết quả:** Hệ thống được hoàn thiện, kiểm thử và triển khai; báo cáo đề tài được hoàn chỉnh chuẩn bị cho buổi bảo vệ.
- **Bài học:** Việc tăng tần suất họp lên 2 lần/tuần từ tháng 4 đã giúp nhóm phát hiện và xử lý vấn đề nhanh hơn, đồng thời duy trì áp lực tiến độ cần thiết trong giai đoạn nước rút.

4.3 Quá trình làm việc nhóm

4.3.1 Vai trò của các thành viên

Nhóm gồm 4 thành viên với sự phân công vai trò rõ ràng. Vị trí nhóm trưởng được luân chuyển giữa các giai đoạn để phù hợp với tình hình thực tế: trong giai đoạn 1 và nửa đầu giai đoạn 2, bạn Trịnh Nguyên Lương đảm nhận vai trò nhóm trưởng; từ tháng 4/2026, bạn Trần Thảo Ngân tiếp nhận và dẫn dắt nhóm đến khi kết thúc dự án.

(Xem Bảng 4.1)

Về thiết kế giao diện, nhóm sử dụng Claude Design hỗ trợ tạo ra bản thiết kế ban đầu. Phía front-end web hoàn thiện và chỉnh sửa thiết kế, trong khi front-end mobile tham khảo để đảm bảo tính nhất quán về trải nghiệm người dùng.

4.3.2 Phân chia công việc và quản lý mã nguồn

Nhóm phân chia task theo độ phức tạp để giảm phụ thuộc giữa các thành viên: tính năng đơn giản/độc lập giao trọn cho một thành viên đảm nhiệm cả back-end lẫn front-end; tính năng cốt lõi hoặc phức tạp thì hai thành viên back-end xây dựng API trước, front-end tích hợp và hiển thị

Bảng 4.1: Phân công vai trò các thành viên trong nhóm

Thành viên	Vai trò chính	Mô tả
Trần Thảo Ngân	Front-end Web, Nhóm trưởng (từ T4/2026)	Phát triển ứng dụng web, hỗ trợ một số tác vụ back-end để đẩy nhanh tiến độ, thiết kế giao diện
Hoàng Lê Nam	Front-end Mobile	Phát triển ứng dụng di động, hỗ trợ một số tác vụ back-end
Nguyễn Thanh Nhã	Back-end, DevOps	Xây dựng API, quản lý server, triển khai hệ thống
Trịnh Nguyên Lương	Back-end, AI, Nhóm trưởng (GD1 – T3/2026)	Xây dựng API, tích hợp các tính năng AI; hai thành viên back-end đồng thời đảm nhận vai trò kiểm thử (tester)

sau. Với tính năng phức tạp, front-end thường thiết kế giao diện trước để back-end thiết kế API bám sát; hai thành viên back-end đảm nhận thêm vai trò tester trên cả web và mobile trước khi xác nhận hoàn thành. Giao tiếp nội bộ hàng ngày qua Zalo, với GVHD qua Messenger, họp định kỳ qua Google Meet.

Mã nguồn quản lý trên GitHub với 3 repository độc lập (back-end, front-end web, front-end mobile); không ai được commit trực tiếp lên `main`. Front-end push trực tiếp lên `dev`, chỉ merge lên `main` khi đã kiểm thử đầy đủ. Back-end theo branch-based workflow: tạo nhánh tính năng từ `dev`, rebase từ `origin/dev` trước khi tạo Pull Request, hai thành viên back-end review lẫn nhau trước khi merge.

Khi phát sinh lỗi, nhóm áp dụng nguyên tắc *ai làm lỗi, người đó fix*: lỗi front-end web do Trần Thảo Ngân xử lý, front-end mobile do Hoàng Lê Nam, back-end do Nguyễn Thanh Nhã và Trịnh Nguyên Lương (riêng lỗi liên quan AI do Trịnh Nguyên Lương phụ trách), sự cố server/hạ tầng do Nguyễn Thanh Nhã hotfix; kết quả xử lý được thông báo lại cho cả nhóm.

4.3.3 Lịch họp và theo dõi tiến độ

Nhóm áp dụng mô hình Scrum, sprint 2 tuần, ước tính công việc theo story point (1 điểm $\approx$ 4 giờ làm việc hiệu quả), mỗi thành viên cam kết tối thiểu 5 điểm/tuần; task lấy từ product backlog đã ưu tiên hóa, kết hợp feedback của GVHD và kết quả kiểm thử sprint trước, nhóm trưởng điều phối trong mỗi buổi họp. Mỗi ticket Jira đi qua 3 trạng thái To Do – In Progress – Done (Done ngay khi merge vào dev); kiểm thử tách thành giai đoạn riêng sau khi merge, do hai thành viên back-end đảm nhận, lỗi phát sinh ghi nhận lên Trello.

Tần suất họp tăng dần theo tiến độ dự án (Bảng 4.2).

Bảng 4.2: Lịch họp nhóm theo từng giai đoạn

Giai đoạn	Tần suất	Nội dung
Giai đoạn 1 (T9–12/2025)	2 tuần/lần, tối CN, 1–2 giờ	Thảo luận ý tưởng, xác định yêu cầu, thống nhất hướng đi
Giai đoạn 2, nửa đầu (T2–3/2026)	1 tuần/lần, tối CN, 30 phút	Báo cáo tiến độ cá nhân, theo dõi công việc, phân công task tiếp theo
Giai đoạn 2, nửa sau (T4–7/2026)	2 lần/tuần, tối CN & tối Thứ Tư	Tần suất dày hơn để đẩy nhanh tiến độ, xử lý sớm vấn đề phát sinh

4.4 Những khó khăn gặp phải

Trong suốt quá trình thực hiện đề tài, nhóm đã đối mặt với một số khó khăn nhất định ở nhiều khía cạnh khác nhau. Bảng 4.3 tổng hợp các khó khăn chính cùng hướng giải quyết mà nhóm đã áp dụng.

Bảng 4.3: Khó khăn và giải pháp trong quá trình thực hiện đề tài

Khó khăn	Nguyên nhân/Biểu hiện	Giải pháp
Thiết kế và phân tích yêu cầu	Phạm vi tính năng khó xác định ngay từ đầu, yêu cầu thay đổi trong quá trình thực hiện khiến một số chức năng phải thiết kế lại.	Dành nhiều thời gian hơn cho phân tích, thiết kế trước khi triển khai; trao đổi thường xuyên với GVHD để chốt yêu cầu sớm, giảm thay đổi lớn về sau.
Phân chia công việc	Một số task phụ thuộc lẫn nhau giữa back-end và front-end khiến một bên phải chờ API mới tích hợp được, ảnh hưởng tiến độ sprint.	Ưu tiên thiết kế API trước khi triển khai chi tiết; với tính năng đơn giản, giao hẳn một thành viên phụ trách toàn bộ để giảm phụ thuộc giữa hai phía.
Triển khai và vận hành	Gặp vấn đề về cấu hình server, biến môi trường và khác biệt giữa môi trường phát triển và sản xuất khi deploy.	Chuẩn hóa lại cấu hình môi trường, kiểm tra kỹ biến môi trường trước khi deploy; debug và ổn định dần hệ thống qua từng lần triển khai.
Kiểm thử	Không có tester chuyên biệt, hai thành viên back-end vừa phát triển vừa kiểm thử nên một số lỗi nhỏ bị bỏ sót ở giai đoạn đầu.	Tách kiểm thử thành giai đoạn riêng sau khi merge; ghi nhận lỗi lên Trello để theo dõi và xử lý, tránh bỏ sót.
Phối hợp nhóm	4 thành viên có lịch học, lịch làm việc khác nhau, dễ lệch pha tiến độ.	Điều chỉnh linh hoạt tần suất họp theo từng giai đoạn (tăng lên 2 lần/tuần ở giai đoạn nước rút) để đồng bộ tiến độ và xử lý sớm bất đồng.
Tích hợp AI	Đòi hỏi kiến thức chuyên sâu, khó đánh giá chất lượng đầu ra của mô hình và điều chỉnh prompt phù hợp cho từng bài toán cụ thể.	Thử nghiệm và tinh chỉnh prompt qua nhiều vòng lặp, so sánh chất lượng đầu ra thực tế để chọn cách tiếp cận phù hợp cho từng tính năng AI.

4.5 Demo và đánh giá ứng dụng

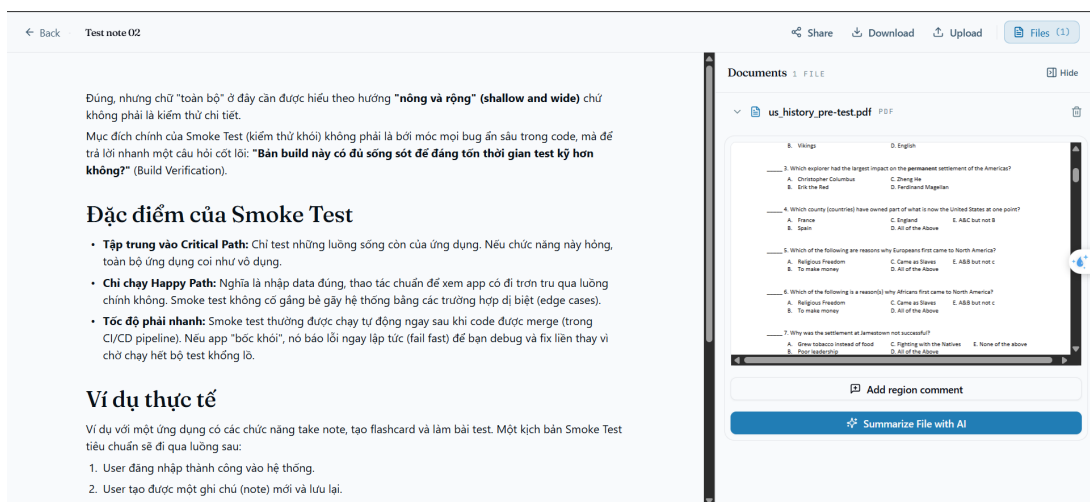
Nhóm đã xây dựng thành công ứng dụng ProLearning – một nền tảng hỗ trợ học tập đa chức năng, được hoàn thiện trên cả hai nền tảng: **ứng dụng Web** và **ứng dụng Mobile**.

Toàn bộ các chức năng chính đều được hỗ trợ đồng thời trên cả hai nền tảng Web và Mobile. Để tránh trùng lặp, phần demo dưới đây chỉ trình bày một số giao diện tiêu biểu trên Web và một số giao diện tiêu biểu trên

Mobile cho từng tính năng.

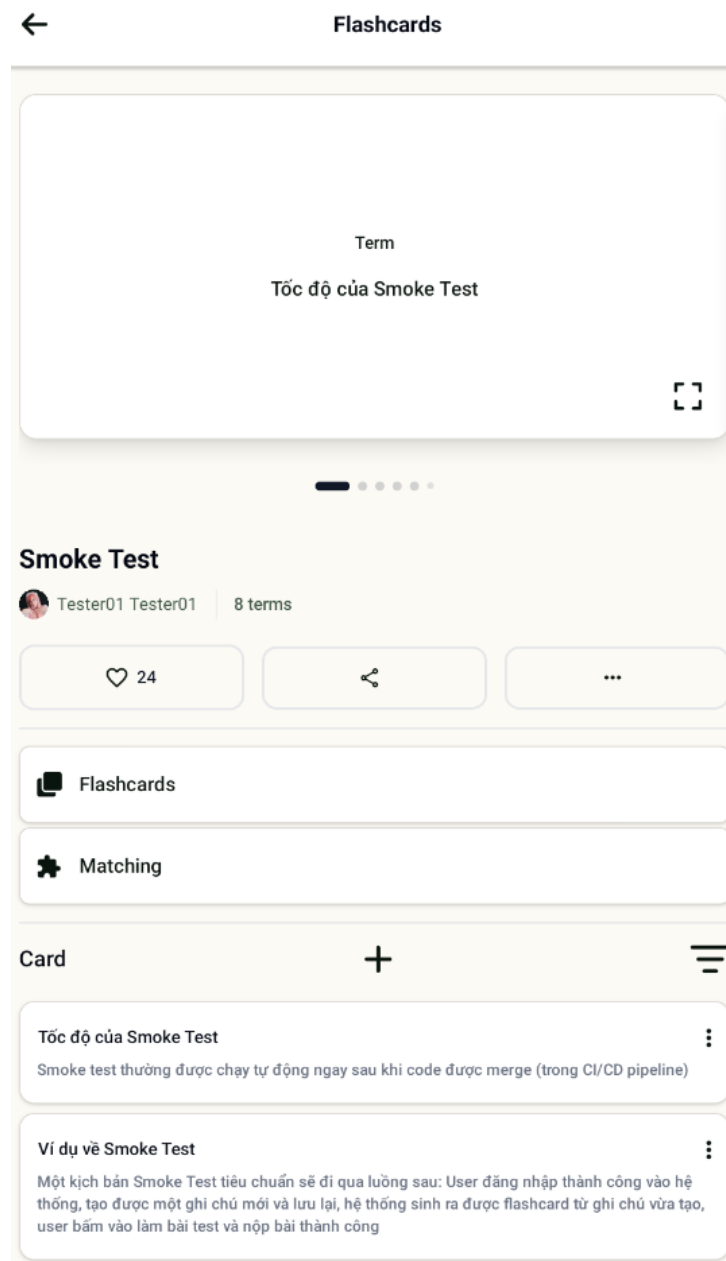
Dưới đây là các tính năng chính của hệ thống:

- **Ghi chép thông minh kết hợp tài liệu:** Người dùng có thể ghi chép thủ công đồng thời tải tài liệu lên hệ thống. Giao diện sẽ hiển thị song song nội dung tài liệu và vùng ghi chú. Hệ thống tích hợp AI để tự động trích xuất kiến thức từ tài liệu (Giải thích và tóm tắt nội dung từ tài liệu), giúp sinh viên mở rộng và hoàn thiện bài ghi chú ngay trên lớp.



Hình 4.1: Giao diện Web - Ghi chép kết hợp tài liệu

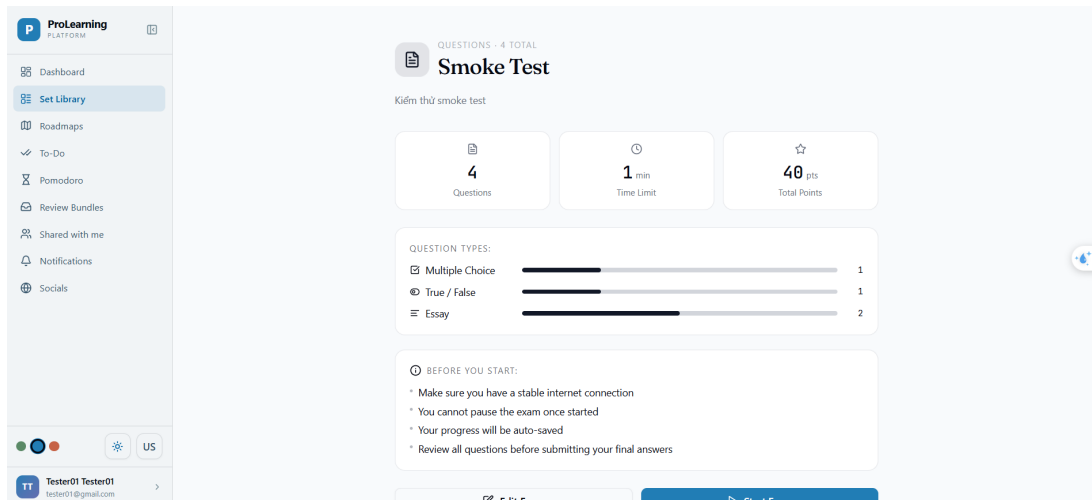
- **Tạo Flashcard (Thủ công hoặc AI):** Người học có thể tạo thủ công hoặc tải học liệu có sẵn để AI tự động sinh ra Flashcard (thẻ ghi nhớ, game ghép thẻ. Đồng thời hệ thống cũng có chế độ nhắc nhở người dùng ôn luyện những card chưa vững kiến thức.



Hình 4.2: Giao diện Mobile - Flashcard

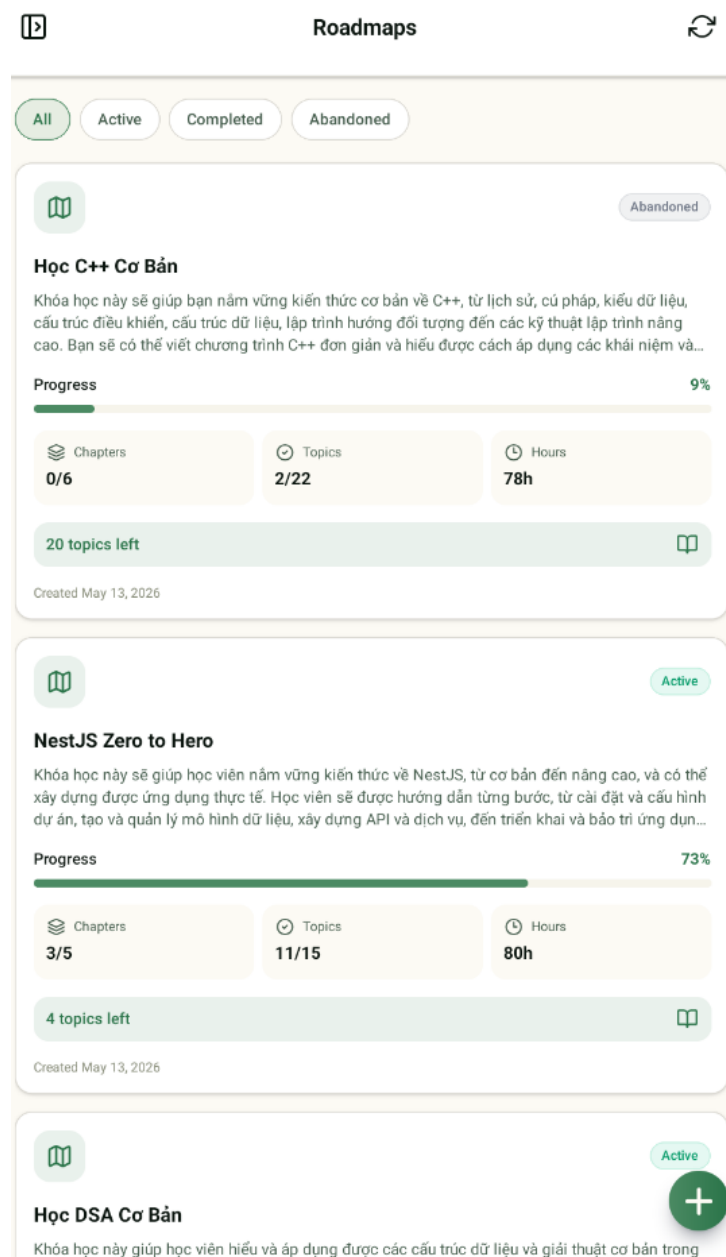
- **Tạo bài kiểm tra (Thủ công hoặc AI):** Hệ thống cho phép AI tự động chuyển đổi tài liệu học tập thành các bài kiểm tra đa dạng (trắc nghiệm, đúng/sai, điền câu trả lời). Tính năng này giúp đánh giá nhanh mức độ hiểu bài và cung cấp giải thích chi tiết cho từng câu trả lời. Đồng thời cũng có chức năng sinh ra bài kiểm tra dựa

trên format của bài kiểm tra có sẵn.



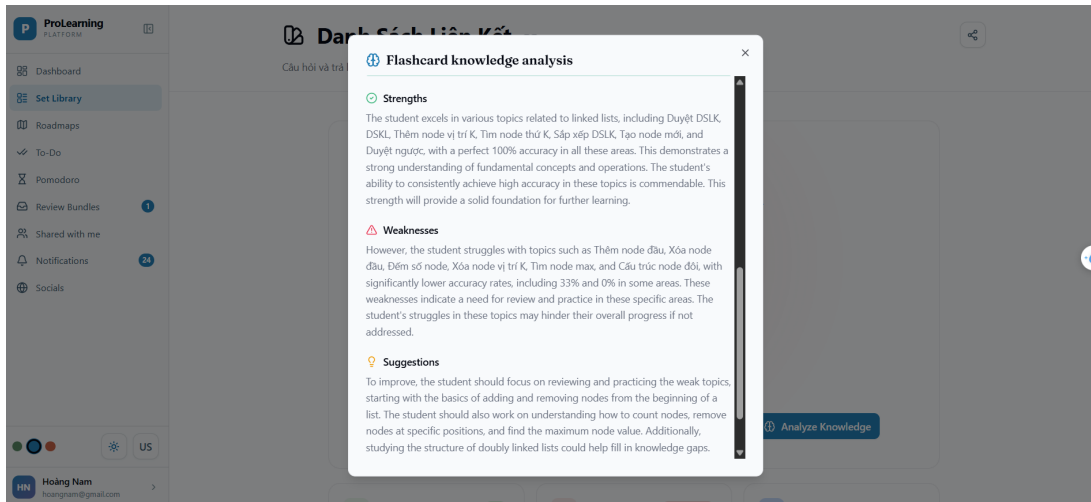
Hình 4.3: Giao diện Web - Bài kiểm tra

- **Xây dựng lộ trình học tập (Roadmap):** Dựa trên dữ liệu học tập và mục tiêu cá nhân, AI sẽ đề xuất một lộ trình học tập tối ưu. Roadmap giúp người dùng nắm bắt được các bước cần thực hiện để làm chủ kiến thức một cách khoa học.



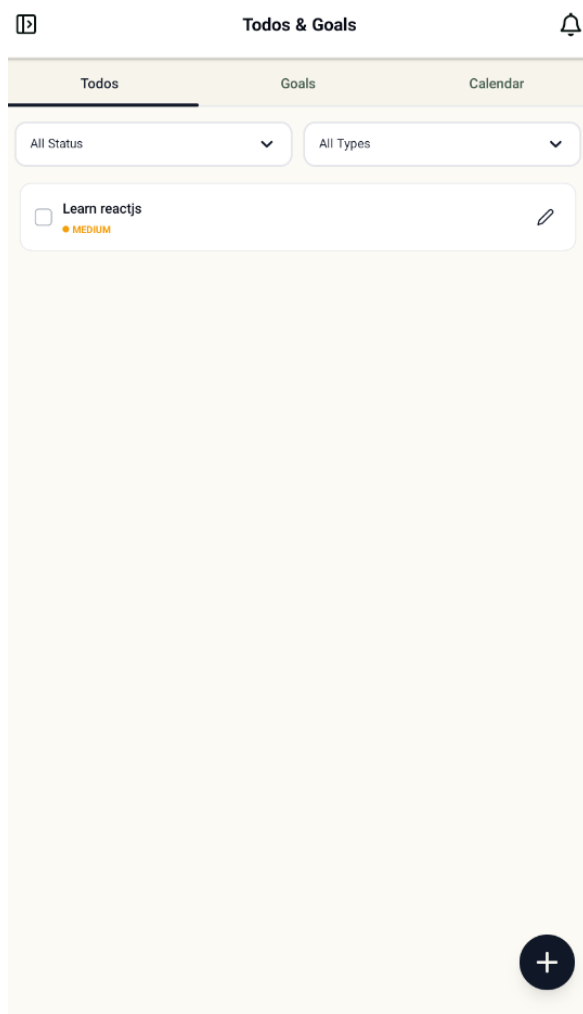
Hình 4.4: Giao diện Mobile - Lộ trình học tập

- **Phân tích năng lực người học:** Hệ thống có chức năng phân tích năng lực của người học ở mỗi Flashcard, Exam và toàn bộ Set đó. Đánh giá điểm mạnh, điểm yếu và đề xuất hướng cải thiện.

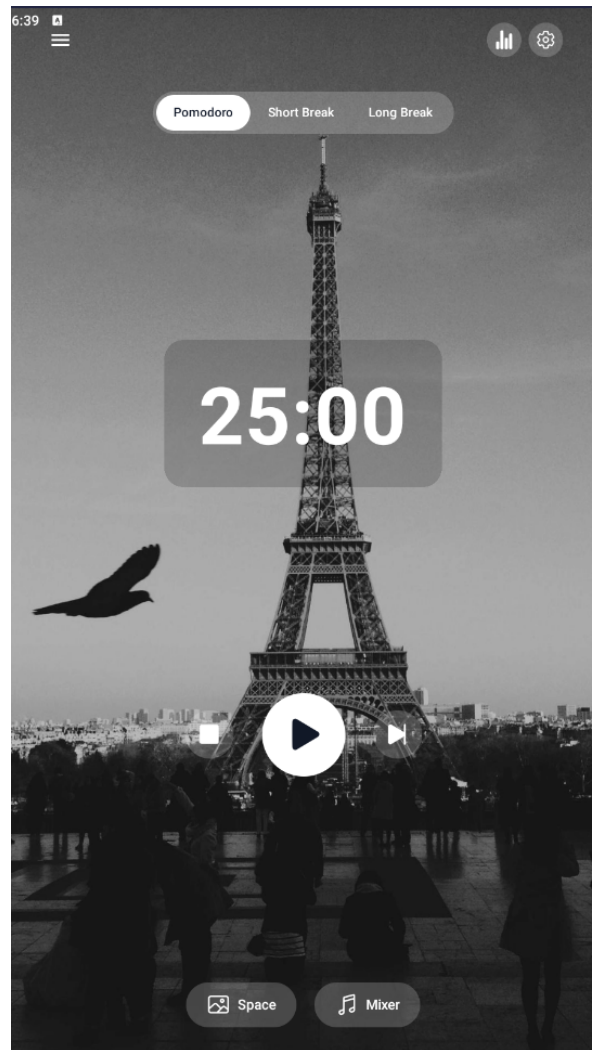


Hình 4.5: Giao diện Web - Phân tích năng lực người học

- **Quản lý Task và Trải nghiệm học:** Tích hợp các công cụ quản lý thời gian như To-do list và bộ đếm Pomodoro. Tính năng này giúp người học duy trì sự tập trung, theo dõi tiến độ công việc và cá nhân hóa trải nghiệm học tập hàng ngày.



Hình 4.6: Giao diện Mobile - Quản lý Task



Hình 4.7: Giao diện Mobile - Pomodoro

4.5.1 Đánh giá chức năng hệ thống

Để tránh việc tự đánh giá thiếu cơ sở, mức độ hoàn thiện của hệ thống được xác định dựa trên hai nguồn dữ liệu độc lập: (1) kết quả kiểm thử chức năng nội bộ theo từng nhóm chức năng, do nhóm phát triển thực hiện (kế hoạch và phạm vi chi tiết tại Mục 4.6); và (2) khảo sát trải nghiệm thực tế từ 22 người dùng thử nghiệm bên ngoài nhóm, thu thập qua Google Form và đánh giá trên thang điểm 1–5. Bảng 4.4 tổng hợp kết quả theo từng nhóm chức năng chính, đối chiếu cả hai nguồn.

Bảng 4.4: Đánh giá chức năng hệ thống theo nhóm, đối chiếu kiểm thử nội bộ và khảo sát người dùng

Nhóm chức năng	Mô tả	Kiểm thử nội bộ	Khảo sát người dùng (n ≤ 22)
Xác thực và phân quyền	Đăng ký, đăng nhập, JWT, kiểm soát quyền truy cập theo vai trò và quyền sở hữu tài nguyên.	138/138 TC đạt	Chưa khảo sát riêng
Workspace học tập (Note, Set, Flashcard, Exam)	Tổ chức học liệu, ghi chú kết hợp tài liệu và AI, tạo/ôn Flashcard, tạo bài kiểm tra.	180/180 TC đạt	Sets: 4.38/5 (n=21); Review Bundles: 4.35/5 (n=17)
AI learning (tóm tắt, sinh nội dung, phân tích, roadmap)	Sinh Flashcard/Exam bằng AI, phân tích năng lực, đề xuất lộ trình học tập cá nhân hóa.	150/150 TC đạt	AI Study Path: 4.05/5 (n=20); mức phù hợp lộ trình: 3.95/5 (n=19)
Cộng tác thời gian thực trên ghi chú	Đồng bộ nội dung, cursor và presence nhiều người dùng qua WebSocket/STOMP và Yjs.	30/30 TC đạt	Chưa khảo sát riêng
Công cụ hỗ trợ học tập (Todo/Goal, Calendar, Pomodoro)	Quản lý công việc, mục tiêu, đồng bộ Google Calendar, bộ đếm tập trung.	90/90 TC đạt	To-do/Goals: 4.60/5 (n=20); Pomodoro: 4.57/5 (n=21)
Thông báo và lịch trình nền	Thông báo trong ứng dụng, push notification, tác vụ nền theo lịch.	50/50 TC đạt	Chưa khảo sát riêng
Thanh toán và nâng cấp PRO	Tạo link thanh toán PayOS, xác minh webhook, kích hoạt tài khoản PRO.	Kiểm thử tích hợp thử công (Mục 4.6.4)	Sẵn sàng nâng cấp PRO: 3.68/5 (n=22)
Quản trị, kiểm duyệt nội dung & đồng bộ web/mobile	Quản trị, xử lý báo cáo nội dung; đồng bộ dữ liệu học tập, tài khoản và thông báo giữa web và mobile.	Kiểm thử thủ công trong quá trình phát triển, chưa đưa vào bộ đếm TC tổng hợp	Chưa khảo sát riêng

Ngoài đánh giá theo từng nhóm chức năng, khảo sát còn ghi nhận cảm nhận tổng thể về hệ thống: mức độ dễ sử dụng đạt 4.14/5, giao diện (UI/UX) rõ ràng đạt 4.00/5, tốc độ phản hồi đạt 4.23/5, và mức sẵn sàng giới thiệu app cho người khác đạt 4.27/5 (n=22 cho cả bốn tiêu chí).

Về lỗi phát sinh, 11/22 người dùng (50%) ghi nhận gặp lỗi nhỏ không ảnh hưởng nhiều đến trải nghiệm, 11/22 (50%) không gặp lỗi nào. Về mức độ sử dụng, Sets và To-do/Goals là hai nhóm chức năng được dùng nhiều nhất (17/22 người, 77%), kế đến là AI Study Path và Pomodoro Timer (14/22, 64%); Review Bundles có tỷ lệ sử dụng thấp nhất (8/22, 36%), phù hợp với việc đây là tính năng phụ thuộc vào Flashcard/Exam đã có sẵn.

Các điểm được người dùng đánh giá cao gồm khả năng tổng hợp nhiều công cụ học tập (ghi chú, flashcard, lộ trình AI) vào cùng một nền tảng thay vì phải chuyển đổi giữa nhiều ứng dụng, tính năng Flashcard hỗ trợ ghi nhớ hiệu quả, và Roadmap giúp định hướng lộ trình học rõ ràng, tránh lan man. Ở chiều ngược lại, khảo sát cũng chỉ ra một số hạn chế cụ thể: điểm phù hợp của lộ trình AI gợi ý (3.95/5) và mức sẵn sàng nâng cấp PRO (3.68/5) là hai tiêu chí thấp nhất, một số người dùng nhận xét Review Bundles khó hình dung cách dùng nếu chưa từng ôn Flashcard, giao diện tiếng Việt ở một vài mục còn lẫn tiếng Anh, cỡ chữ hơi nhỏ, và tính năng cộng đồng/mạng xã hội học tập vẫn còn phát sinh lỗi khi thao tác. Người dùng cũng đề xuất bổ sung chatbot hỏi đáp trực tiếp trong ứng dụng và không gian học nhóm thời gian thực qua liên kết chia sẻ — hai đề xuất này đã được nhóm ghi nhận vào định hướng phát triển tại Mục ??.

4.6 Kiểm thử

Phần này trình bày kế hoạch và kết quả kiểm thử của hệ thống Pro-Learning Platform.

4.6.1 Mục tiêu kiểm thử

Mục tiêu của quá trình kiểm thử là xác nhận hệ thống đáp ứng đúng các yêu cầu chức năng và phi chức năng đã đặt ra. Cụ thể, quá trình kiểm thử tập trung vào các mục tiêu sau:

- Đảm bảo các chức năng chính hoạt động đúng theo đặc tả yêu cầu.
- Đảm bảo dữ liệu người dùng được lưu trữ và đồng bộ chính xác giữa backend, web và mobile.
- Kiểm tra các luồng tích hợp với dịch vụ ngoài như AI Service, Firebase Cloud Messaging, Google Calendar và PayOS.
- Phát hiện lỗi giao diện, lỗi xử lý nghiệp vụ, lỗi phân quyền và lỗi phát sinh khi thao tác ở các trường hợp biên.
- Đánh giá mức độ ổn định của hệ thống trước khi triển khai và bàn giao.

4.6.2 Phạm vi kiểm thử

Phạm vi kiểm thử bao gồm các nhóm chức năng chính của hệ thống. Bảng 4.5 trình bày các nhóm chức năng cần kiểm thử và nội dung kiểm thử tương ứng.

Bảng 4.5: Phạm vi kiểm thử hệ thống

Nhóm chức năng	Nội dung kiểm thử
Xác thực và phân quyền	Đăng ký, đăng nhập, đăng xuất, refresh token, truy cập tài nguyên theo quyền sở hữu và vai trò.
Workspace học tập	Tạo, cập nhật, xóa và truy xuất note, file, flashcard set, exam và các dữ liệu học tập liên quan.
Ghi chú và cộng tác realtime	Chỉnh sửa note, đồng bộ nội dung, hiển thị cursor/presence, lưu và khôi phục snapshot.
AI learning	Sinh flashcard, sinh exam, phân tích kiến thức và tạo roadmap từ dữ liệu học tập.
Flashcard review	Lấy danh sách card đến hạn, cập nhật kết quả học, tính lịch ôn tiếp theo và trạng thái card.
Todo, Pomodoro và Calendar	Quản lý todo, Pomodoro session, đồng bộ Google Calendar và xử lý thay đổi trạng thái công việc.
Notification	Tạo thông báo, hiển thị inbox, đánh dấu đã đọc, gửi push notification và xử lý device token không hợp lệ.
Thanh toán và subscription	Tạo payment link, xác minh webhook PayOS, cập nhật giao dịch và kích hoạt tài khoản PRO.

4.6.3 Chiến lược kiểm thử

Nhóm áp dụng kết hợp nhiều hình thức kiểm thử nhằm bao phủ cả backend, web và mobile:

- **Kiểm thử chức năng:** Kiểm tra từng chức năng theo các luồng sử dụng chính và các trường hợp biên.
- **Kiểm thử API:** Gọi trực tiếp các endpoint backend để kiểm tra request, response, mã lỗi và phân quyền.
- **Kiểm thử tích hợp:** Kiểm tra các flow có nhiều thành phần phối hợp như AI Service, WebSocket, Google Calendar, FCM và PayOS.
- **Kiểm thử giao diện:** Kiểm tra khả năng hiển thị, điều hướng, nhập liệu và phản hồi lỗi trên web/mobile.
- **Kiểm thử hồi quy:** Sau khi sửa lỗi hoặc thêm tính năng mới, kiểm tra lại các luồng chính để tránh phát sinh lỗi cũ.

4.6.4 Kiểm thử tích hợp với dịch vụ ngoài

Các chức năng phụ thuộc dịch vụ ngoài cần được kiểm thử riêng vì có thể phát sinh lỗi do timeout, sai cấu hình, token hết hạn hoặc dữ liệu trả về không hợp lệ. Các trường hợp kiểm thử chính như sau:

- **AI Service:** Kiểm tra sinh flashcard, sinh exam, phân tích kiến thức và tạo roadmap với dữ liệu đầu vào hợp lệ/không hợp lệ.
- **Firebase Cloud Messaging:** Kiểm tra gửi push notification, cập nhật trạng thái gửi và xóa token không hợp lệ.
- **Google Calendar:** Kiểm tra OAuth callback, lưu credential, tạo/cập nhật/xóa event tương ứng với todo.

- **PayOS:** Kiểm tra tạo payment link, xác minh chữ ký webhook, xử lý webhook trùng lặp và kích hoạt PRO idempotent.
- **Cloudinary và dịch vụ lưu trữ file:** Kiểm tra upload, truy xuất và xóa file/tài nguyên liên quan nếu có.

4.6.5 Kết quả kiểm thử

Bảng 4.6 là tổng hợp kết quả kiểm thử theo từng nhóm chức năng chính của hệ thống.

Bảng 4.6: Tổng hợp kết quả kiểm thử

Nhóm chức năng	Số TC	Đạt	Không đạt	Ghi chú
Xác thực và phân quyền	138	138	0	<i>Dăng nhập/dăng ký và phân quyền hệ thống về cơ bản hoạt động tốt</i>
Workspace học tập	180	180	0	<i>Workspace học tập với các set, flashcard, exam cũng như roadmap hoạt động tốt</i>
Các tool hỗ trợ học tập	90	90	0	<i>Các tool hỗ trợ học tập gồm Pomodoro, Todo & Goal kết hợp Google Calendar hoạt động tốt</i>
AI learning	150	150	0	<i>Các chức năng AI hoạt động ổn định</i>
Realtime collaboration	30	30	0	<i>Tính năng cộng tác ghi chú và làm việc hoạt động bình thường</i>
Notification và scheduler	50	50	0	<i>Thông báo và nhắc nhở đầy đủ, không bị miss thông báo</i>

4.6.6 Đánh giá sau kiểm thử

Sau khi hoàn tất kiểm thử, nhóm tổng hợp các nhận xét chính về mức độ ổn định của hệ thống, các lỗi đã phát hiện và những hạn chế còn tồn tại. Kết quả đánh giá được trình bày trong Bảng 4.7.

Bảng 4.7: Đánh giá sau kiểm thử

Nội dung đánh giá	Nhận xét	Hướng xử lý/cải thiện
Các chức năng hoạt động ổn định	Các nhóm chức năng như xác thực và phân quyền người dùng, quản lý workspace, quản lý tài liệu học tập, Pomodoro, Roadmap và đồng bộ Google Calendar về cơ bản đáp ứng yêu cầu sử dụng.	Tiếp tục kiểm thử hồi quy khi bổ sung chức năng mới hoặc thay đổi API để đảm bảo các luồng chính không bị ảnh hưởng.
Các lỗi quan trọng đã phát hiện	Một số lỗi phát sinh ở các chức năng tích hợp với dịch vụ bên ngoài như Firebase Cloud Messaging, Google APIs hoặc các luồng phụ thuộc token/cấu hình môi trường.	Ghi nhận lỗi trên Trello, phân công thành viên phụ trách, kiểm tra lại cấu hình môi trường và bổ sung xử lý lỗi rõ ràng hơn ở backend/frontend.
Hạn chế còn tồn tại	Hệ thống còn phụ thuộc nhiều vào dịch vụ ngoài; bộ kiểm thử tự động chưa đầy đủ; chưa thực hiện sâu kiểm thử hiệu năng, kiểm thử tải và kiểm thử bảo mật.	Bổ sung automated test cho API và các luồng nghiệp vụ chính; xây dựng bộ regression test; lên kế hoạch kiểm thử performance/security trong các phiên bản tiếp theo.

Chương 5

Kết luận và hướng phát triển

5.1 Triển khai thực nghiệm và Mô hình tài chính

Nền tảng triển khai: Sản phẩm đã được triển khai hoàn thiện và người dùng có thể trực tiếp trải nghiệm hệ thống thông qua các kênh:

- **Trang Web:** Hệ thống được hosting trực tuyến tại địa chỉ: <https://prolearning.io.vn/>
- **App Mobile:** Do giới hạn về chi phí phát hành, bản thử nghiệm hiện đang được phân phối cho người dùng thông qua file **.apk**.

Mô hình tài chính: Hiện tại, nền tảng được cung cấp hoàn toàn miễn phí cho cộng đồng sinh viên nhằm thu thập phản hồi và tối ưu hóa hệ thống. Các chi phí liên quan đến duy trì máy chủ và gọi API từ mô hình AI đều do nhóm tự chi trả.

5.2 Đánh giá kết quả đề tài

Nhìn chung, đề tài đã hoàn thành các mục tiêu ban đầu đề ra. Nhóm đã thiết kế và xây dựng thành công **Pro Learning Platform** – một trợ

lý học tập toàn diện đa nền tảng (Web và Mobile), giải quyết triệt để các khó khăn của người học hiện nay thông qua các điểm nổi bật sau:

- Tích hợp toàn bộ công cụ học tập (Ghi chú, Flashcard, Bài kiểm tra) và quản lý thời gian (To-do list, Pomodoro) vào một không gian duy nhất.
- Ứng dụng thành công công nghệ AI để tự động hóa quá trình học: tóm tắt tài liệu, sinh thẻ ghi nhớ, tạo bài kiểm tra, nhắc nhở ôn luyện và phân tích năng lực người học.
- Xây dựng lộ trình học tập cá nhân hóa dựa trên dữ liệu phân tích năng lực và tần suất sai sót của người dùng.
- Bước đầu hình thành môi trường học tập cộng đồng thông qua các tính năng chia sẻ tài nguyên.

5.3 Kiến thức và kinh nghiệm đạt được

Trong suốt 11 tháng triển khai và phát triển dự án, các thành viên trong nhóm đã tích lũy được nhiều kiến thức và kỹ năng quý giá.

5.3.1 Về mặt kỹ thuật

- **Quản lý dự án và Thiết kế:** Sử dụng thành thạo Jira để quản lý dự án theo mô hình Agile/Scrum và Figma để thiết kế giao diện UI/UX.
- **Phát triển Frontend:** Nắm vững và áp dụng thành công React, TypeScript, Redux, Tanstack Query cho nền tảng Web và React Native cho nền tảng Mobile.

- **Thiết kế và kiến trúc Backend:** Xây dựng kiến trúc hệ thống bền vững với Java Spring Framework, sử dụng Redis để Caching và tích hợp Message Queue để xử lý các tác vụ bất đồng bộ.
- **Tích hợp AI:** Sử dụng Python và thư viện LangChain để kết nối và tương tác với các API của mô hình ngôn ngữ lớn (LLM API).
- **Cơ sở dữ liệu:** Có kinh nghiệm thiết kế ERD, vận hành PostgreSQL và sử dụng Cloudinary để lưu trữ tài nguyên đa phương tiện.
- **DevOps & Testing:** Thiết lập CI/CD thông qua GitHub Actions để tự động hóa quy trình triển khai.

5.3.2 Về kỹ năng mềm

- **Làm việc nhóm:** Phối hợp nhịp nhàng giữa 4 thành viên với các vai trò đa dạng (Frontend, Backend, AI, DevOps, UI/UX, Tester); hợp rà soát lỗi và phản hồi nhanh khi có yêu cầu thay đổi.
- **Kỹ năng lên kế hoạch:** Quản lý thời gian hiệu quả thông qua các Sprint 2 tuần, đảm bảo luôn có sản phẩm bàn giao đúng hạn.
- **Triển khai và vận hành:** Trải nghiệm thực tế quy trình đưa sản phẩm từ bản vẽ ý tưởng lên môi trường Production.

5.4 Những vấn đề còn tồn đọng

Mặc dù hệ thống đã hoạt động ổn định và đáp ứng các yêu cầu cơ bản, vẫn còn một số giới hạn và thách thức chưa được giải quyết triệt để:

- **Giới hạn xử lý tài liệu lớn:** Quá trình AI đọc và trích xuất dữ liệu từ các file lớn (hàng trăm trang PDF) đôi khi tốn nhiều thời gian hoặc xảy ra lỗi timeout do giới hạn hạ tầng server hiện tại.

- **Chi phí vận hành AI:** Phụ thuộc vào API của mô hình AI bên ngoài đòi hỏi chi phí duy trì khá lớn, đặc biệt khi lượng người dùng tăng đột biến.
- **Độ chính xác của AI:** Chức năng tạo Flashcard và Bài kiểm tra bằng AI đôi khi sinh ra đáp án nhiễu, chưa thực sự logic với ngữ cảnh tài liệu chuyên ngành sâu.
- **Đồng bộ hóa Offline:** Ứng dụng Mobile hiện yêu cầu kết nối Internet liên tục cho các tính năng AI; hỗ trợ học offline chưa được tối ưu hoàn toàn.

5.5 Hướng phát triển trong tương lai

Để hoàn thiện và nâng tầm Pro Learning Platform, nhóm đề xuất một số định hướng phát triển trong tương lai:

- **Phát triển Extension trình duyệt:** Xây dựng Chrome/Edge Extension để người dùng có thể quét, tóm tắt và lưu trữ kiến thức trực tiếp từ các trang web vào hệ thống.
- **Tích hợp Gamification:** Bổ sung hệ thống điểm thưởng, huy hiệu (badges) và bảng xếp hạng (Leaderboard) để kích thích hứng thú và tăng tính cạnh tranh tích cực giữa người học.
- **Phòng học nhóm ảo:** Phát triển tính năng Group Study Room cho phép nhiều người dùng cùng ghi chú trên một không gian chung theo thời gian thực.
- **Liên kết với hệ thống nhà trường (LMS):** Xây dựng API kết nối với các hệ thống quản lý đào tạo như Moodle hoặc Canvas, giúp sinh viên đồng bộ bài tập và tài liệu tự động.

- **Tối ưu hóa Local AI Model:** Nghiên cứu việc fine-tune các mô hình AI mã nguồn mở cỡ nhỏ (như Llama 3) để chạy trực tiếp trên server nội bộ, giảm chi phí và tăng bảo mật dữ liệu.

Tài liệu tham khảo

Tiếng Anh

- [1] Roediger, H. L. and Karpicke, J. D., “Test-enhanced learning: Taking memory tests improves long-term retention,” *Psychological Science*, vol. 17, no. 3, pp. 249–255, 2006. DOI: 10.1111/j.1467-9280.2006.01693.x
- [2] Dunlosky, J., Rawson, K. A., Marsh, E. J., Nathan, M. J., and Willingham, D. T., “Improving students’ learning with effective learning techniques: Promising directions from cognitive and educational psychology,” *Psychological Science in the Public Interest*, vol. 14, no. 1, pp. 4–58, 2013. DOI: 10.1177/1529100612453266
- [3] Google. “Notebooklm help,” Accessed: Jun. 7, 2026. [Online]. Available: <https://support.google.com/notebooklm/>
- [4] Google. “Use sources in notebooklm,” Accessed: Jun. 7, 2026. [Online]. Available: <https://support.google.com/notebooklm/answer/15338161>
- [5] Quizlet. “Online flashcard maker and flashcard app,” Accessed: Jun. 7, 2026. [Online]. Available: <https://quizlet.com/features/flashcards>
- [6] Quizlet. “Ai flashcard generator,” Accessed: Jun. 7, 2026. [Online]. Available: <https://quizlet.com/features/ai-flashcard-generator>
- [7] Notion. “Notion,” Accessed: Jun. 7, 2026. [Online]. Available: <https://www.notion.com/product>

- [8] Notion. “Notion ai,” Accessed: Jun. 7, 2026. [Online]. Available: <https://www.notion.com/product/ai>
- [9] Spring. “Spring boot reference documentation,” Accessed: Jun. 6, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/>
- [10] Spring. “Spring security oauth2 resource server jwt,” Accessed: Jun. 6, 2026. [Online]. Available: <https://docs.spring.io/spring-security/reference/6.5/servlet/oauth2/resource-server/jwt.html>
- [11] Spring. “Spring data jpa reference documentation,” Accessed: Jun. 6, 2026. [Online]. Available: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- [12] Spring. “Spring framework websocket/stomp,” Accessed: Jun. 6, 2026. [Online]. Available: <https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html>
- [13] PostgreSQL Global Development Group. “Postgresql about,” Accessed: Jun. 6, 2026. [Online]. Available: <https://www.postgresql.org/about/>
- [14] Redis. “What is redis?” Accessed: Jun. 6, 2026. [Online]. Available: <https://redis.io/tutorials/what-is-redis/>
- [15] RabbitMQ. “Rabbitmq tutorials and amqp concepts,” Accessed: Jun. 6, 2026. [Online]. Available: <https://www.rabbitmq.com/tutorials>
- [16] Cloudinary. “Cloudinary java sdk documentation,” Accessed: Jun. 6, 2026. [Online]. Available: https://cloudinary.com/documentation/java_integration
- [17] Firebase. “Firebase cloud messaging documentation,” Accessed: Jun. 6, 2026. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>

- [18] Google for Developers. “Google calendar api java quickstart,” Accessed: Jun. 6, 2026. [Online]. Available: <https://developers.google.com/calendar/api/quickstart/java>
- [19] PayOS. “Payos webhook documentation,” Accessed: Jun. 6, 2026. [Online]. Available: <https://docs.payos.money/core-concepts-webhooks>
- [20] Meta Platforms. “React native documentation,” Accessed: Jun. 6, 2026. [Online]. Available: <https://reactnative.dev/docs/getting-started>
- [21] Expo. “Expo documentation,” Accessed: Jun. 6, 2026. [Online]. Available: <https://docs.expo.dev/>
- [22] Meta Platforms. “React documentation,” Accessed: Jun. 6, 2026. [Online]. Available: <https://react.dev/>
- [23] VoidZero. “Vite documentation,” Accessed: Jun. 6, 2026. [Online]. Available: <https://vite.dev/guide/>
- [24] SuperMemo. “Algorithm sm-2,” Accessed: Jun. 7, 2026. [Online]. Available: <https://www.supermemo.com/en/archives1990-2015/english/ol/sm2>